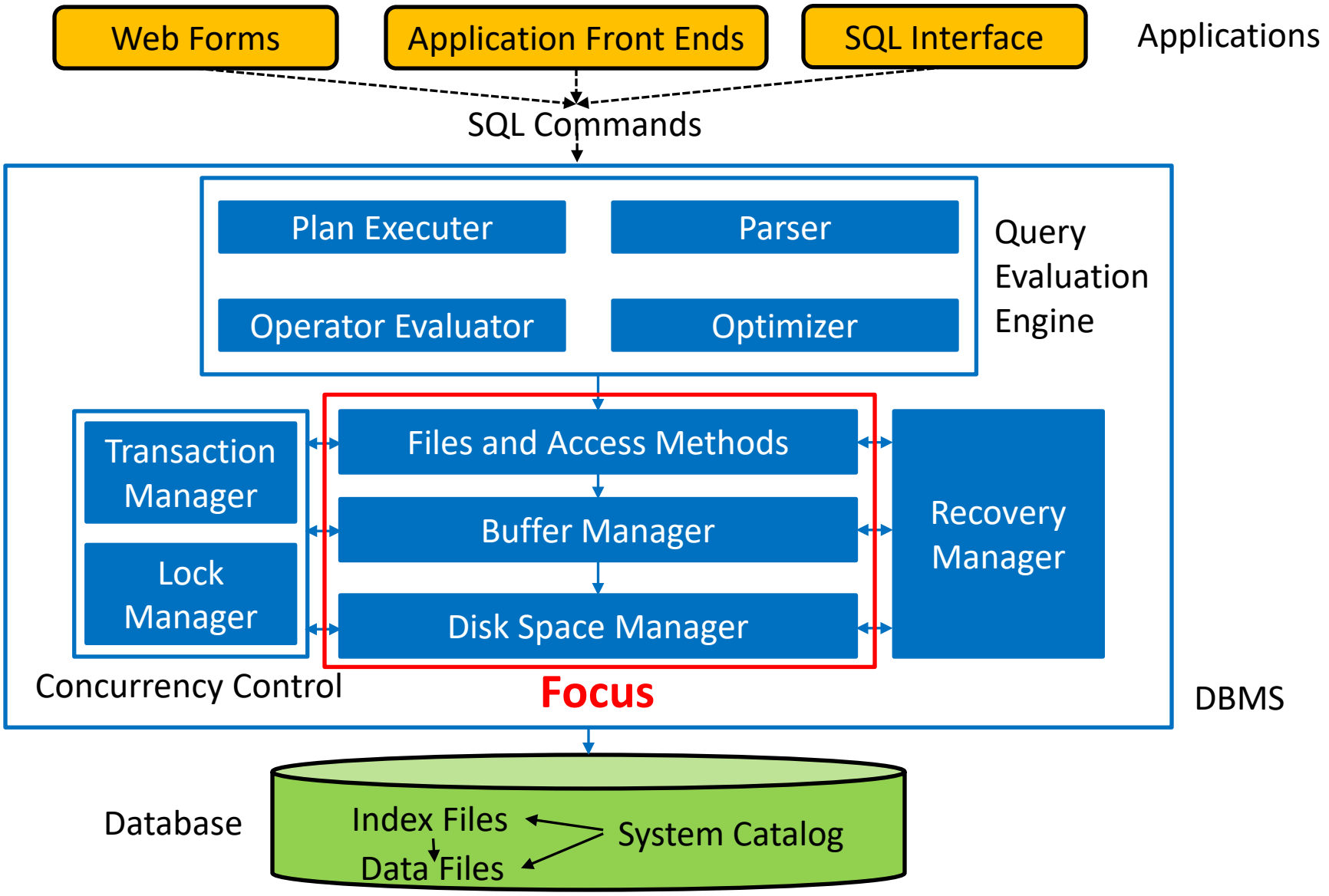
Lecture 5
Data Storage, File Organization and Indexing

Dr. Yu Yang
Department of Mathematics and Information Technology
The Education University of Hong Kong

All teaching and learning materials of INT6164 MUST NOT be distributed or shared without the permission of the Instructor!

The Architecture of a Relational DBMS

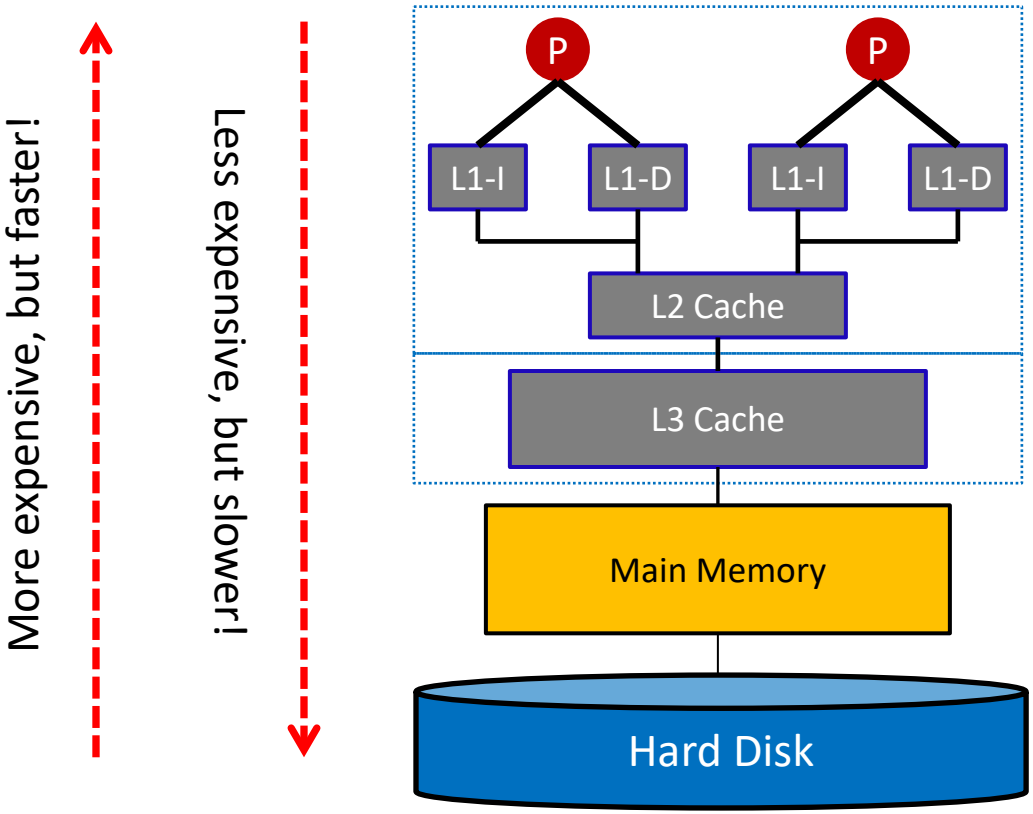


Outline

- Where Do DBMSs Store Data? ✓
- Reliability and Performance Implications of Multiple Disk Organization
- DBMS Disk Space Management
- File Organizations and Operations
- Why Indexing?
- Indexed Static Access Method (ISAM) Trees
- B+ Trees

Memory Hierarchy

- Storage devices play an important role in database systems
- How systems arrange storage?



2-3 GHz
(Executes 2-3 billion cycles* per second)

Size: 16KB-64KB
Access time: 2-4 Cycles

Size: 512KB-8MB
Access time: 6-15 Cycles

Size: 4MB-32MB
Access time: 30-50 Cycles

Size: 8GB-128GB
Access time: 600+ Cycles

Size: 160GB- 4TB
Access time: 1000s of times slower

* A "cycle" is the basic unit that measures a CPU's speed.

Where to Store Data?

- DBMSs can store and maintain *large amount* of data (what about Big Data?– as of now, we assume *centralized* DBMSs)

- Typically, buying enough memory to store all data is prohibitively expensive (let alone that classical memories are *volatile*)

- Thus, databases are usually stored on hard disks

But, Is Memory Gone?

- Data must be brought into memory to be processed!
  - READ: A requested page is transferred from disk to main memory (if not fetched earlier & found in memory)
    - The time to complete this transfer process is called **I/O time**
  - WRITE: Evicted pages are transferred from memory to disk
  - I/O time **dominates** the time taken for database operations!
  - To minimize I/O time, it is necessary to store and locate data ***strategically***

Queries and Transactions

- **Queries**: requests to the DBMS to retrieve data from the database (=READs)

- **Updates**: requests to the DMBS to insert, delete or modify/update existing data (=READs + WRITEs)

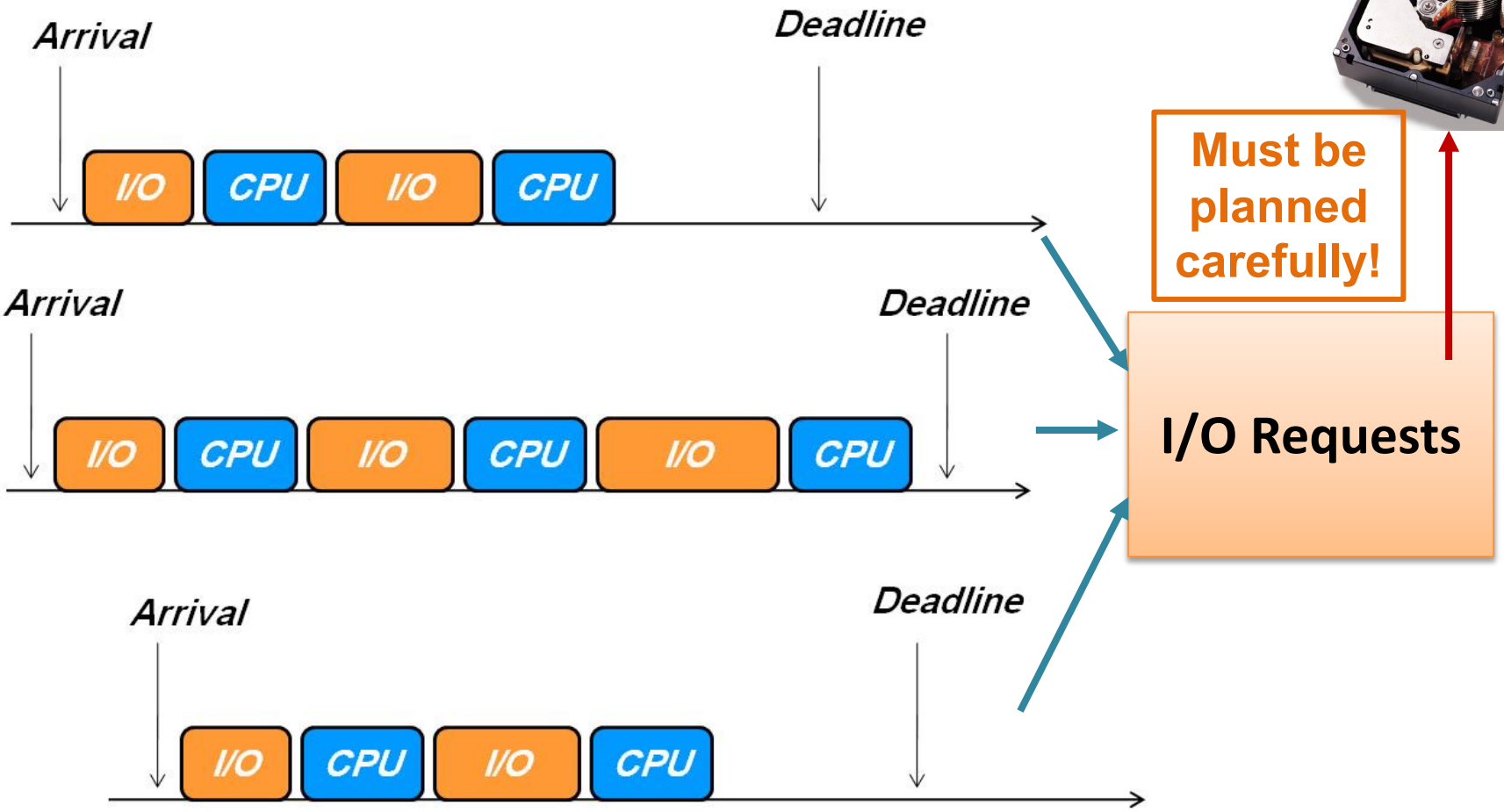
- **Transactions事务**: logical grouping of query and update requests to perform a task
  - Logical unit of work (like a function/subroutine)
  - Example: ATM withdraw

    x=read(balance), x=x-$200, write(balance, x)

Transaction I/Os

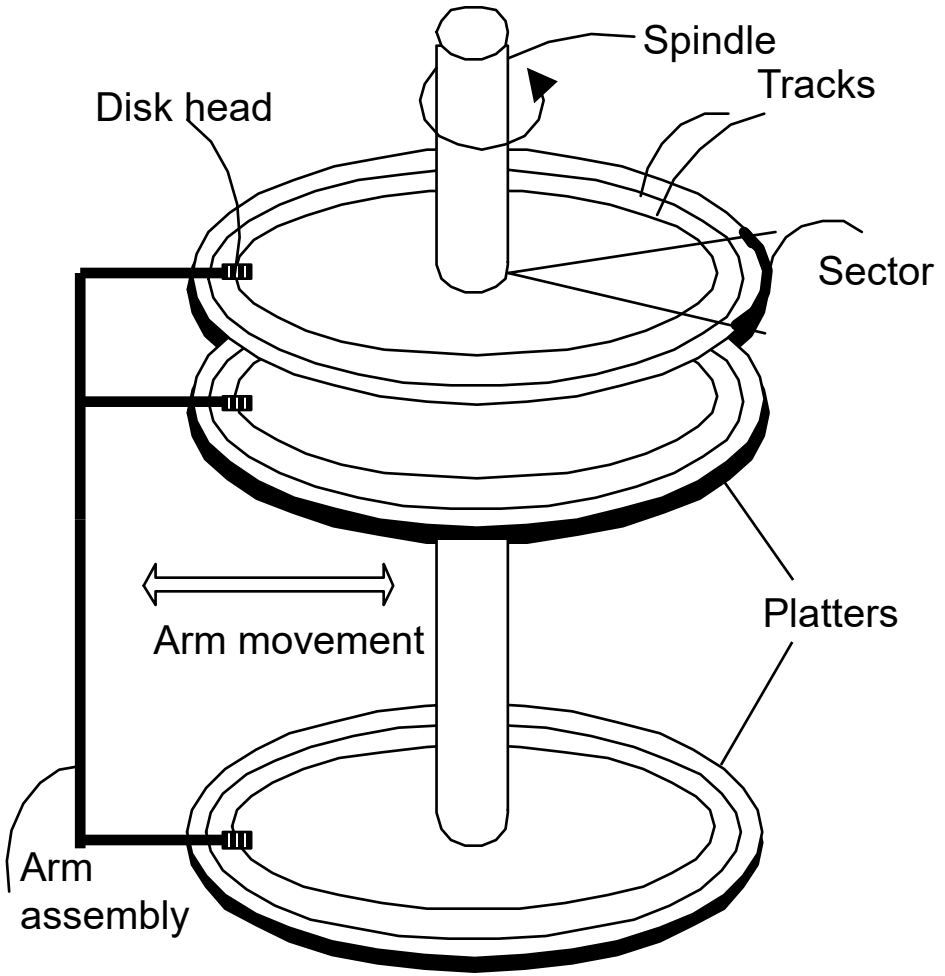
- I/O requests: read or write





Magnetic Disks

- Platters are arranged in concentric rings called tracks磁道
- Each track is divided into arcs called sectors扇区 (whose size is fixed)
- Data are stored in disk blocks磁盘块 (block size is a multiple of sector size)
- The set of all tracks with the same diameter is called cylinder柱面
- To read/write data, the arm assembly is moved in or out to position a head on a desired track



Accessing a Disk Block

- I/O time: Time to access (read/write) a disk block
  - **Seek Time寻道时间**: The time to move the disk heads to the track on which a desired block is located
  - **Rotational Time旋转时间**: The waiting time for the desired block to rotate under the disk head
  - **Transfer Time传输时间**: The time to actually read or write the data in the block once the head is positioned

- I/O time = seek time + rotational time + transfer time

Implications on DBMSs

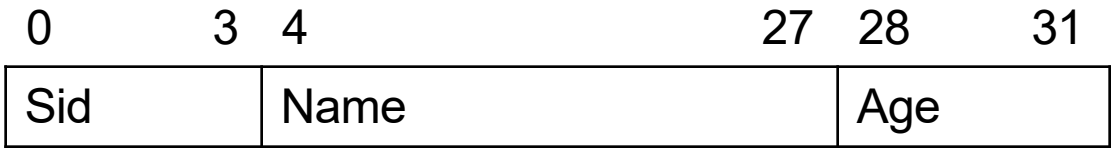
- Seek time and rotational delay dominate!
- Key to lower I/O cost: reduce seek/rotation delays!
- How to minimize seek and rotational delays?
  - Blocks on same track 块, followed by
  - Blocks on same cylinder 柱面, followed by
  - Blocks on adjacent cylinder
  - Hence, *sequential* arrangement of blocks of a file is a big win!

How is Data Stored on Disk?

Data Elements

- **Field**: a database attribute (sequence of bytes)
- **Record**: sequence of fields

```
CREATE TABLE Student (
        Sid INTEGER,
        Name CHAR(24),
        Age INTEGER);
```



- **Block**: sequence of records
- **File**: sequence of blocks

Blocks & Files

```
CREATE TABLE Student (
        Sid INTEGER,
        Name CHAR(24),
        Age INTEGER);
```

		SID	Name	Age
Block 0	Record 0	546007	Peter	18
	Record 1	546100	Bob	19
	Record 2	546107	Ann	21
	Record 3	546207	Jane	20
Block 1	Record 4	546240	John	24
	Record 5	546350	Ben	18
	Record 6	546420	Suzy	27
		546500	Peter	20



Block Header



- Block header includes:
  - Block ID
  - Role info such as data block, index block, etc.
  - Links to other blocks of the same table/file
  - Free-list
  - Timestamp

Unordered Files

- The simplest file structure: Records are stored in no particular order
- Also called: Heap, Pile, or Random File
- New records are inserted at the **end** of file
  - The last disk block is copied into buffer (i.e., memory)
  - New record is added
  - Block is rewritten back to disk
- New records can also be inserted in any file block with free space
- Record **insertion** is quite efficient

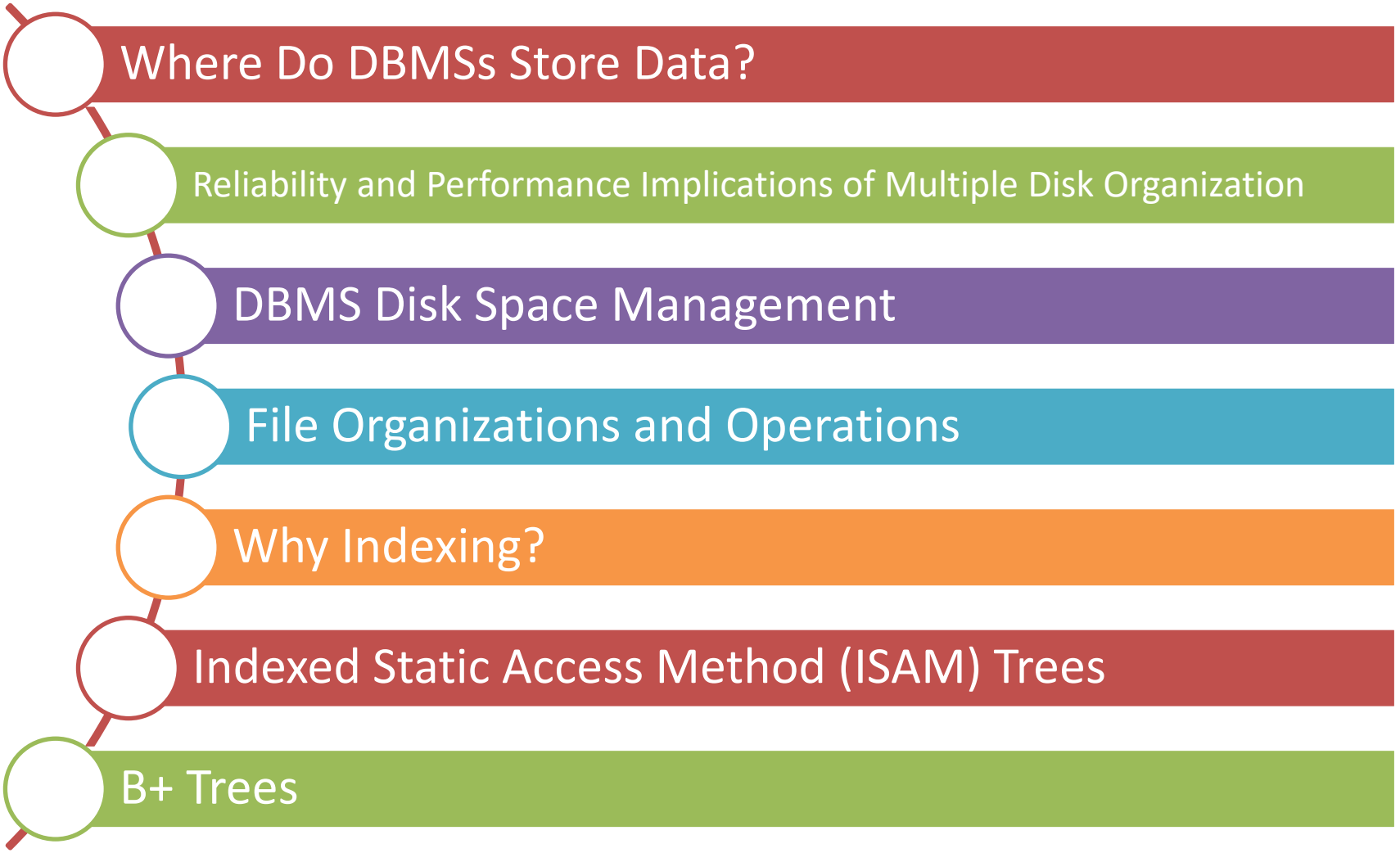


Unordered Files – Delete

- To delete a record:
  - Find its block
  - Copy the block into a buffer
  - Delete the record from the buffer
    - An extra bit is stored for each record (deletion marker)
    - Set the deletion marker to 1 (invalid record)
  - Rewrite the block back to disk

- What to do with the unused space?
  - Periodic reorganization: records are packed by removing deleted records, or
  - Insert new records in the empty space

Outline

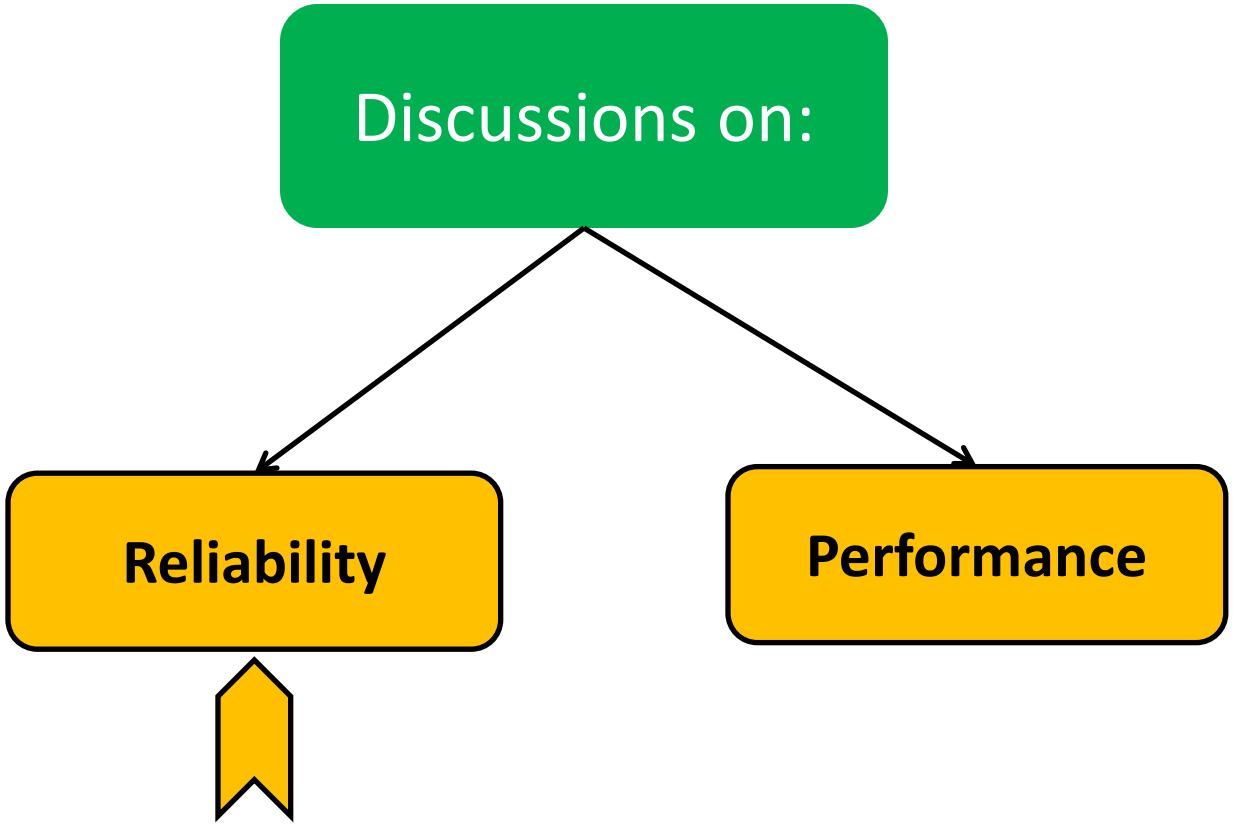

Where Do DBMSs Store Data?
Reliability and Performance Implications of Multiple Disk Organization
DBMS Disk Space Management
File Organizations and Operations
Why Indexing?
Indexed Static Access Method (ISAM) Trees
B+ Trees

Many Disks vs. One Disk

- Although disks provide cheap, non-volatile storage for DBMSs, they are usually bottlenecks for DBMSs
  - One factor to consider is ***reliability*** *可靠性*
  - Another factor to consider is ***performance*** *性能*
  - ***Capacity*** *容量* is something that can be considered as well

- How about adopting *multiple disks*?
  1. More data can be held as opposed to one disk — **Capacity**
  2. Data can be stored redundantly; hence, if one disk fails, data can be found on another — **Reliability**
  3. Data can be accessed concurrently — **Performance**

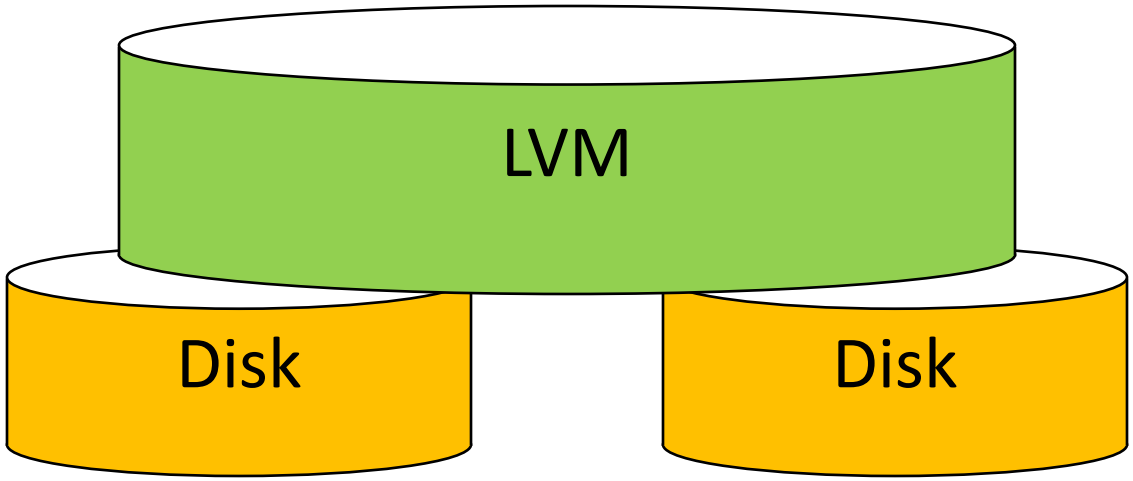
Multiple Disks



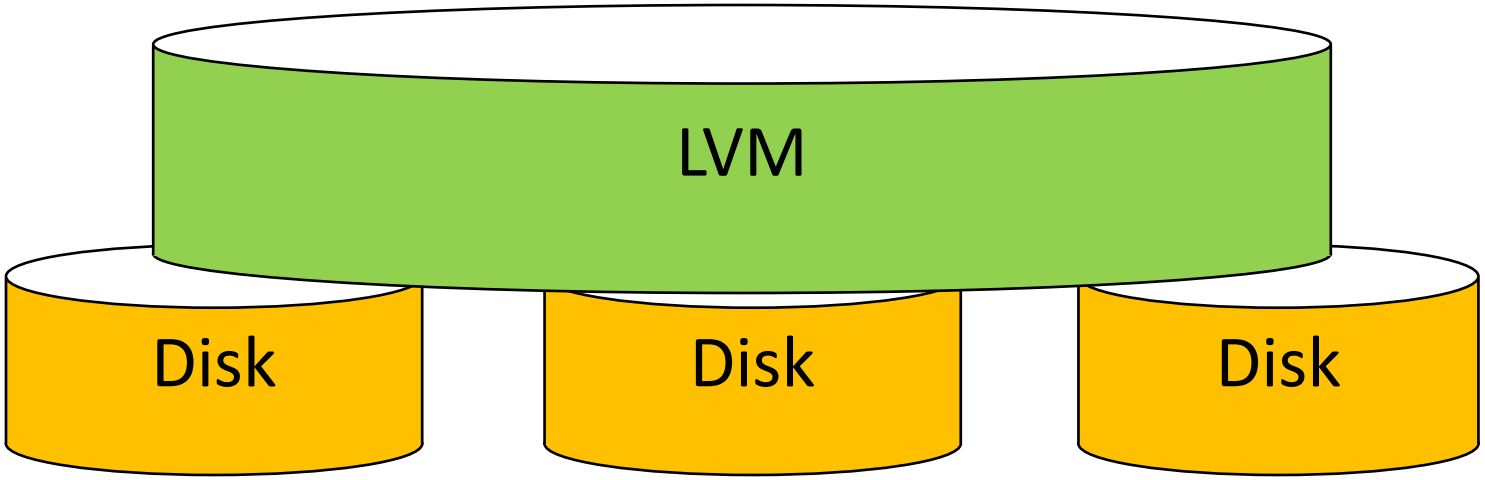
Logical Volume Managers (LVMs)

- Disk addresses used within a file system are assumed to refer to one particular disk (or sub-disk)

- What about providing an abstraction that makes a number of disks *appear* as one disk?



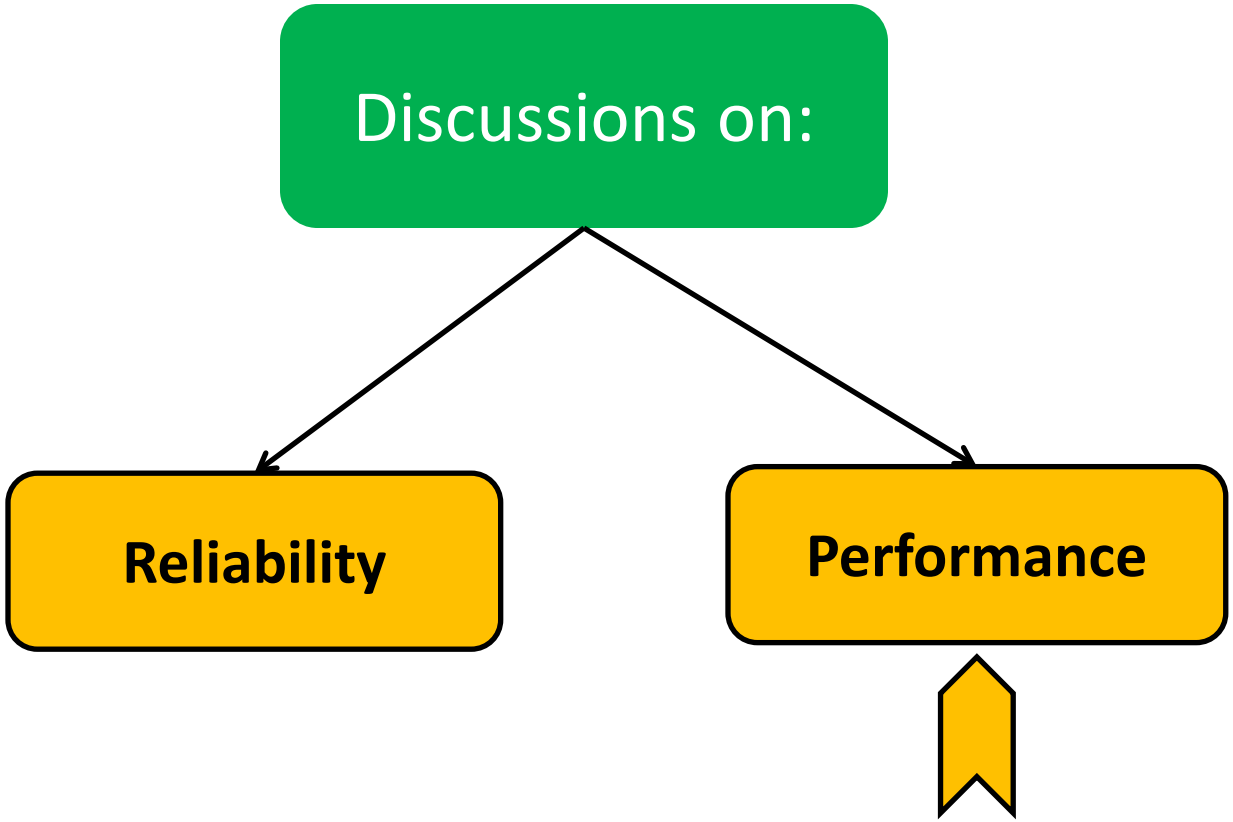
Logical Volume Managers (LVMs)



- What can LVMs do?
  - Spanning:
    - LVM transparently maps a *larger* address space to different disks
  - Mirroring:
    - Each disk can hold a separate, identical copy of data
    - LVM directs writes to the same block address on each disk
    - LVM directs a read to *any* disk (e.g., to the less busy one)

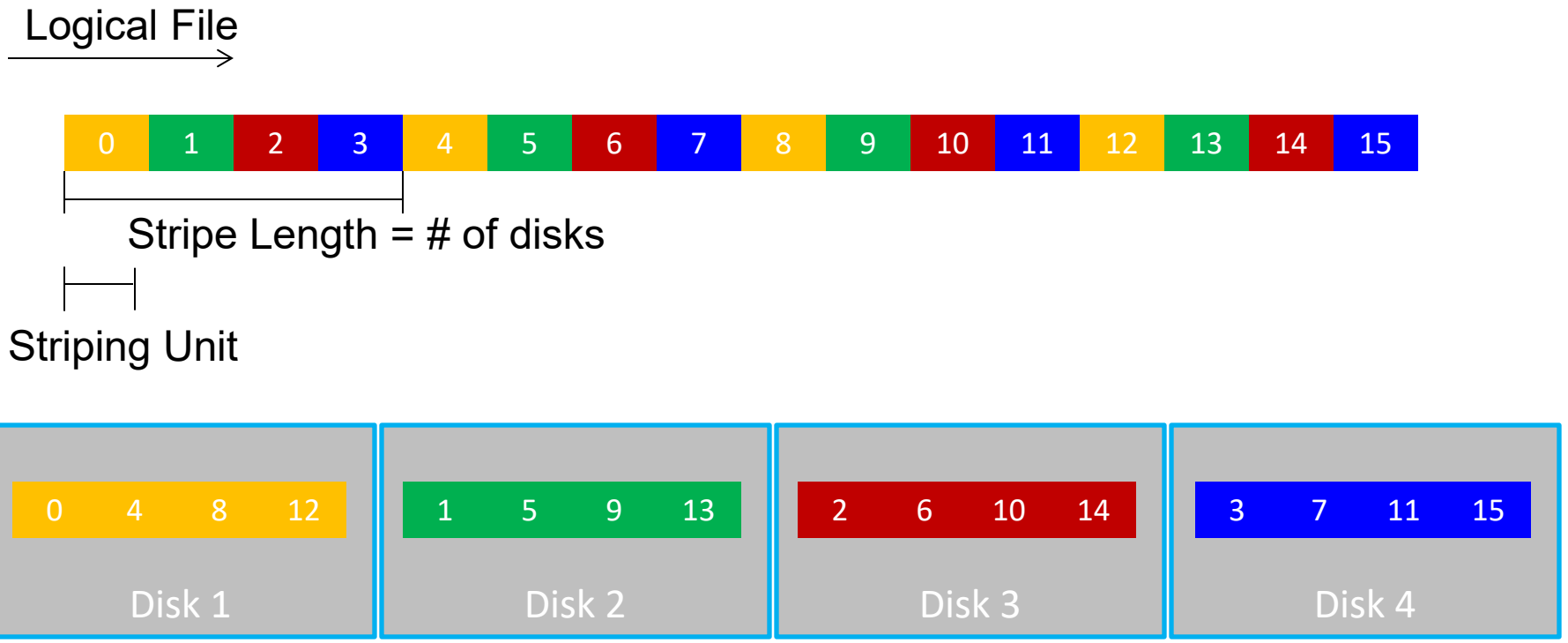
Mainly Provides Redundancy!

Multiple Disks

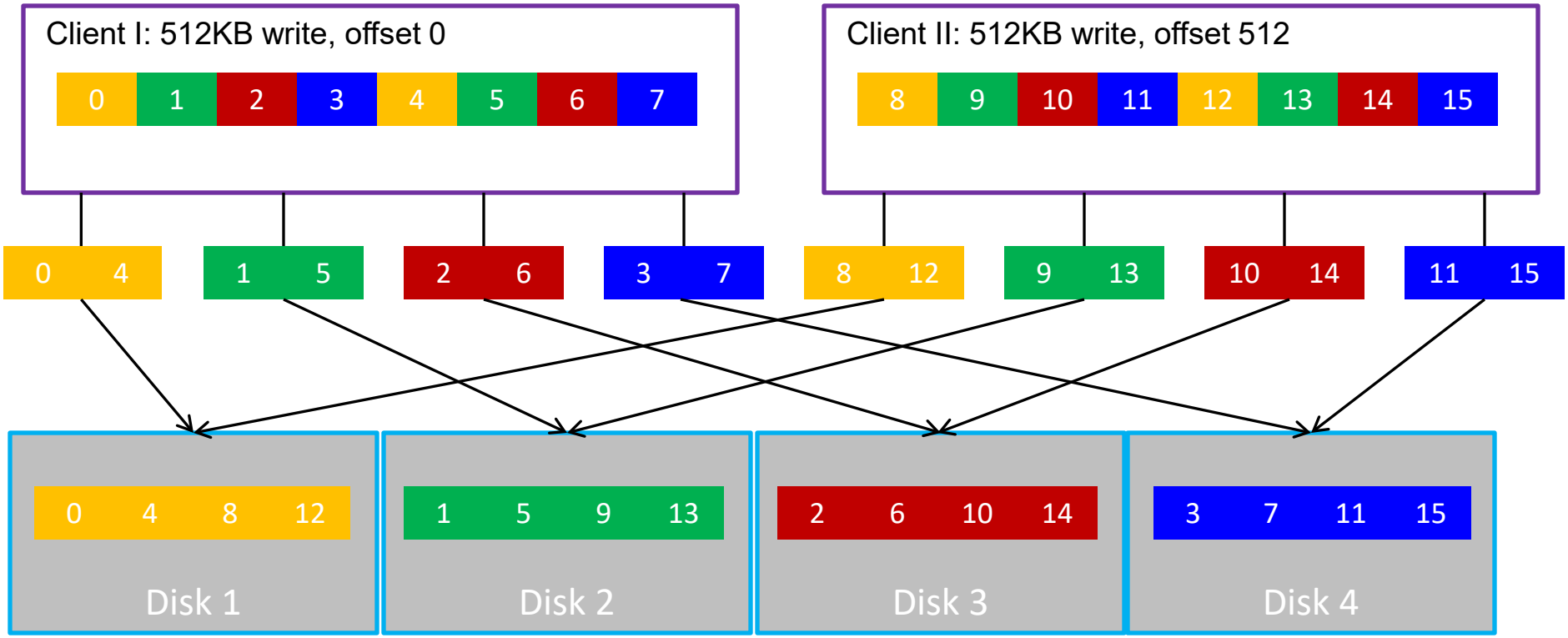


Data Striping 数据条带化

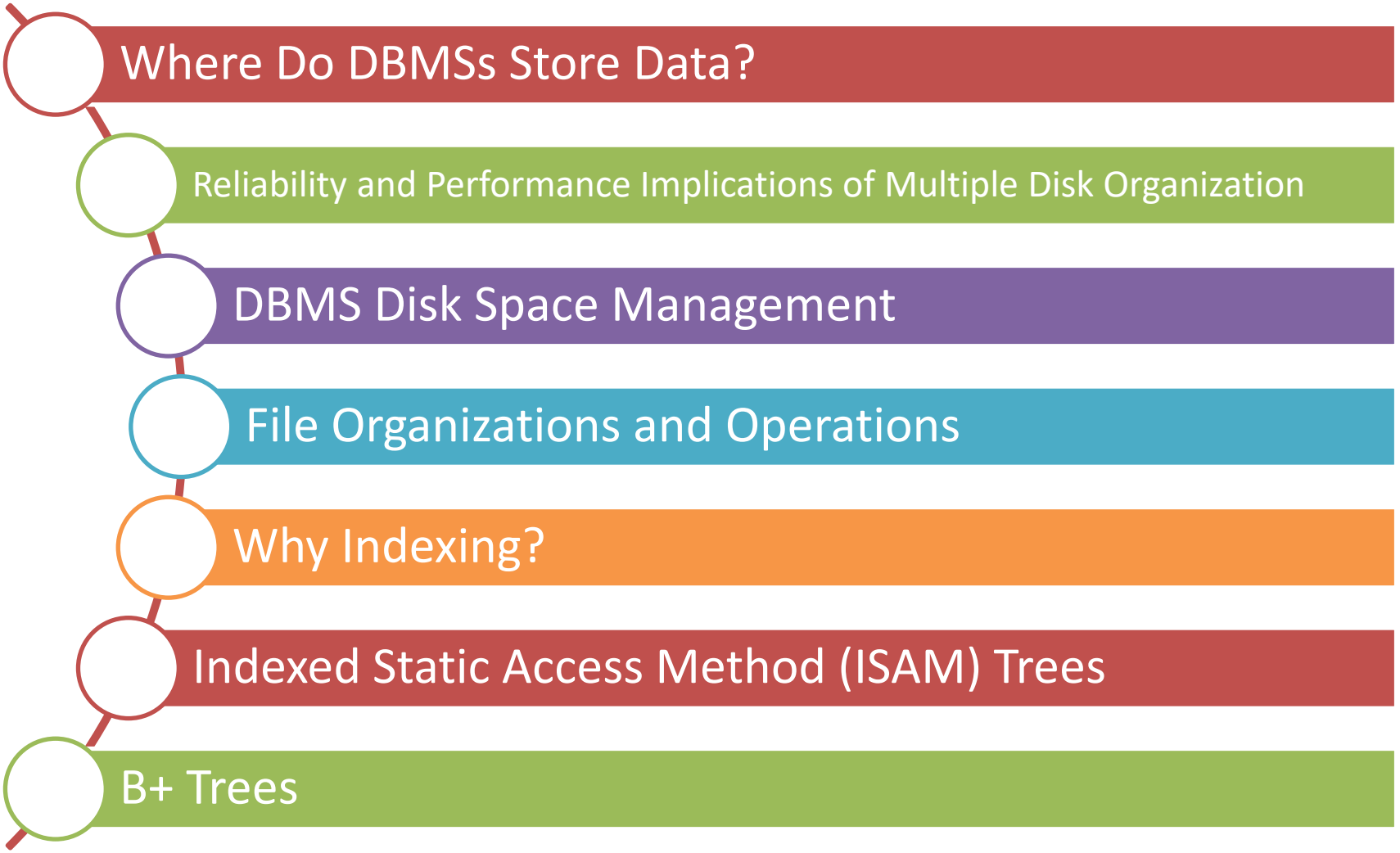
- LVM directs reads/writes to the same block address on each disk, which can be parallelized
- To achieve parallel accesses, we can use a technique called data striping



Data Striping



Outline


Where Do DBMSs Store Data?
Reliability and Performance Implications of Multiple Disk Organization
DBMS Disk Space Management
File Organizations and Operations
Why Indexing?
Indexed Static Access Method (ISAM) Trees
B+ Trees

DBMS Disk Space Management

- Support the concept of a page as a unit of data
  - Page size is usually chosen to be equal to the block size so that reading or writing a page can be done in 1 disk I/O
- Allocate/de-allocate pages as a *contiguous* sequence of blocks on disks
- Abstracts hardware (and possibly OS) details from higher DBMS levels

What to Keep Track of?

- The DBMS disk space manager keeps track of:
  - Which pages are on which disk blocks
  - Which disk blocks are in use (i.e., occupied by data blocks)

- Blocks can be initially allocated contiguously, but allocating and de-allocating blocks usually create *"holes"*

- Hence, a mechanism to keep track of *free blocks* is needed
  - A list of free blocks can be maintained (*storage could be an issue*)
  - Alternatively, a bitmap 位图 with one bit per each disk block can be maintained (*more storage efficient and faster in identifying contiguous free areas!*)

OS File Systems vs. DBMS Disk Space Managers

- Operating Systems already employ disk space managers using *their* "file" abstraction
  - "Read byte ***i*** of file ***f***" ➔ "read block ***m*** of track ***t*** of cylinder ***c*** of disk ***d***"

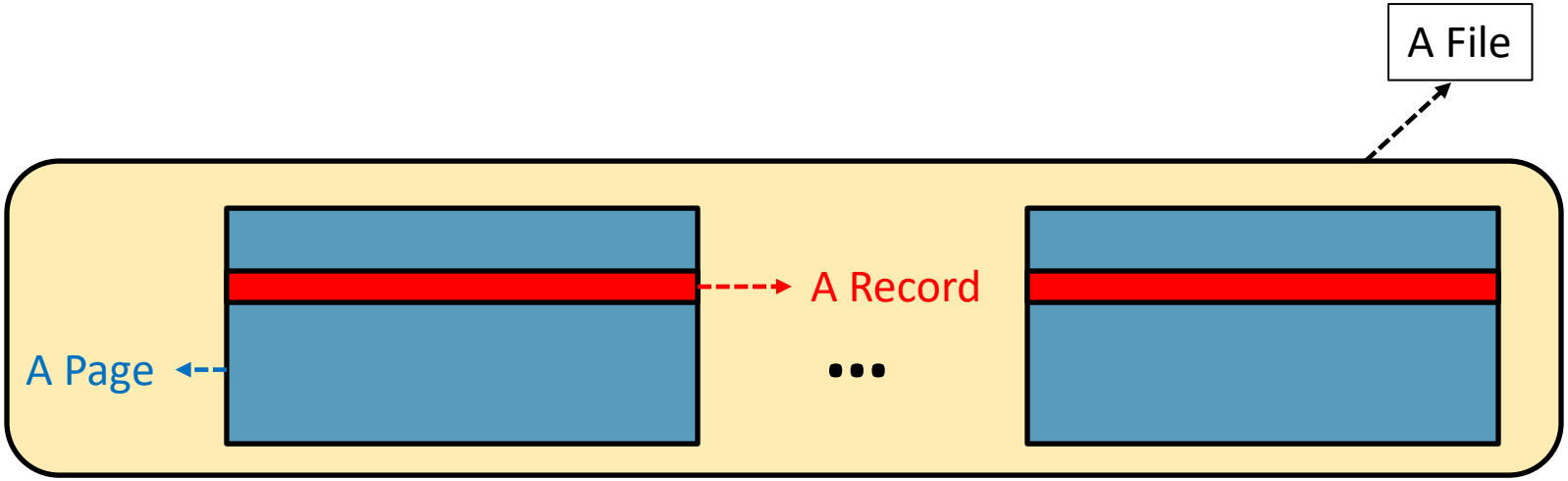
- DBMSs disk space managers usually pursue their own disk management without relying on OS file systems
  - Enables portability
  - Can address larger amounts of data
  - Allows *spanning* and *mirroring*

Outline

- Where Do DBMSs Store Data?
- Reliability and Performance Implications of Multiple Disk Organization
- DBMS Disk Space Management
- File Organizations and Operations ✓
- Why Indexing?
- Indexed Static Access Method (ISAM) Trees
- B+ Trees

Recap: Records, Pages and Files

- Higher-levels of DBMSs deal with records (not pages!)
- At lower-levels, records are stored in pages
- But, a page might not fit all records of a database
  - Hence, multiple pages might be needed
- A collection of pages is denoted as a file



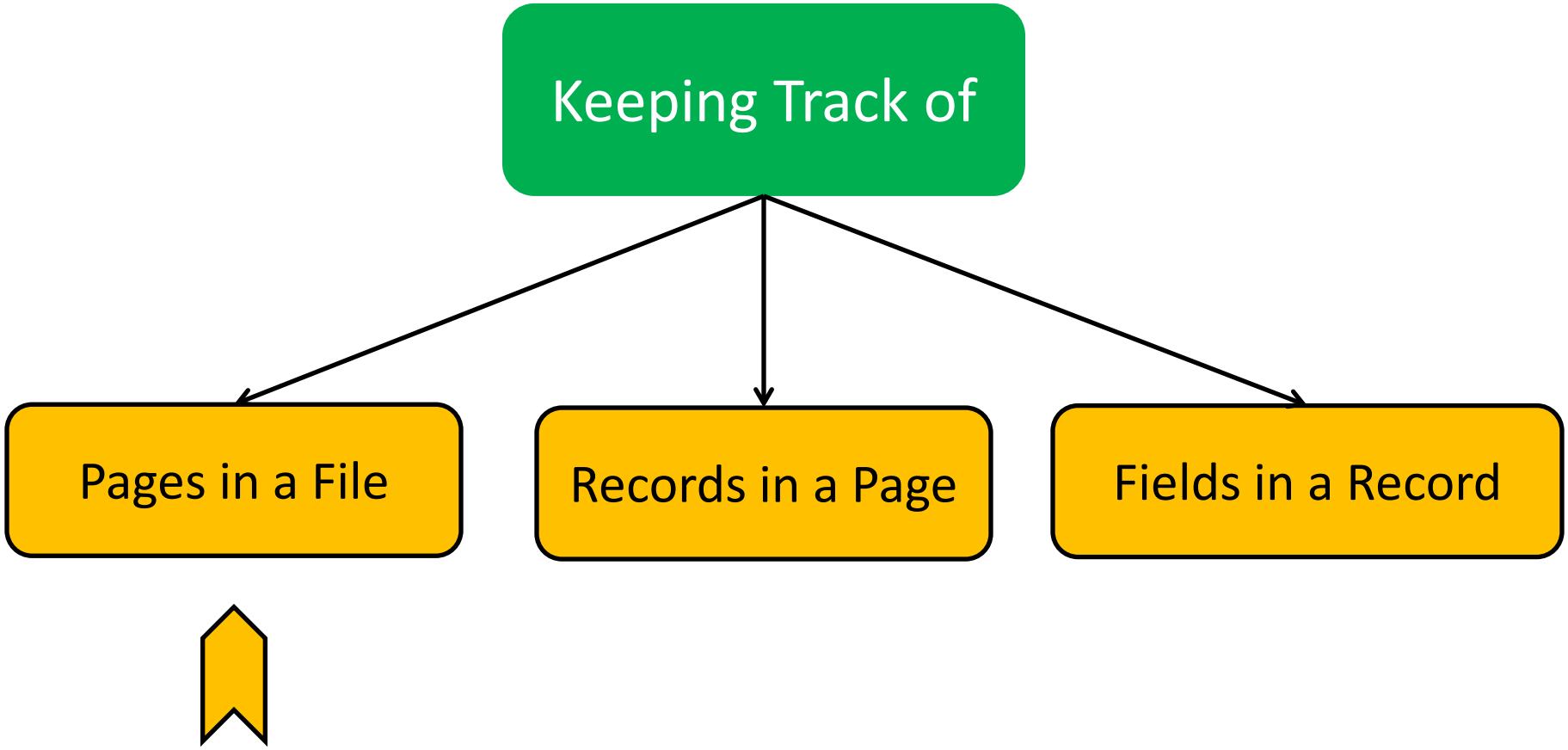
File Operations and Organizations

- A file is a collection of pages, each containing a collection of records

- Files must support operations like:
  - Insert/Delete/Modify records
  - Read a particular record (specified using a record id)
  - Scan all records (possibly with some conditions on the records to be retrieved)

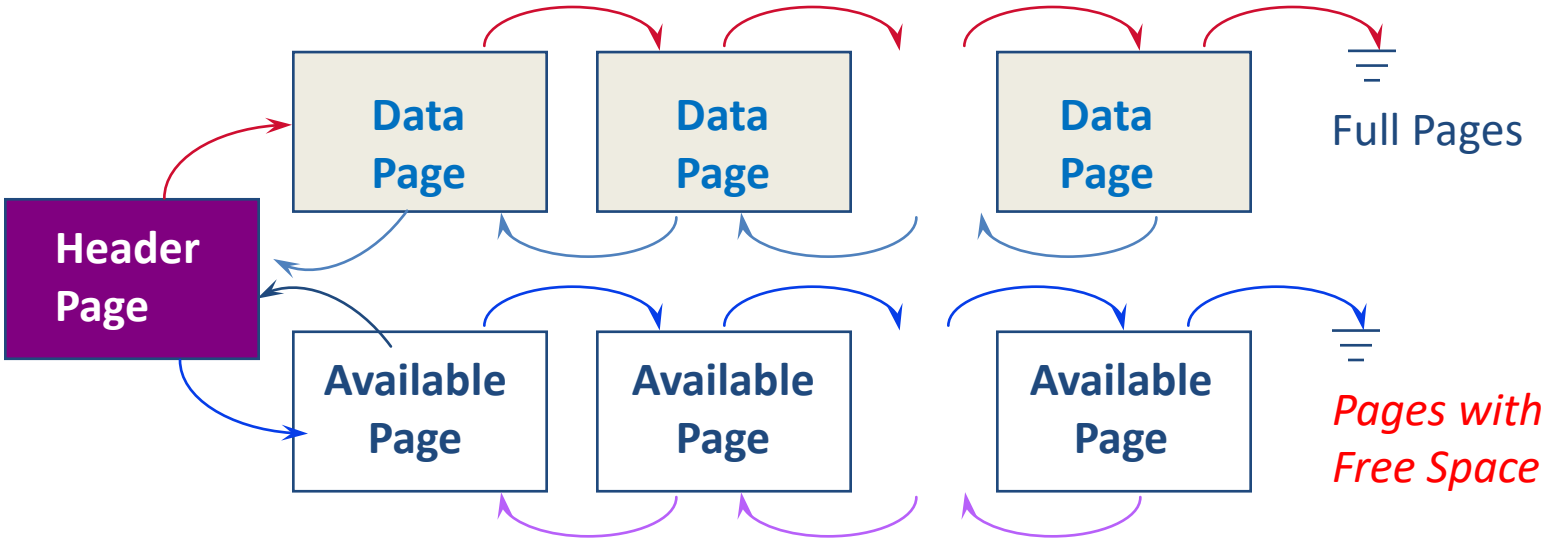
- There are several organizations of files:
  - Unordered (Heap)
  - Sorted
  - Indexed

Supporting Record Level Operations



Unordered Files Using *Lists* of Pages

- An unordered file can be organized as a *doubly linked list* of pages



- The Header Page (i.e., <*heap_file_name*, *page_1_addr*> is stored in a known location on disk

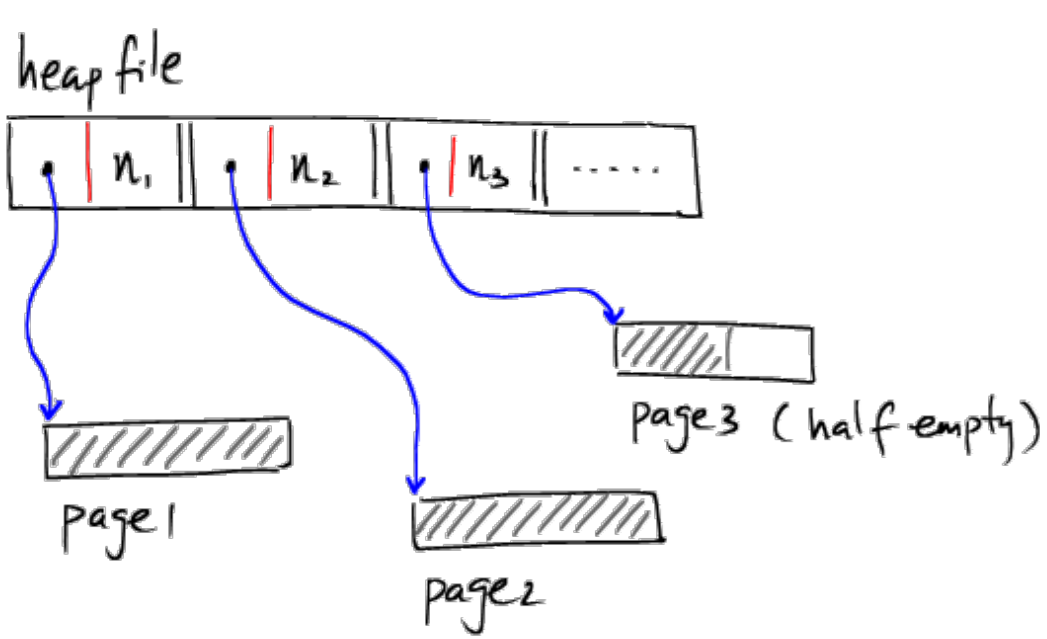
- Each page contains 2 ‘pointers’ plus data

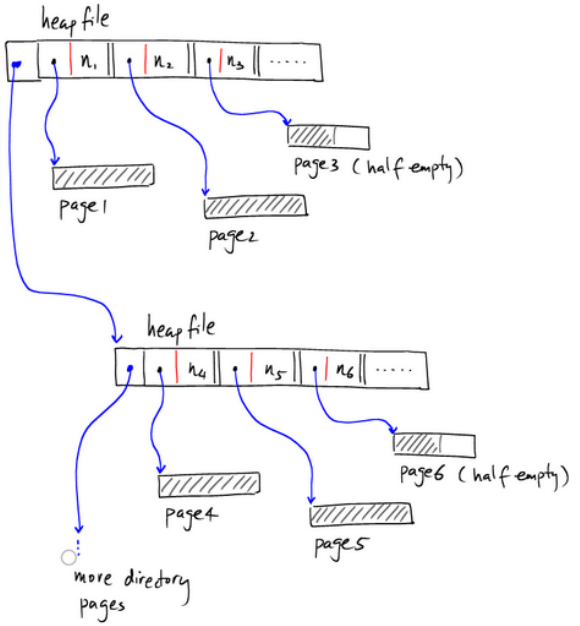
Unordered Files Using *Lists* of Pages

- It is likely that every page has at least a few free bytes
- Thus, virtually all pages in a file will be on the free list!
- To insert a typical record, we must retrieve and examine several pages on the free list before one with *enough* free space is found
- This problem can be addressed using an alternative design known as the directory-based heap file organization 基于目录的堆文件组织

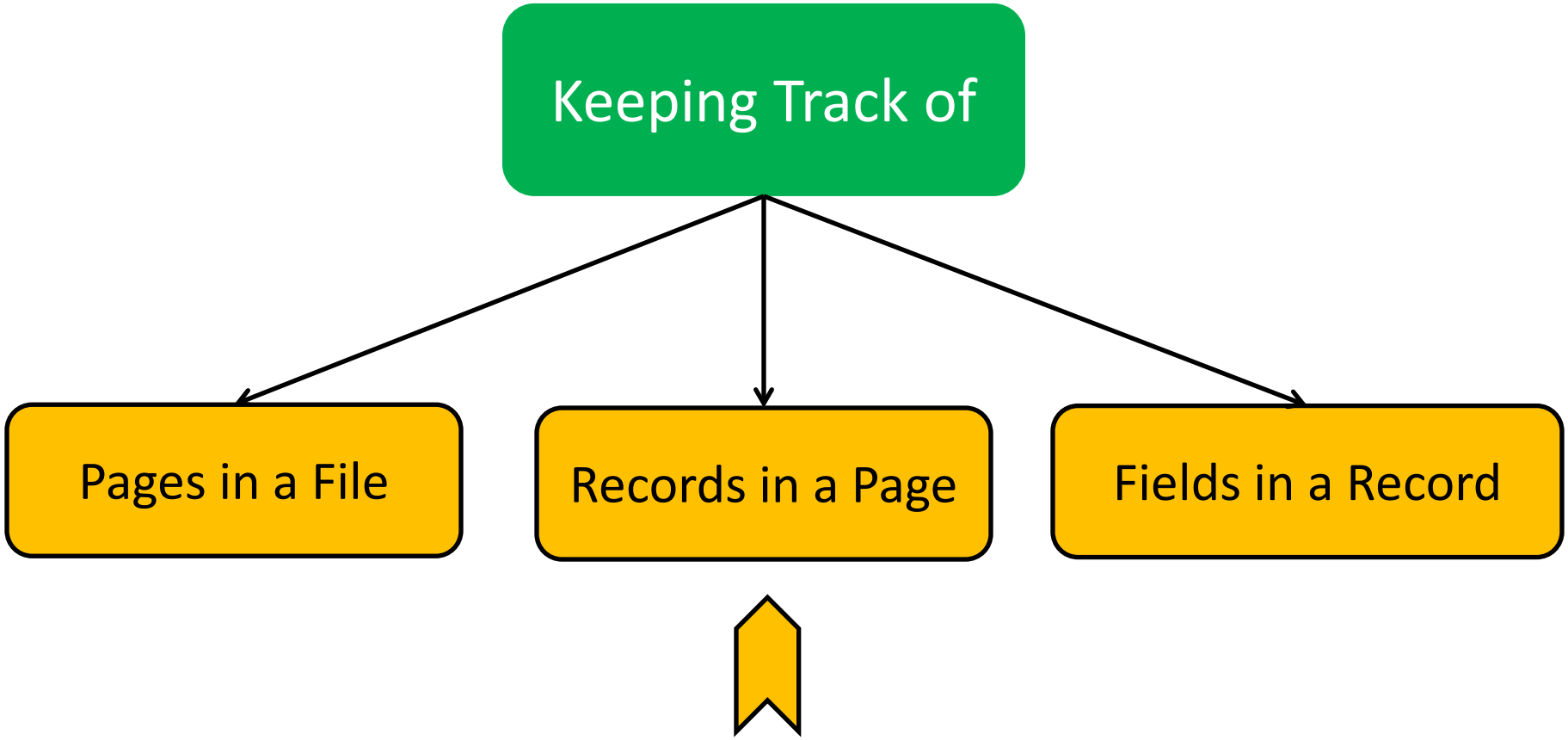
Unordered Files Using *Directory* of Pages

- A directory of pages can be maintained whereby each directory entry identifies a page in the heap file
- Free space can be managed via maintaining:
  - A *bit* per entry (indicating whether the corresponding page has any free space)
  - A *count* per entry (indicating the amount of free space on the page)



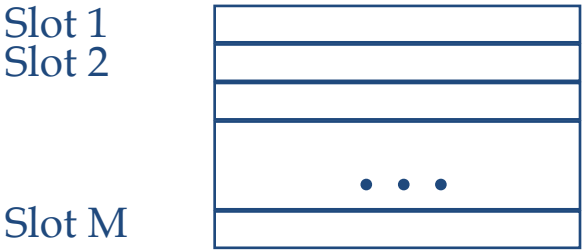


Supporting Record Level Operations



Page Formats

- A page of a file can be thought of as a collection of slots, each of which contains a record

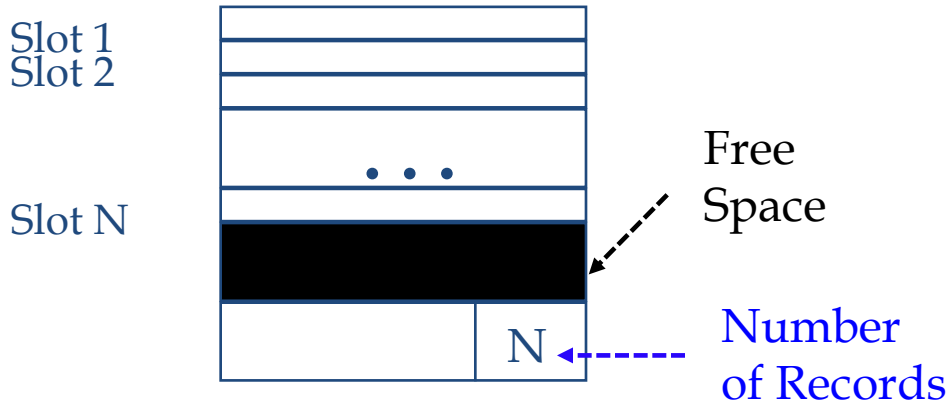


- A record can be identified using the pair <page_id, slot_#>, which is typically referred to as record id (rid)

- Records can be either:
  - Fixed-Length
  - Variable-Length

Fixed-Length Records

- When records are of fixed-length, slots become *uniform* and can be arranged *consecutively*

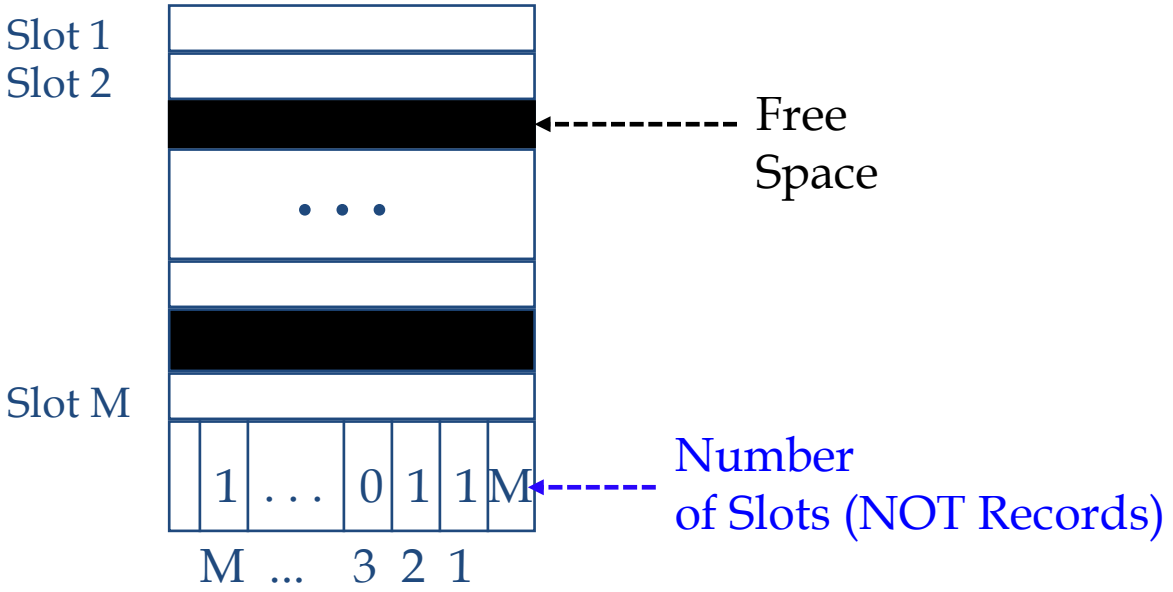


- Records can be located by simple offset calculations

- Whenever a record is *deleted*, the last record on the page is *moved* into the vacated slot

Fixed-Length Records

- Alternatively, we can handle deletions by using an array of bits



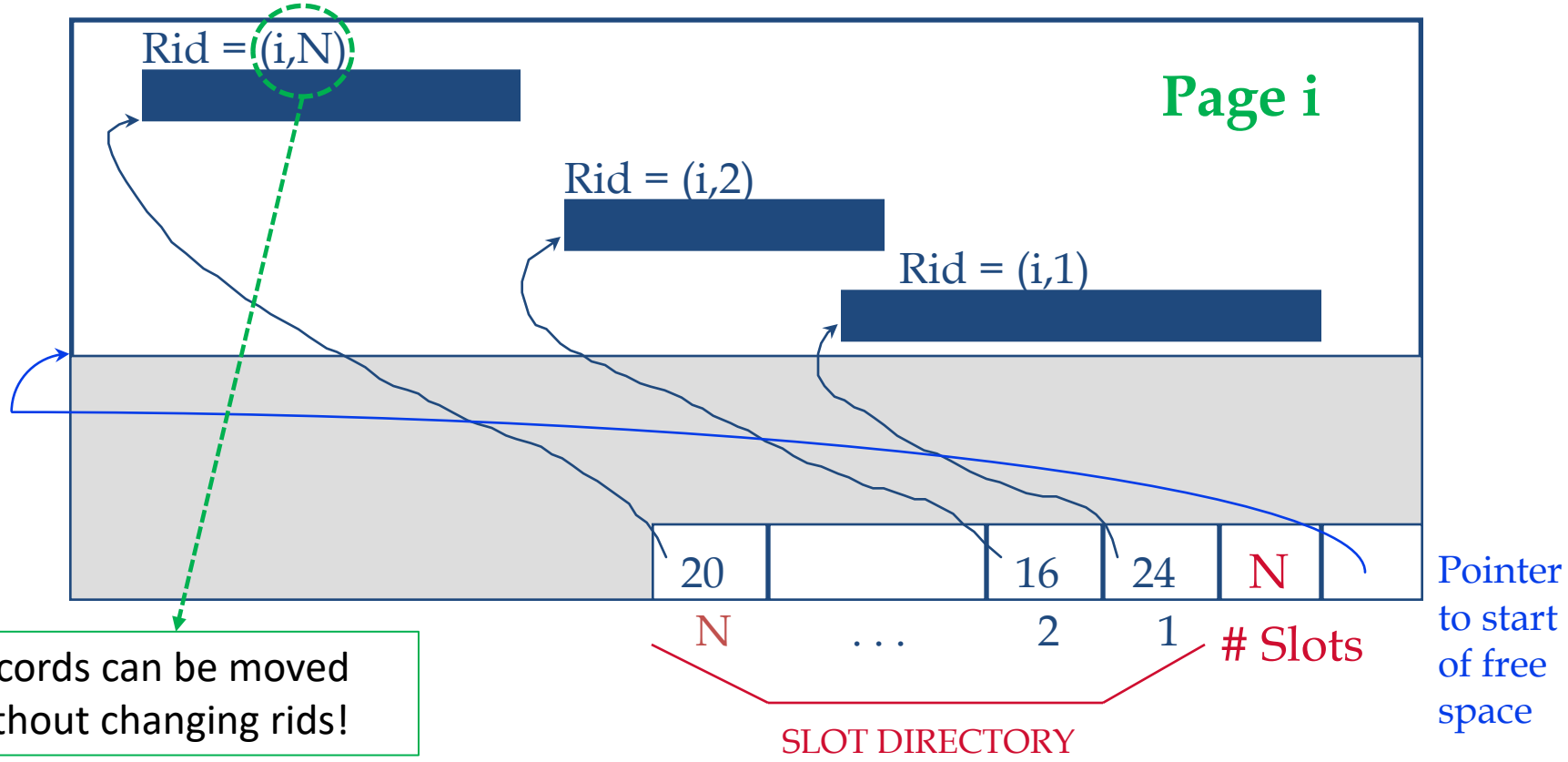
- When a record is deleted, its bit is turned off (0), thus, the rids of currently stored records remain the same!

Variable-Length Records

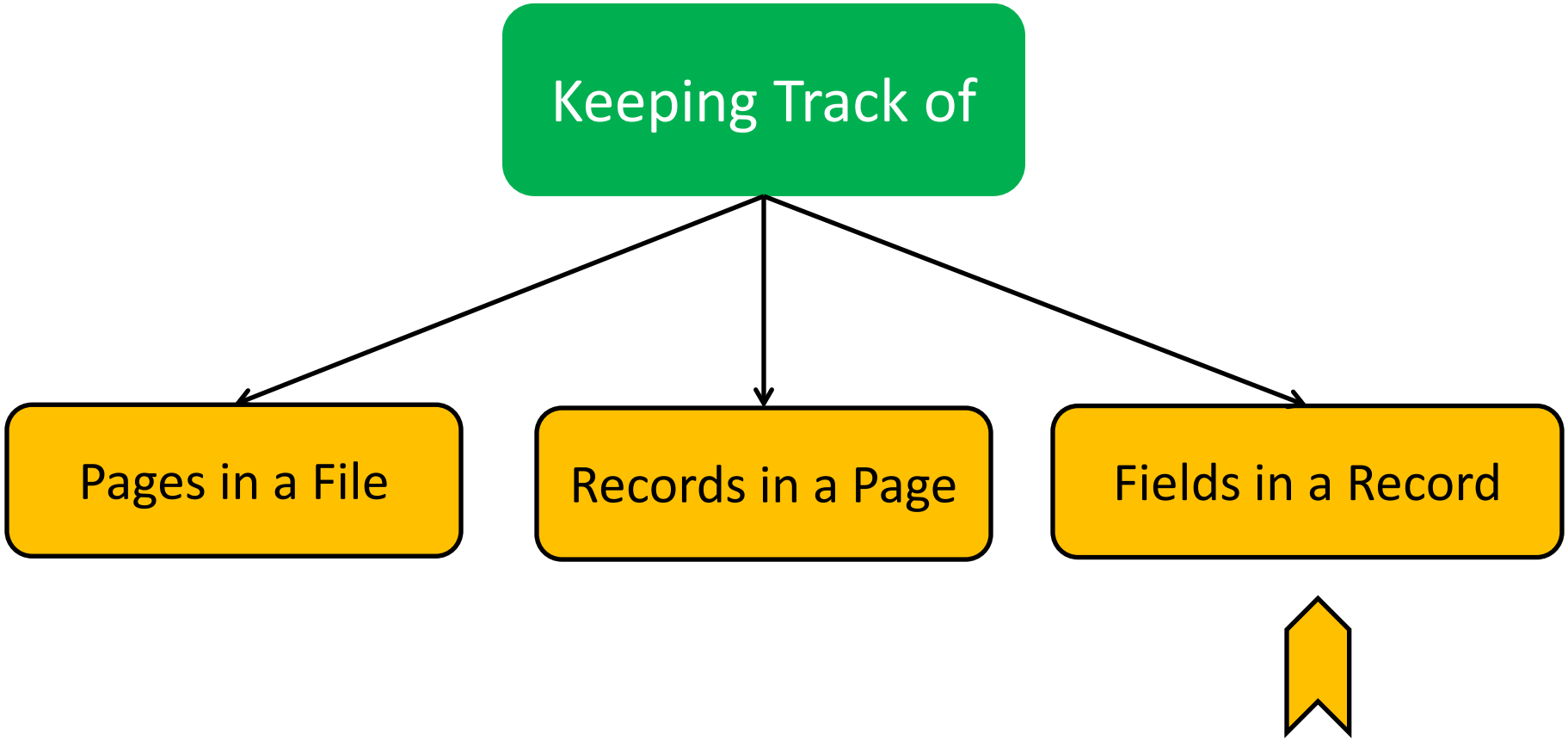
- If the records are of variable length, we cannot divide the page into a fixed collection of slots
- When a new record is to be inserted, we have to find an empty slot of "just" the right length
- Thus, when a record is deleted, we better ensure that all the free space is contiguous
- The ability of moving records "*without changing rids*" becomes crucial!

Pages with Directory of Slots

- A flexible organization for variable-length records is to maintain a directory of slots with a *<record_offset, record_length>* pair per a page



Supporting Record Level Operations

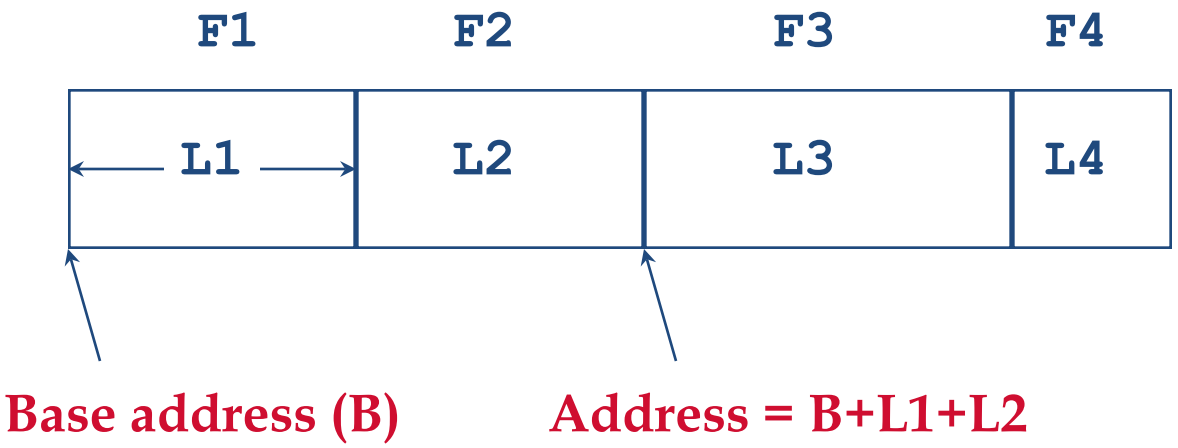


Record Formats

- Fields 字段 in a record can be either of:
  - Fixed-Length: each field has a fixed length and the number of fields is also fixed
  - Variable-Length: fields are of variable lengths but the number of fields is fixed
- Information common to all records (e.g., number of fields and field types) are stored in the system catalog 系统目录

Fixed-Length Fields

- Fixed-length fields can be stored consecutively and their addresses can be calculated using information about the lengths of preceding fields



Variable-Length Fields

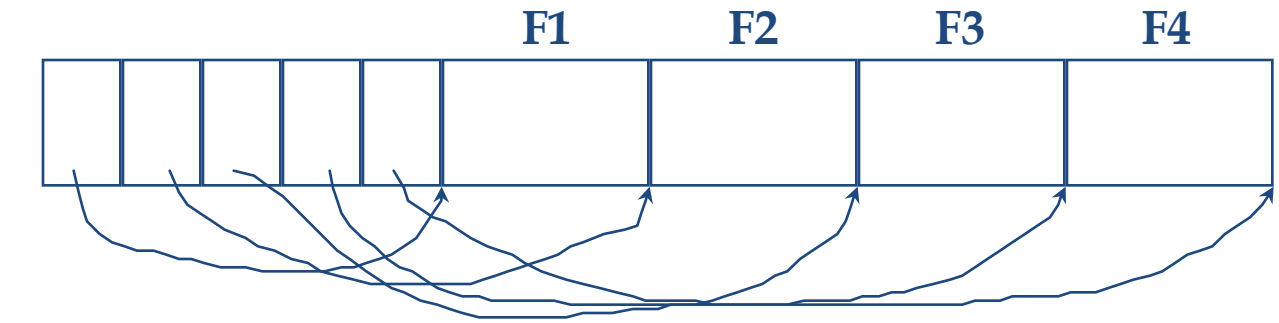
- There are two possible organizations to store variable-length fields
  1. Consecutive storage of fields separated by delimiters 分隔符



This entails a scan of records to locate a desired field!

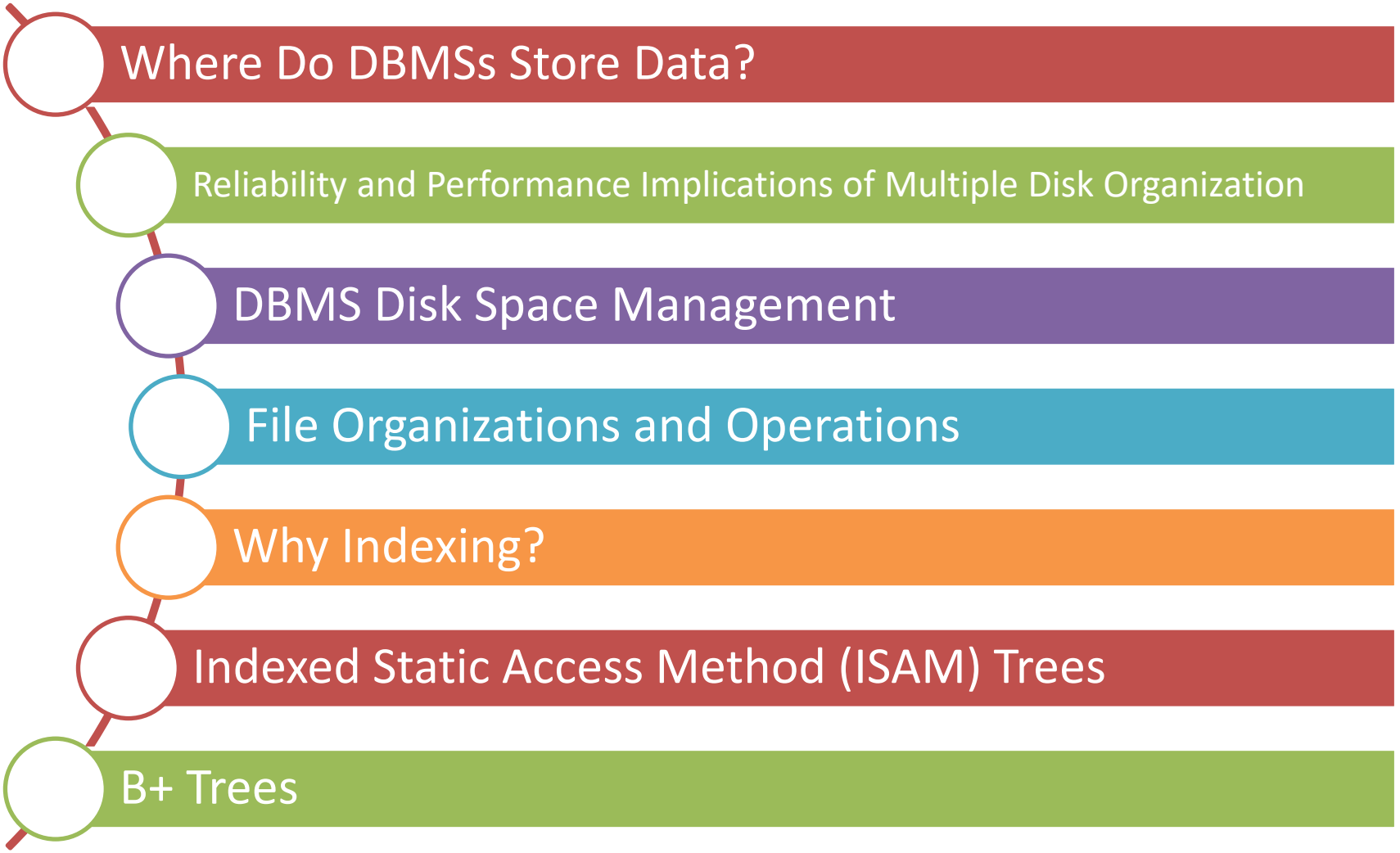
Variable-Length Fields

- There are two possible organizations to store variable-length fields
  1. Consecutive storage of fields separated by delimiters
  2. Storage of fields with an array of integer offsets



This offers direct access to a field in a record

Outline


Where Do DBMSs Store Data?
Reliability and Performance Implications of Multiple Disk Organization
DBMS Disk Space Management
File Organizations and Operations
Why Indexing?
Indexed Static Access Method (ISAM) Trees
B+ Trees

Book Index

INDEX

A

abbreviations, list of common, 237–239
Accredited Social Health Activists (ASHAs), 71
ACM (Association for Computing Machinery), xxviii
alpine natural hazards, forecasting, 48–49
amateurs. *See* citizen science
Amazon.com, 166
Anderson, Chris, 218
Antarctic Treaty, 203, 204
Apache Web server, 212
application-based science vs. basic science, 14–18. *See also* science of environmental applications
archiving. *See also* curation; digital data libraries
  as core function in scholarly communication, 195
  data vs. literature, xii, xxvii–xxviii, xxx
Australian Square Kilometre Array Pathfinder (ASKAP), xiii, 147
Automatic Tape-Collecting Lathe Ultramicrotome (ATLUM), 79, 80
avatars, in healthcare, 96–97
Axial Seamount, 32
Azure platform, 133

B

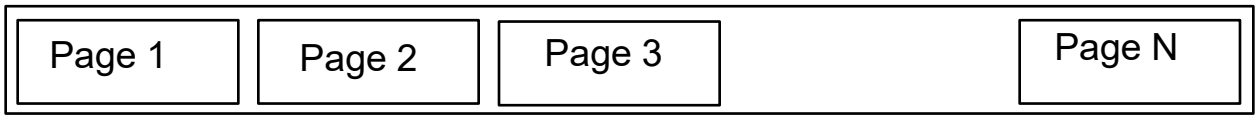
basic science vs. science based on applications, 14–18
Beowulf clusters, xx, xxiv, 126
Berlin Declaration on Open Access to Knowledge in the Sciences and Humanities, 203
Bermuda Principles, 203
Berners-Lee, Tim, 171, 188–189
BGI Shenzhen, 120–121
Bing, xxvi
BioCatalogue, 143

Book Index

- Book index: lists important terms in alphabetical order with a list of page number(s) where the term appears
- Searching in a book:
  - Search the book index for a term to find a list of addresses (i.e., page numbers)
  - Use these addresses to retrieve the specified pages
  - Search for the term in each page
- Otherwise, search through the entire book word by word (≈ linear search)
- A database index is similar to a book index!

Another Example

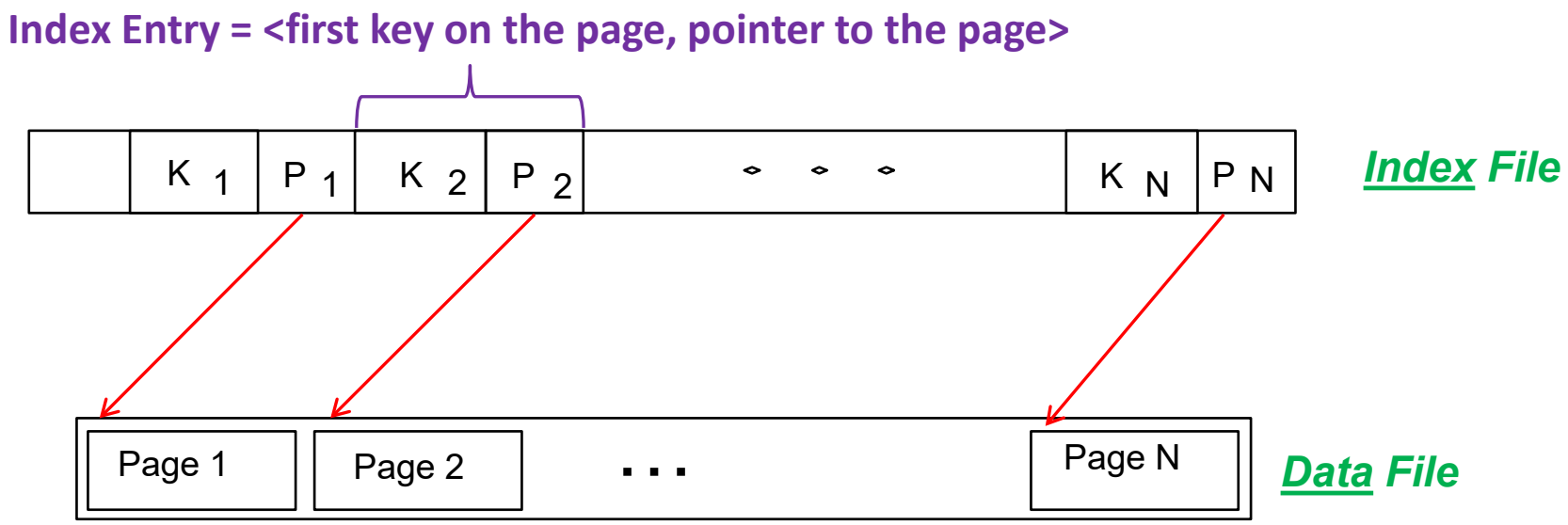
- Consider a file of student records *sorted* by GPA



- How can we answer a *range selection* (E.g., *"Find all students with a GPA higher than 3.0"*)?
  - What about doing a *binary search* followed by a *scan*?
    - Yes, but…
  - What if the file becomes "very" large?
    - Cost is proportional to the number of pages fetched
    - Hence, may become very slow!

Indexing

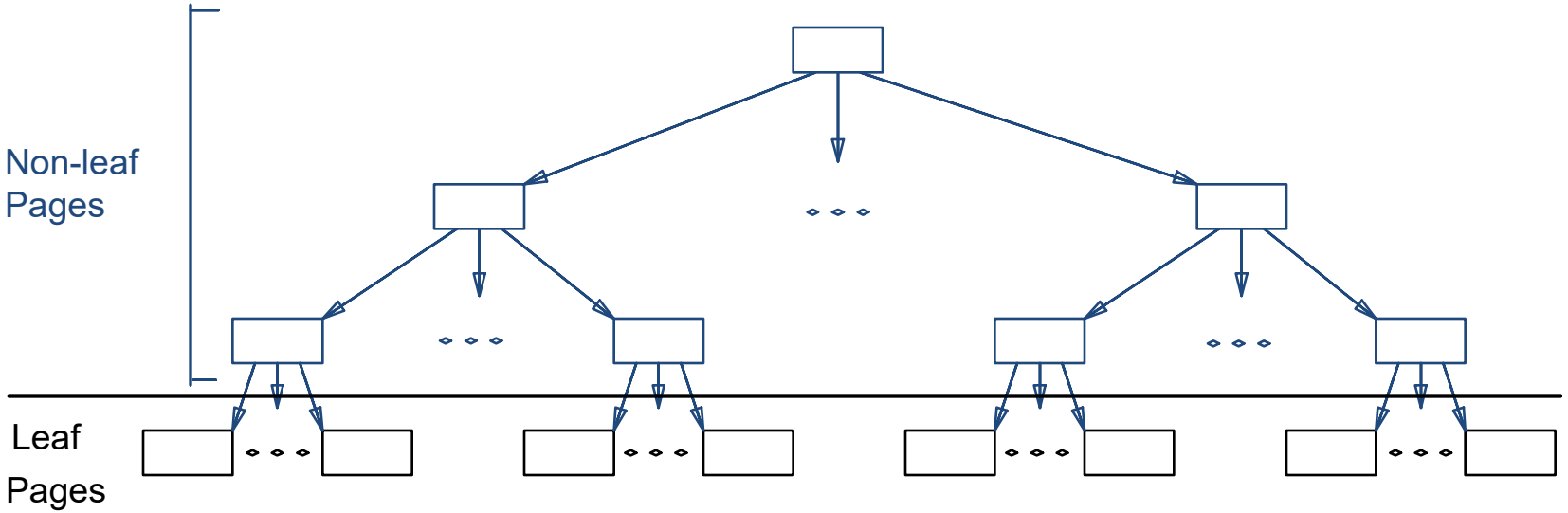
- What about creating an *index file* (with one entry per page) and do binary search there?



- But, what if the index file becomes also "very" large?

Indexing

- Repeat recursively and build a TREE!



Each tree page is a disk block and all data records reside (*if chosen to be part of the index*) in ONLY leaf pages

How else data records can be stored?

Where to Store Data Records?

- In general, *3 alternatives* for "data records" (each referred to as ***K**)*** can be pursued:
  - Alternative (1): *K** is an actual data record with key ***k***

    Leaf pages contain the actual data (i.e., the data records)
  - Alternative (2): *K** is a <***k***, **rid**> pair, where rid is the record id of a data record with search key ***k***

    Leaf pages contain the <key, rid> pairs and actual data records are stored in a separate file
  - Alternative (3): *K** is a <***k***, rid-list> pair, where rid-list is a list of rids of data records with search key ***k***

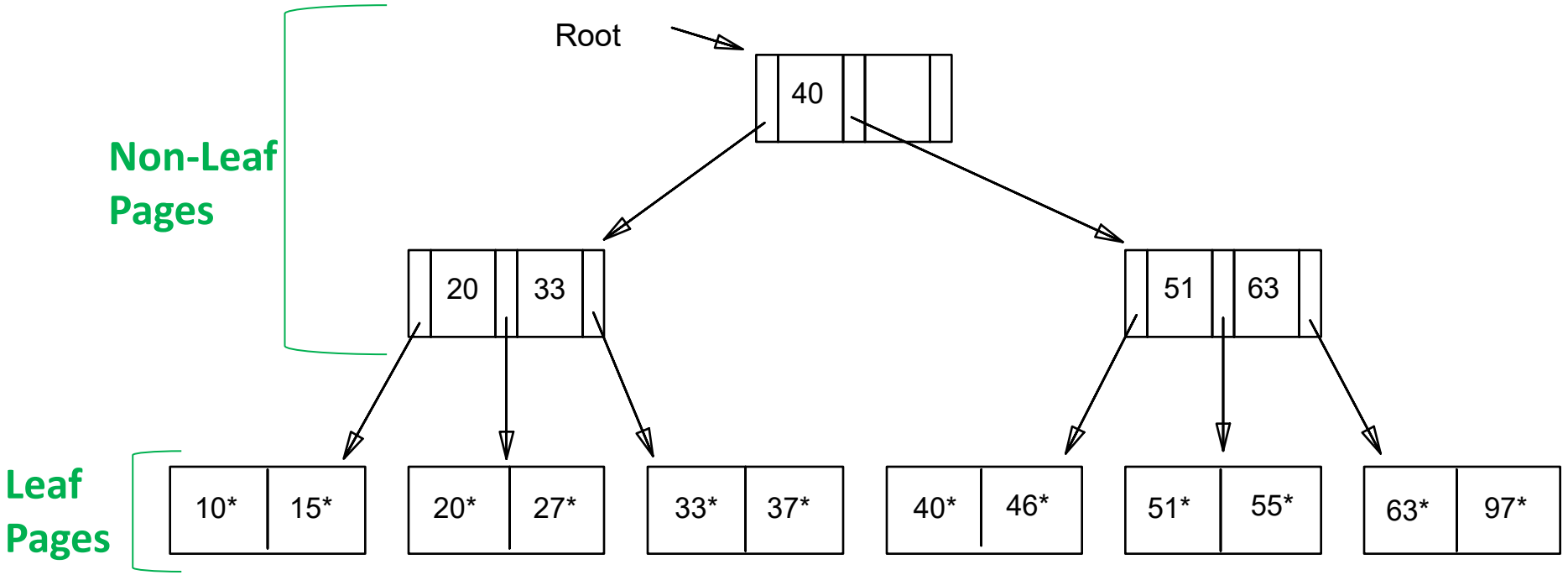
    Leaf pages contain the <key, rid-list> pairs and actual data records are stored in a separate file

Outline

- Where Do DBMSs Store Data?
- Reliability and Performance Implications of Multiple Disk Organization
- DBMS Disk Space Management
- File Organizations and Operations
- Why Indexing?
- Indexed Static Access Method (ISAM) Trees ✓
- B+ Trees

ISAM Trees

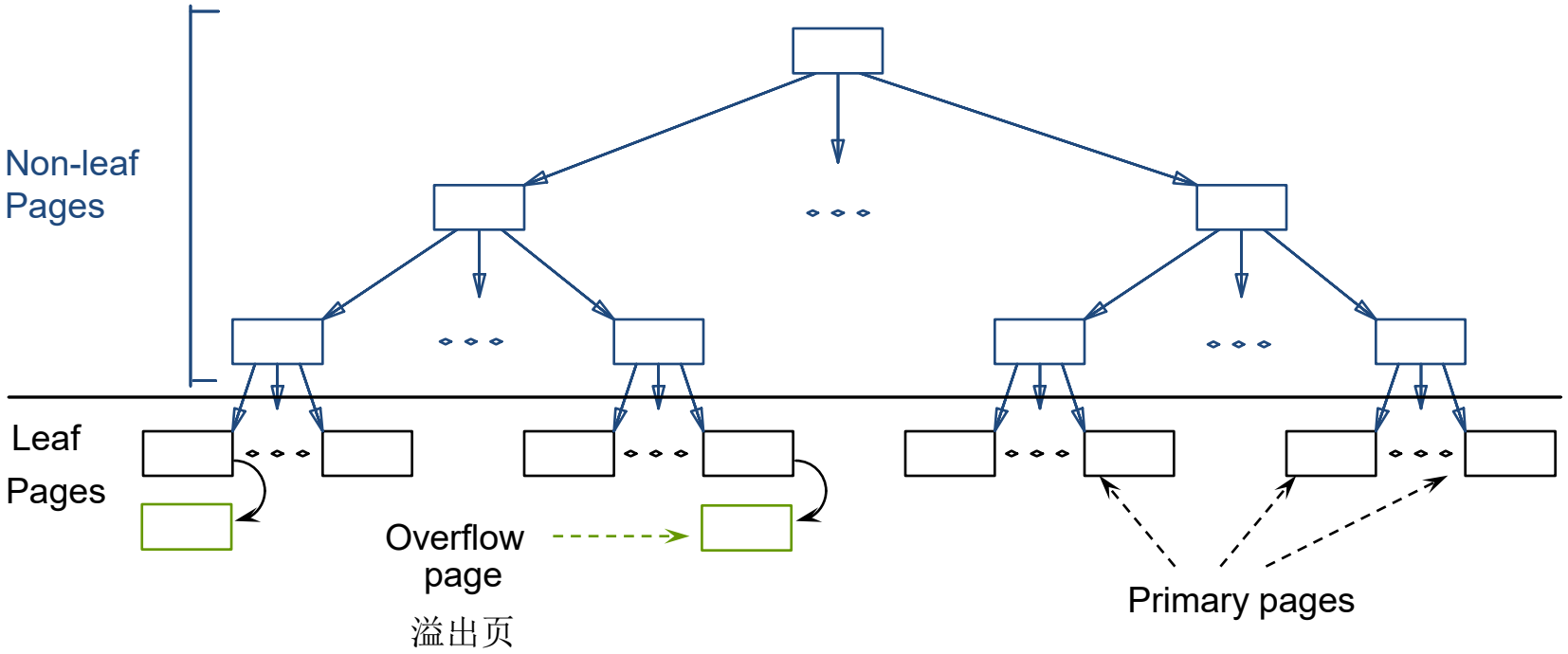
- Indexed Sequential Access Method (ISAM，索引顺序存取法) trees are *static*



E.g., 2 Entries Per Page

ISAM Trees: Page Overflows

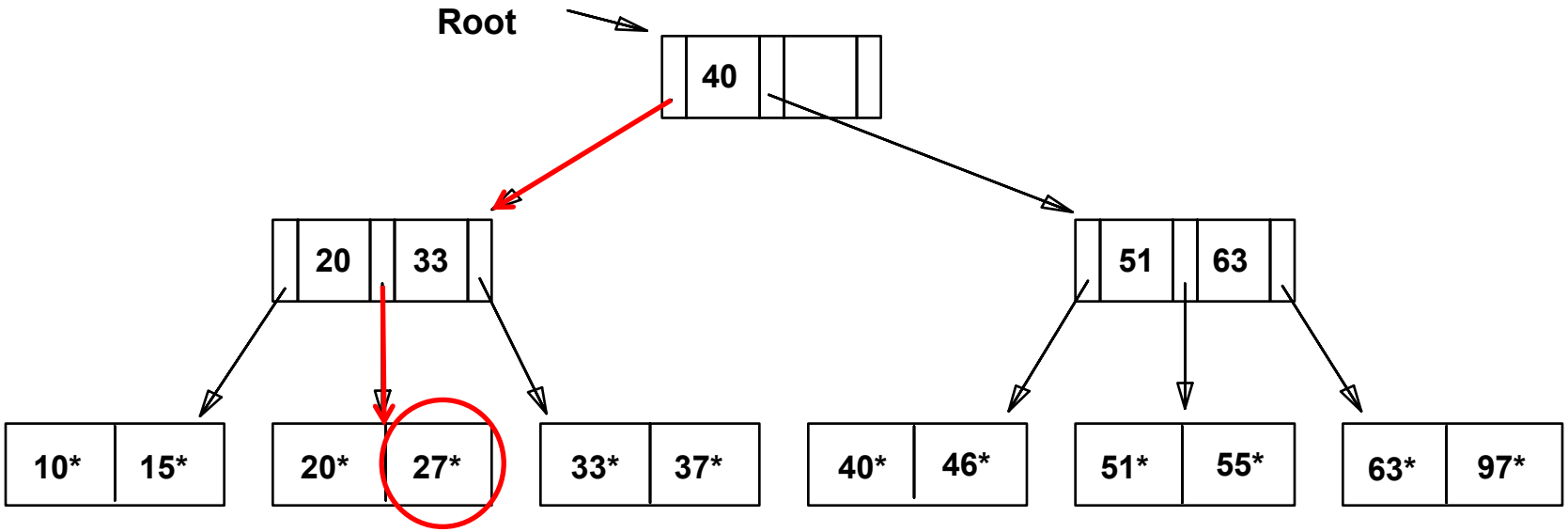
- What if there are a lot of insertions after creating the tree?



ISAM: Searching for Entries

- Search begins at root, and key comparisons direct it to a leaf

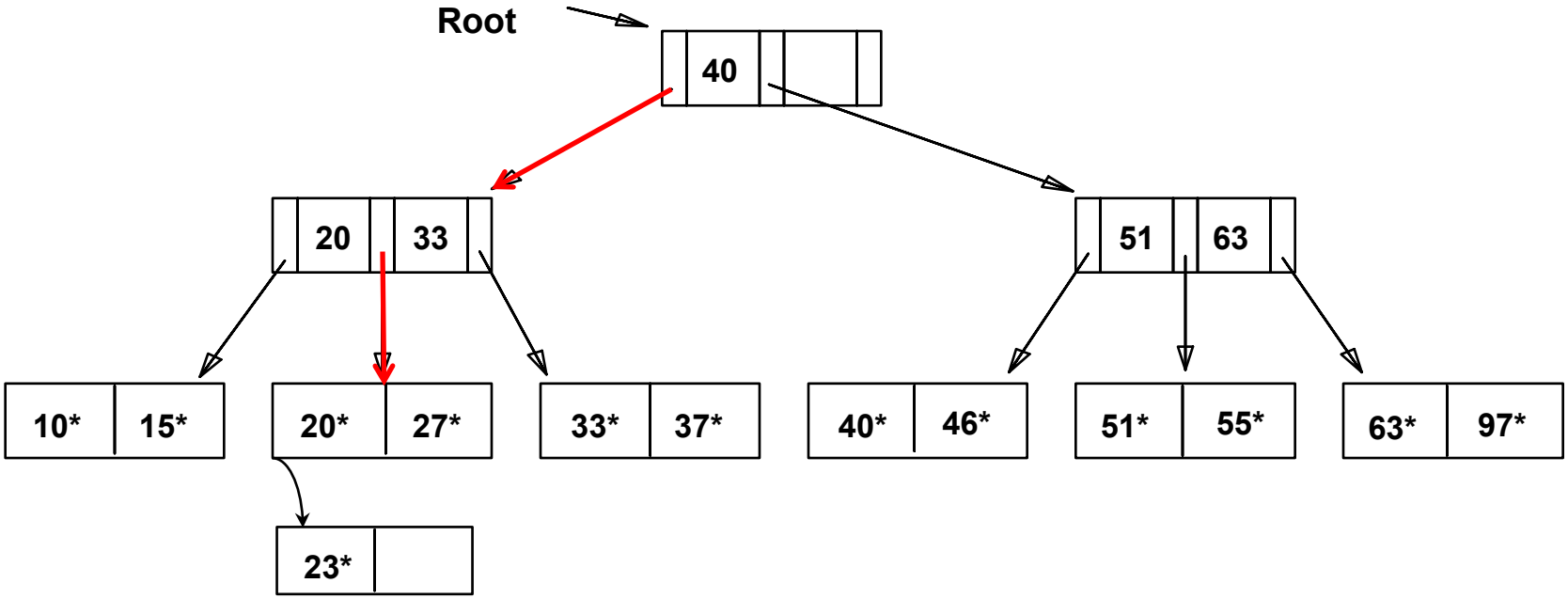
- Search for 27*



ISAM: Inserting Entries

- The appropriate page is determined as for a search, and the entry is inserted (with overflow pages added if necessary)

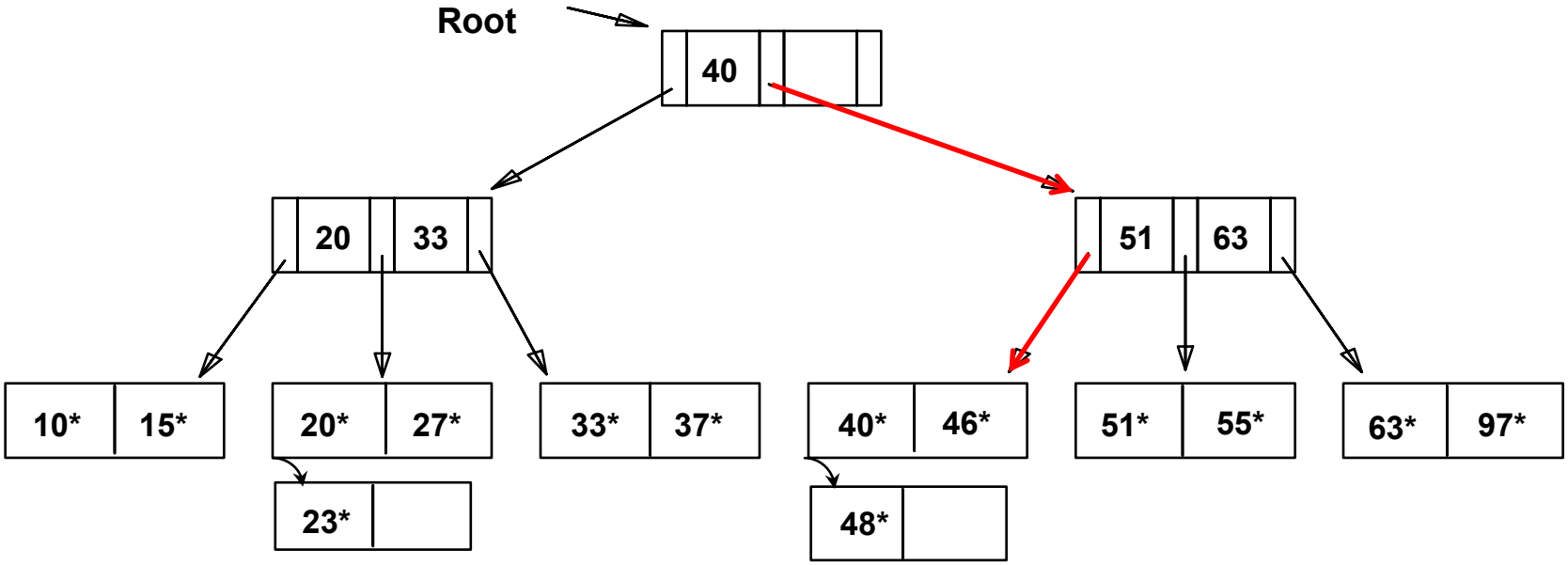
- Insert 23*



ISAM: Inserting Entries

- The appropriate page is determined as for a search, and the entry is inserted (with overflow pages added if necessary)

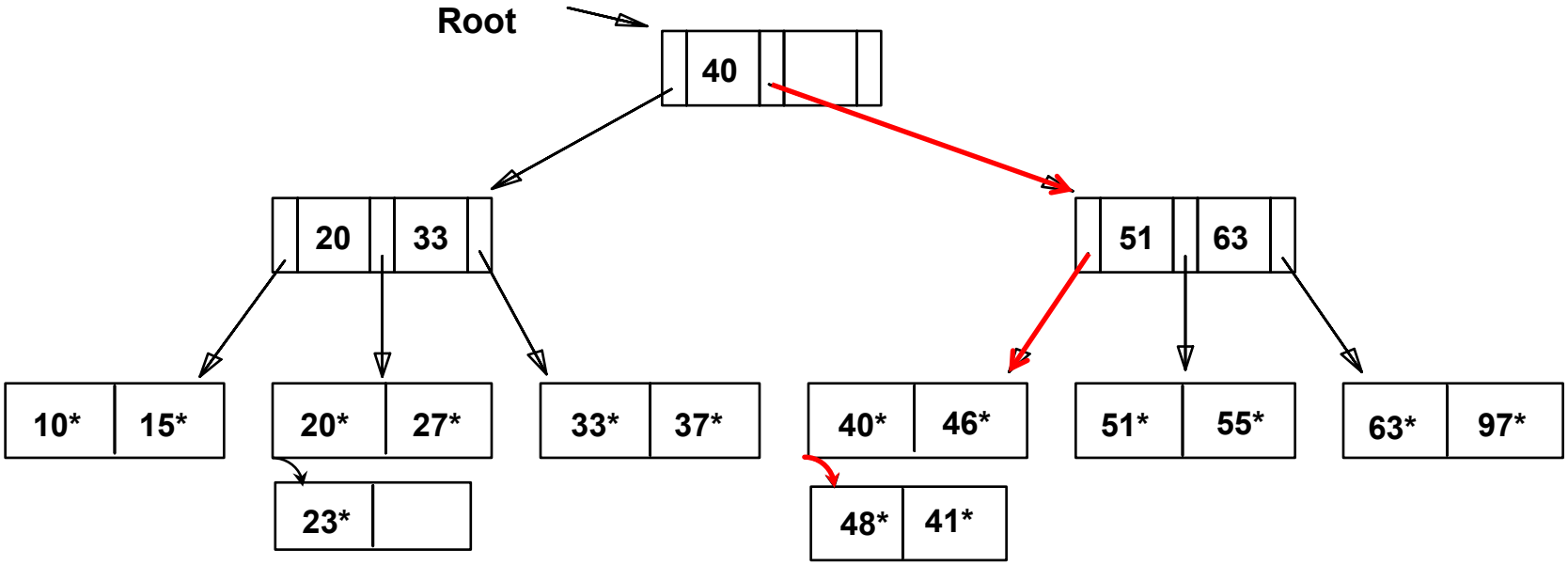
- Insert 48*



ISAM: Inserting Entries

- The appropriate page is determined as for a search, and the entry is inserted (with overflow pages added if necessary)

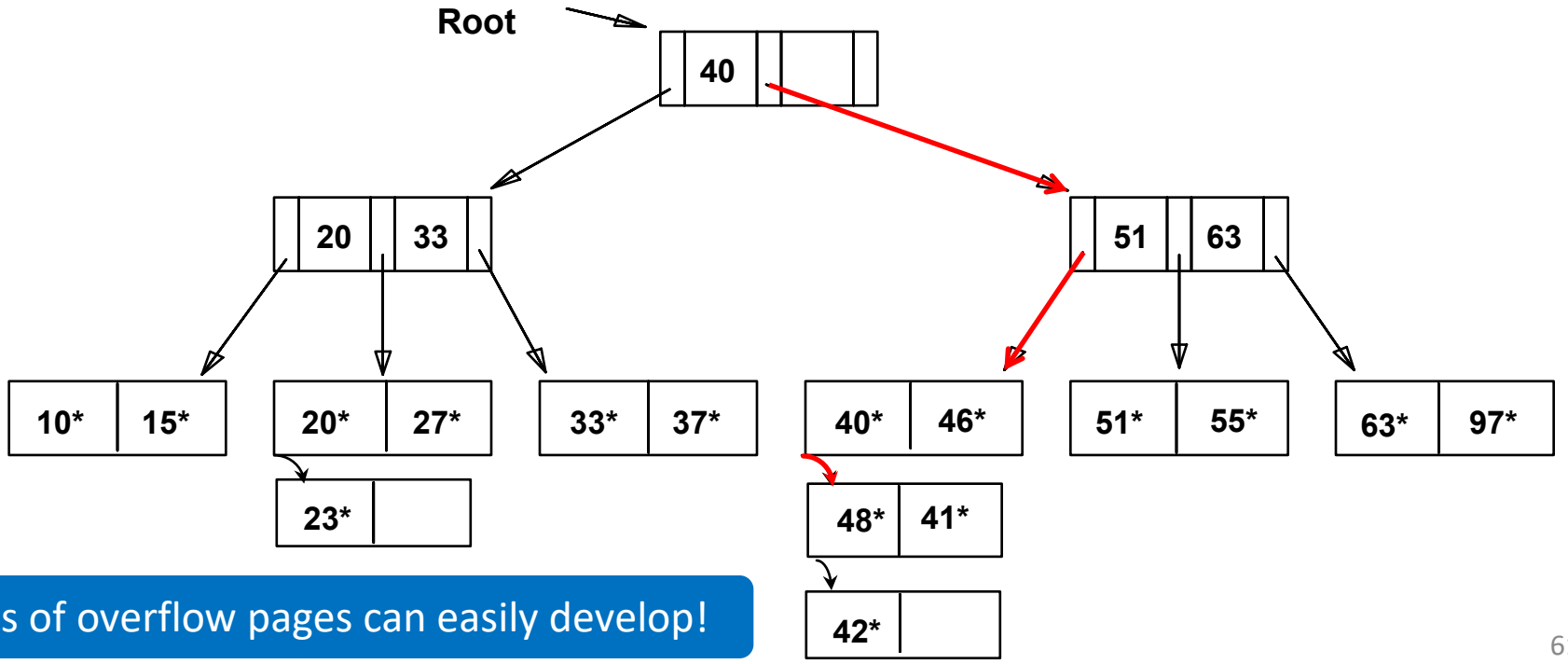
- Insert 41*



ISAM: Inserting Entries

- The appropriate page is determined as for a search, and the entry is inserted (with overflow pages added if necessary)

- Insert 42*

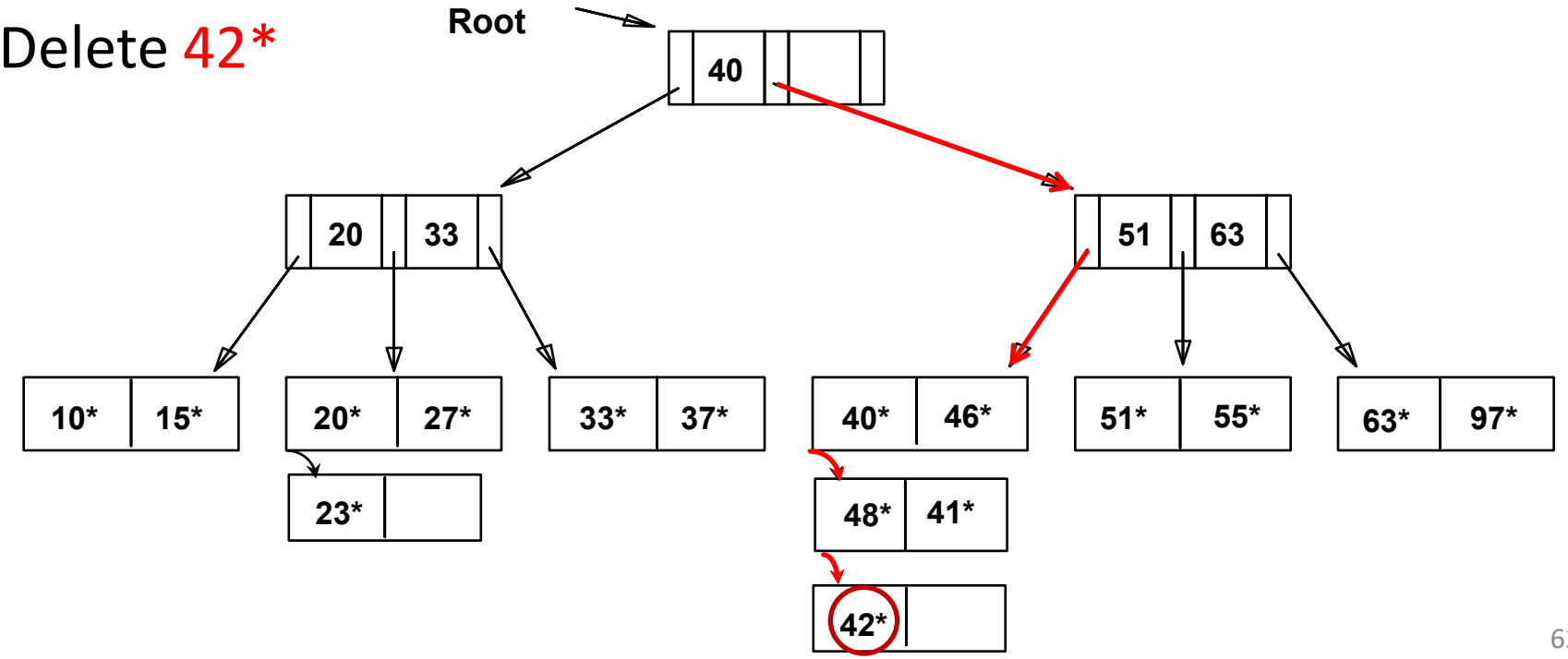


Chains of overflow pages can easily develop!

ISAM: Deleting Entries

- The appropriate page is determined as for a search, and the entry is deleted (*with ONLY overflow pages removed when becoming empty*)

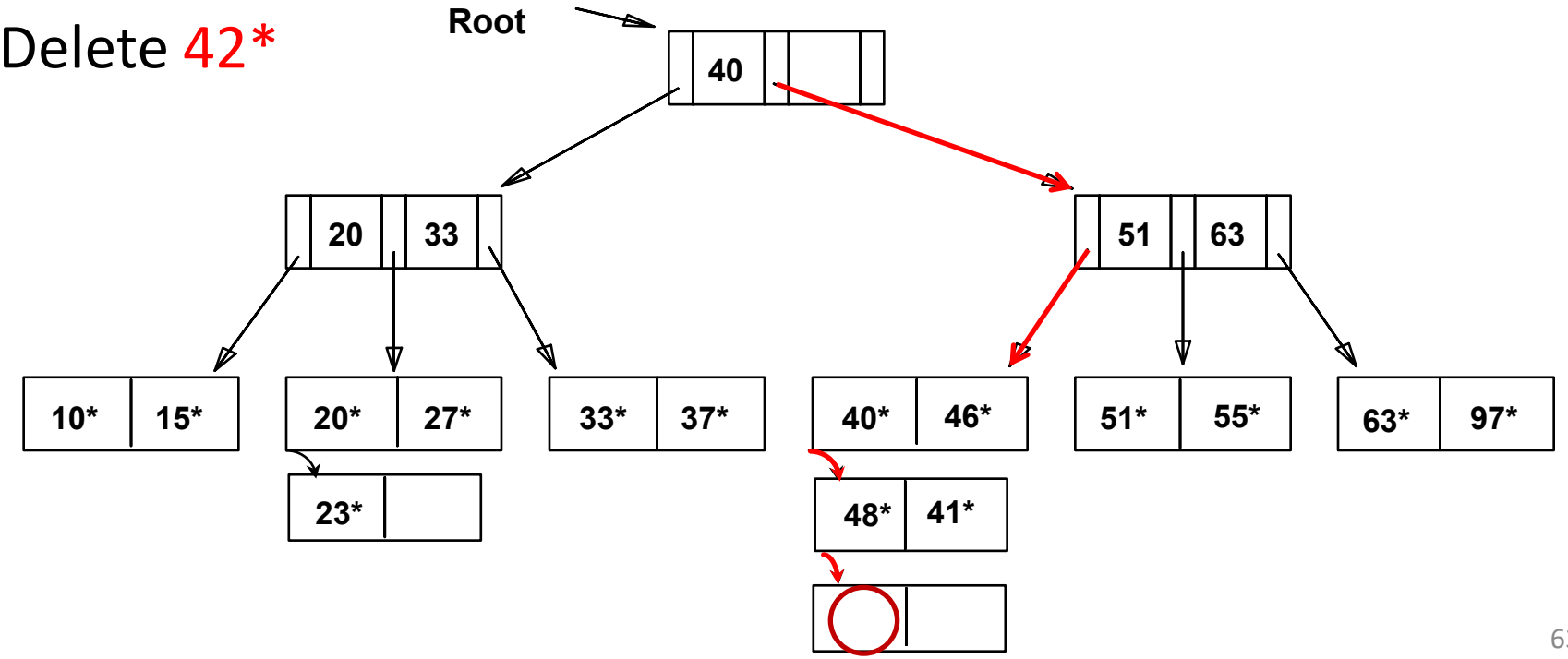
- Delete 42*



ISAM: Deleting Entries

- The appropriate page is determined as for a search, and the entry is deleted (*with ONLY overflow pages removed when becoming empty*)

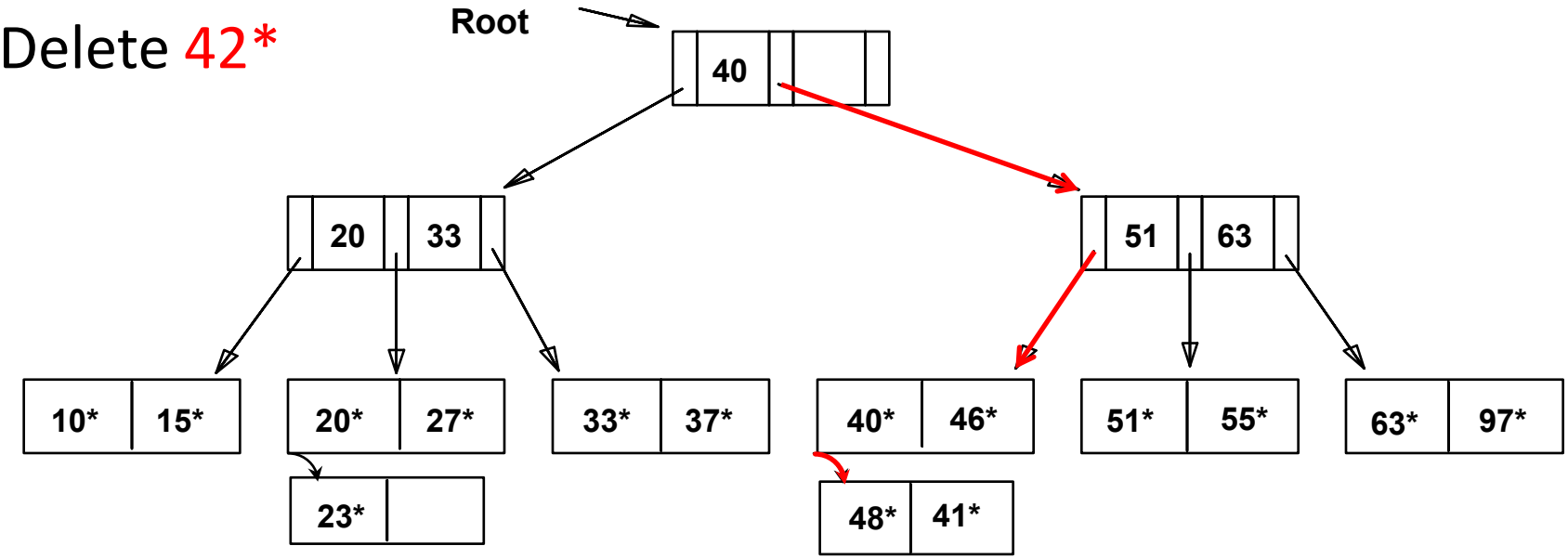
- Delete 42*



ISAM: Deleting Entries

- The appropriate page is determined as for a search, and the entry is deleted (*with ONLY overflow pages removed when becoming empty*)

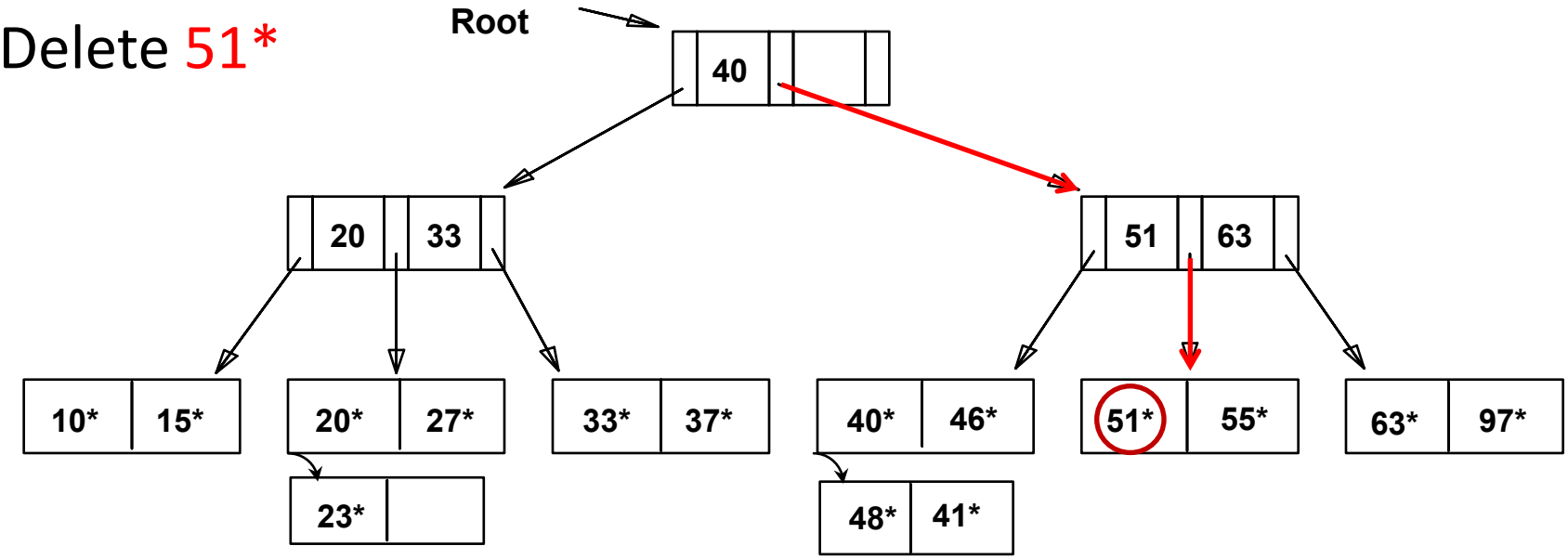
- Delete 42*



ISAM: Deleting Entries

- The appropriate page is determined as for a search, and the entry is deleted (*with ONLY overflow pages removed when becoming empty*)

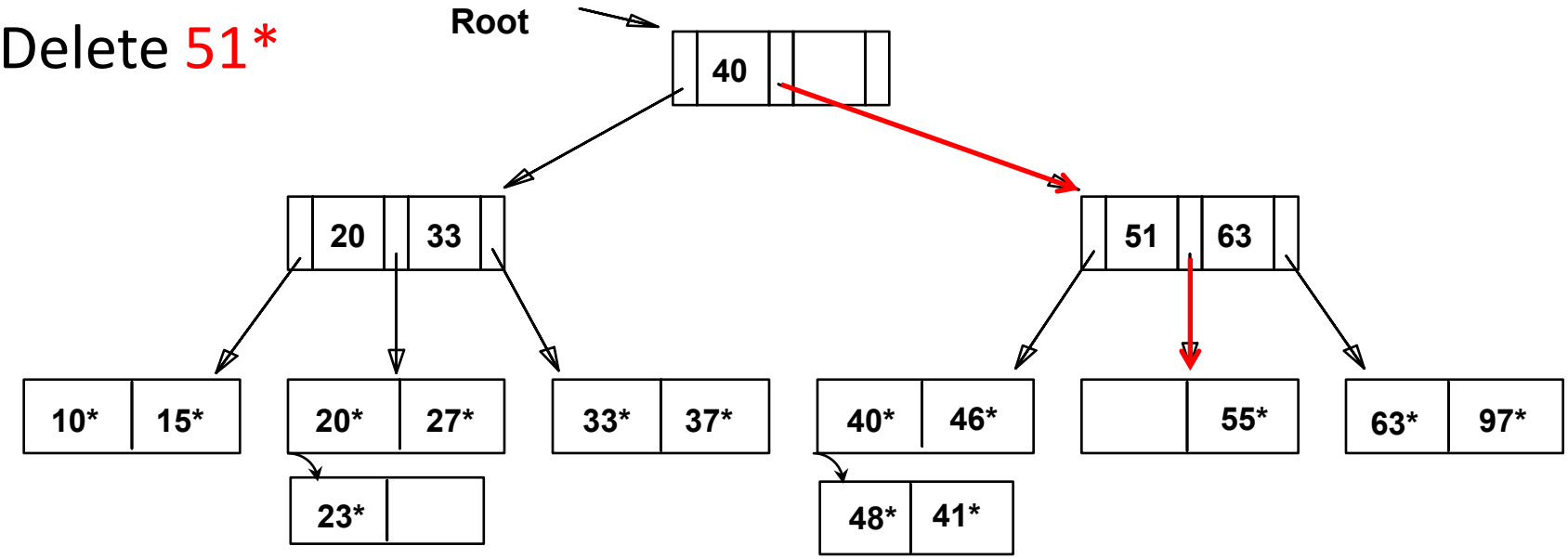
- Delete 51*



ISAM: Deleting Entries

- The appropriate page is determined as for a search, and the entry is deleted (*with ONLY overflow pages removed when becoming empty*)

- Delete 51*

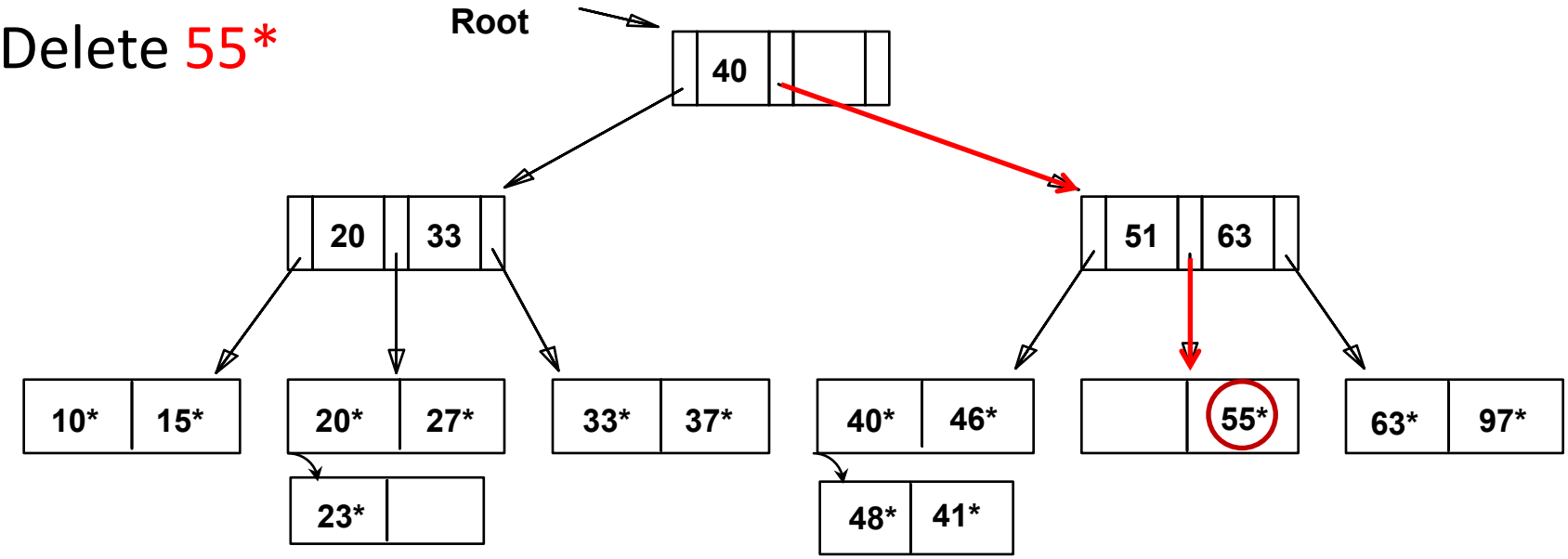


Note that 51 still appears in an index entry, but not in the leaf!

ISAM: Deleting Entries

- The appropriate page is determined as for a search, and the entry is deleted (*with ONLY overflow pages removed when becoming empty*)

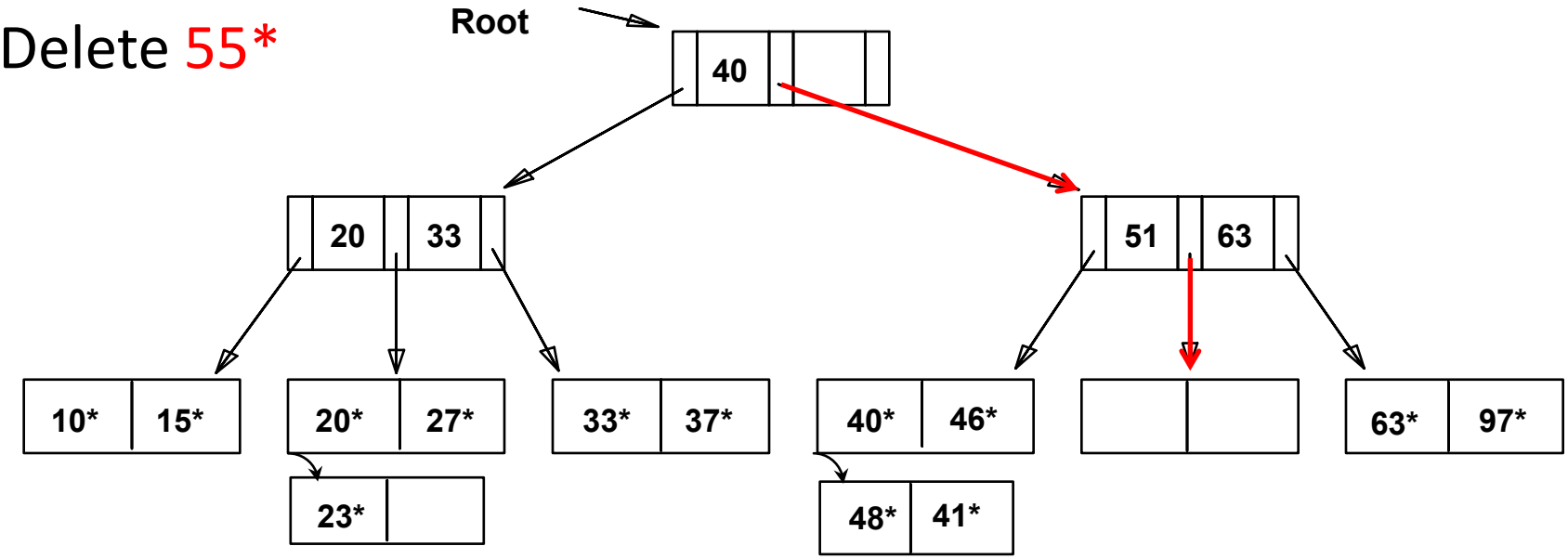
- Delete 55*



ISAM: Deleting Entries

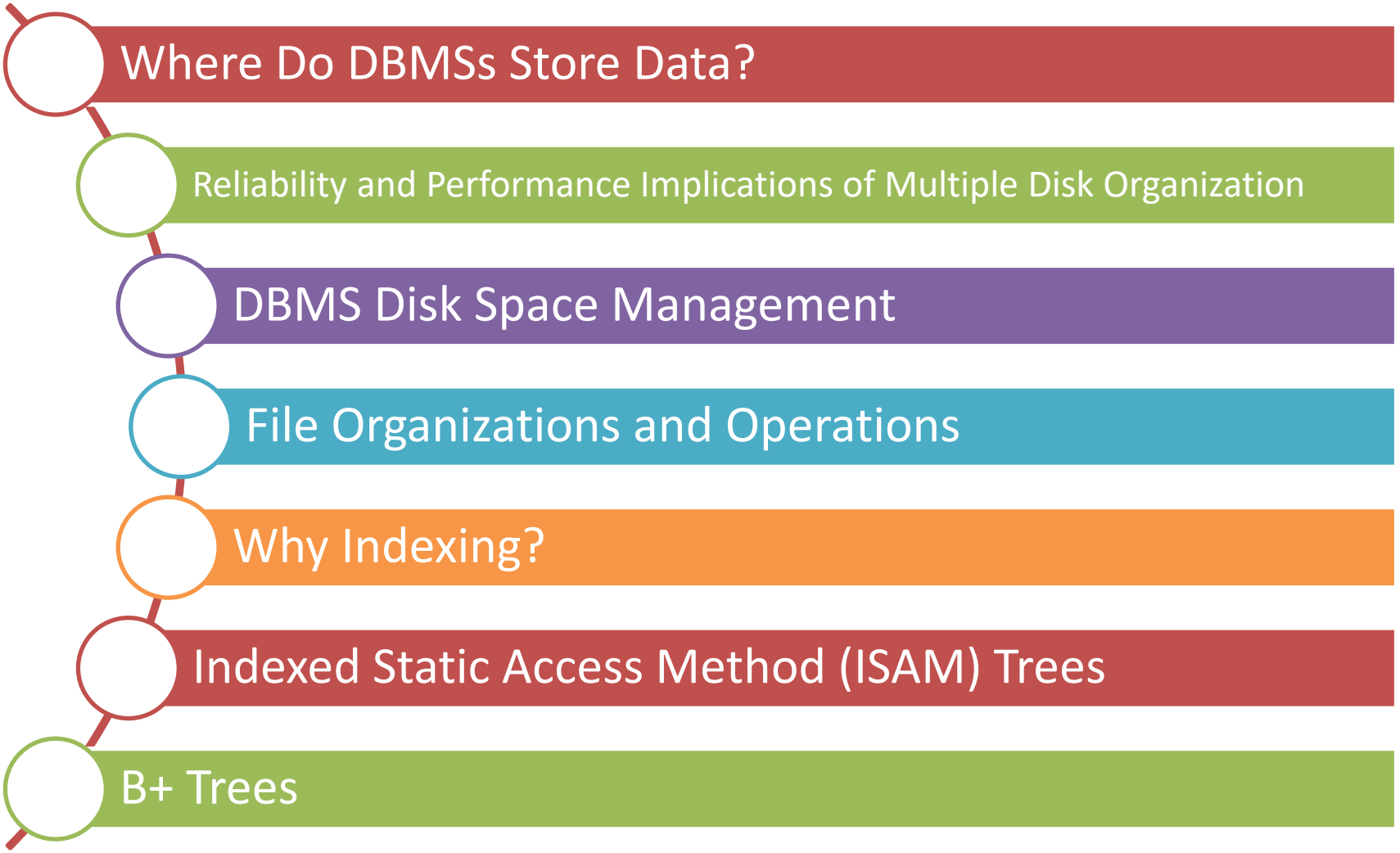
- The appropriate page is determined as for a search, and the entry is deleted (*with ONLY overflow pages removed when becoming empty*)

- Delete 55*



Primary pages are NOT removed, even if they become empty!

Outline


Where Do DBMSs Store Data?
Reliability and Performance Implications of Multiple Disk Organization
DBMS Disk Space Management
File Organizations and Operations
Why Indexing?
Indexed Static Access Method (ISAM) Trees
B+ Trees

✓

Dynamic Trees

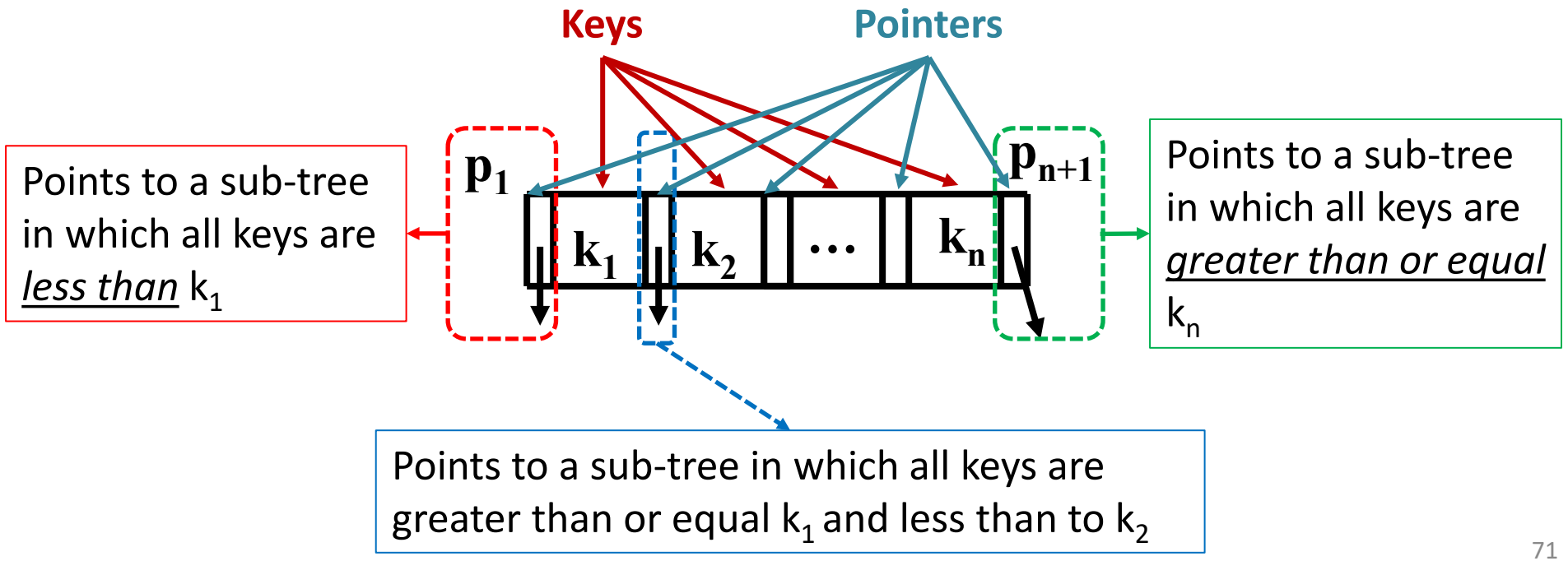
- ISAM indices are static
  - Long overflow chains can develop as the file grows, leading to poor performance

- This calls for more flexible, *dynamic* indices that adjust gracefully to insertions and deletions
  - No need to allocate the leaf pages sequentially as in ISAM

- Among the **most successful** dynamic index schemes is the B+ tree

B+ Tree Properties

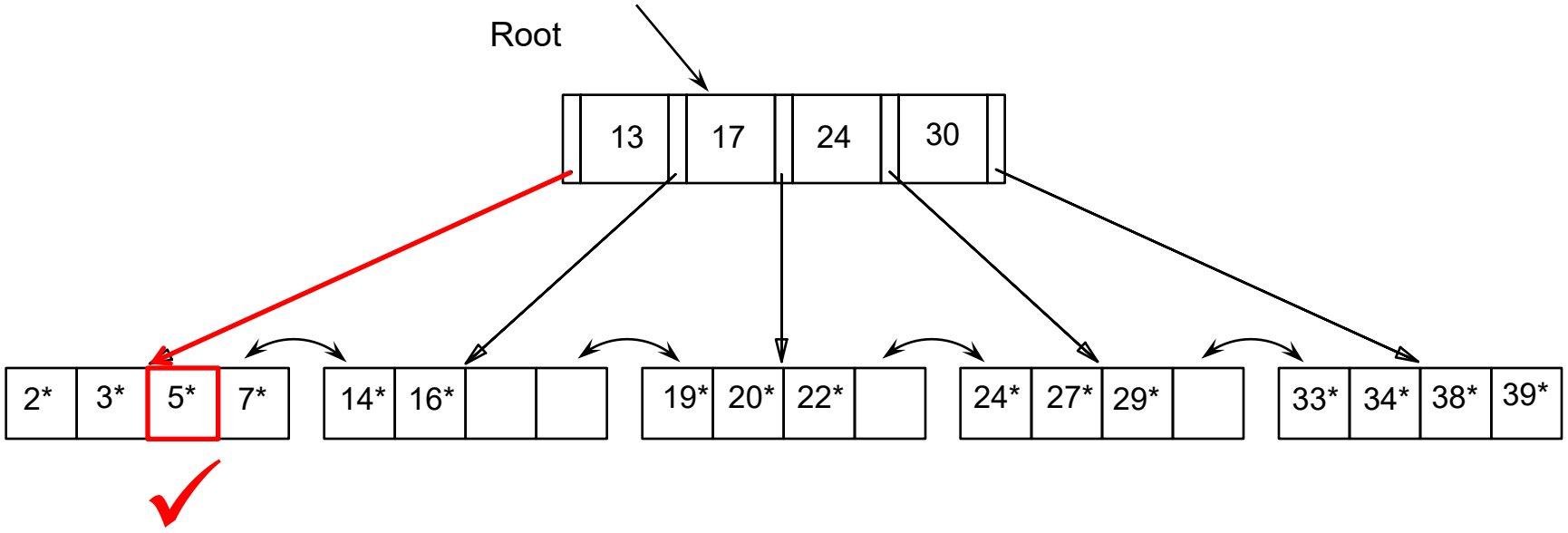
- Each node in a B+ tree of order ***d*** (this is a measure of the capacity of a tree):
  - Has at most ***d*** keys;
  - Has at least ***d/2*** keys (except the root, which may have just 1 key)
  - All leaves are on the same level
  - Has exactly ***n-1*** keys if the number of pointers is ***n***



B+ Tree: Searching for Entries

- Search begins at root, and key comparisons direct it to a leaf (as in ISAM)

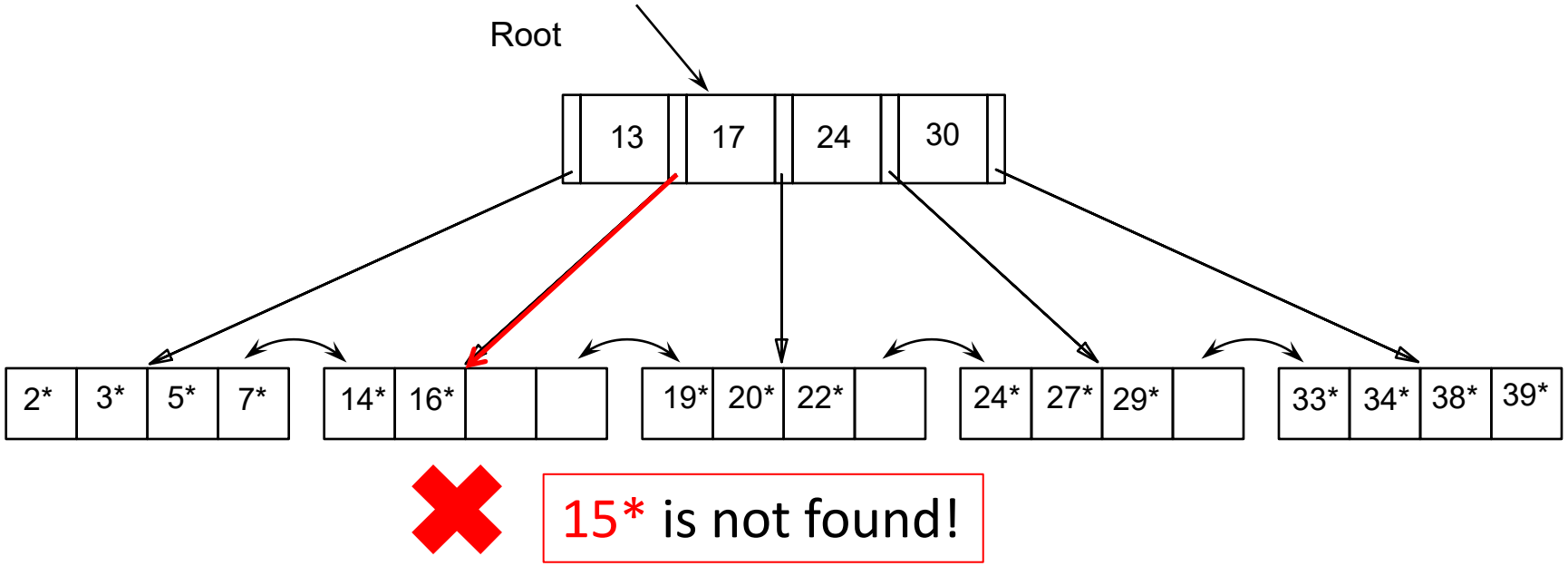
- Example 1: Search for entry 5*



B+ Tree: Searching for Entries

- Search begins at root, and key comparisons direct it to a leaf (as in ISAM)

- Example 2: Search for entry 15*



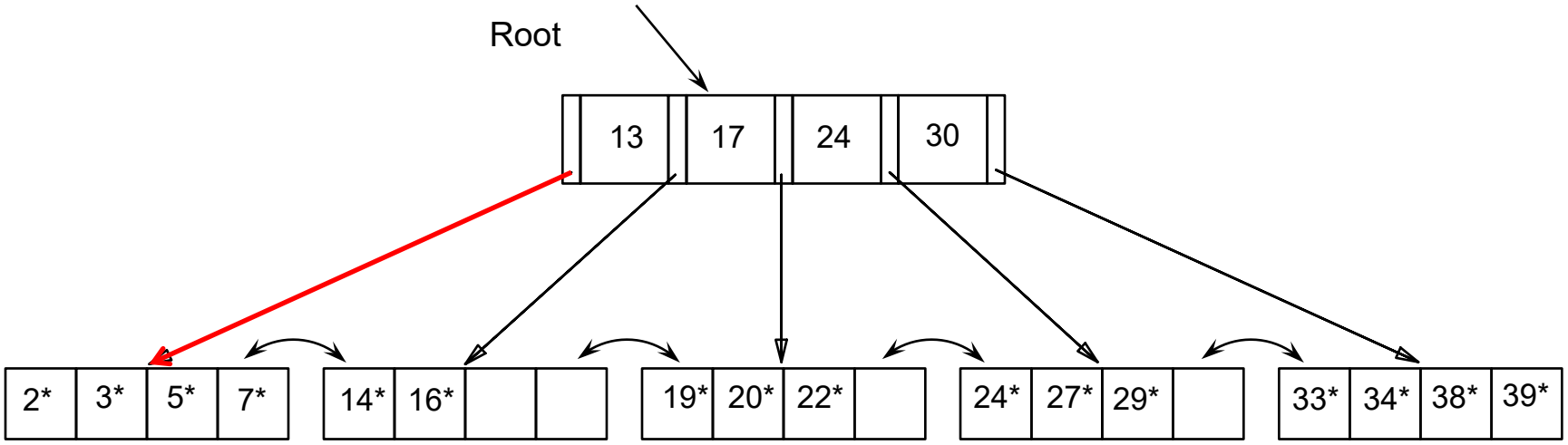
B+ Trees: Inserting Entries

- Find correct leaf ***L***
- Put data entry onto ***L***
  - If ***L*** has enough space, *done*!
  - Else, *split* ***L*** *into* ***L*** *and a new node* L_2
    - Re-partition entries *evenly*, *copying up* the middle key
- Parent node may *overflow*
  - *Push up* middle key (splits "grow" trees; a root split increases the height of the tree)

B+ Tree: Examples of Insertions

- Insert entry 8*

Set the order ***d=4;*** *Max has 4 keys.*

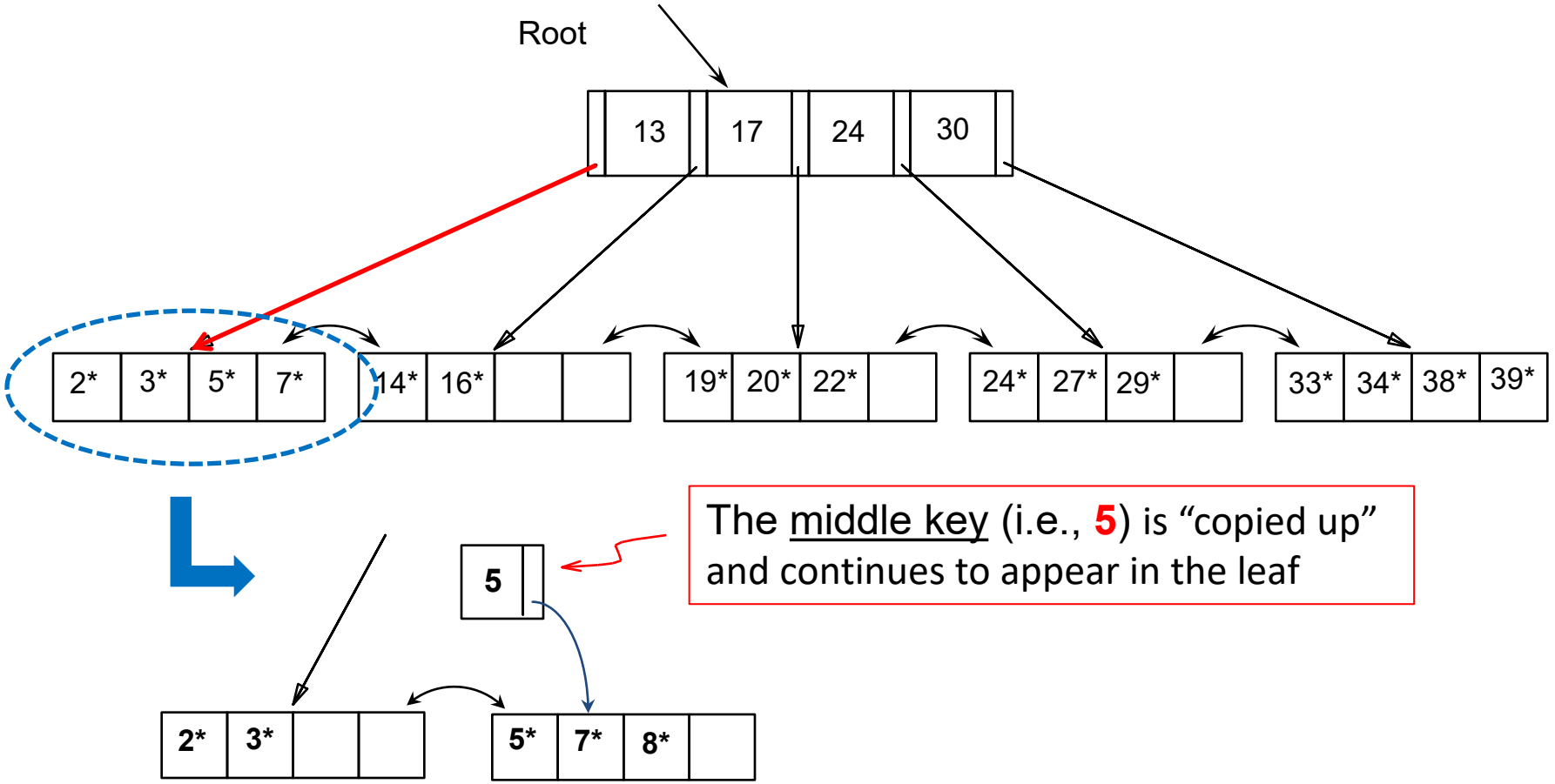


Leaf is ***full***; hence, split!

B+ Tree: Examples of Insertions

- Insert entry 8*

Set the order ***d=4;*** *Max has 4 keys.*

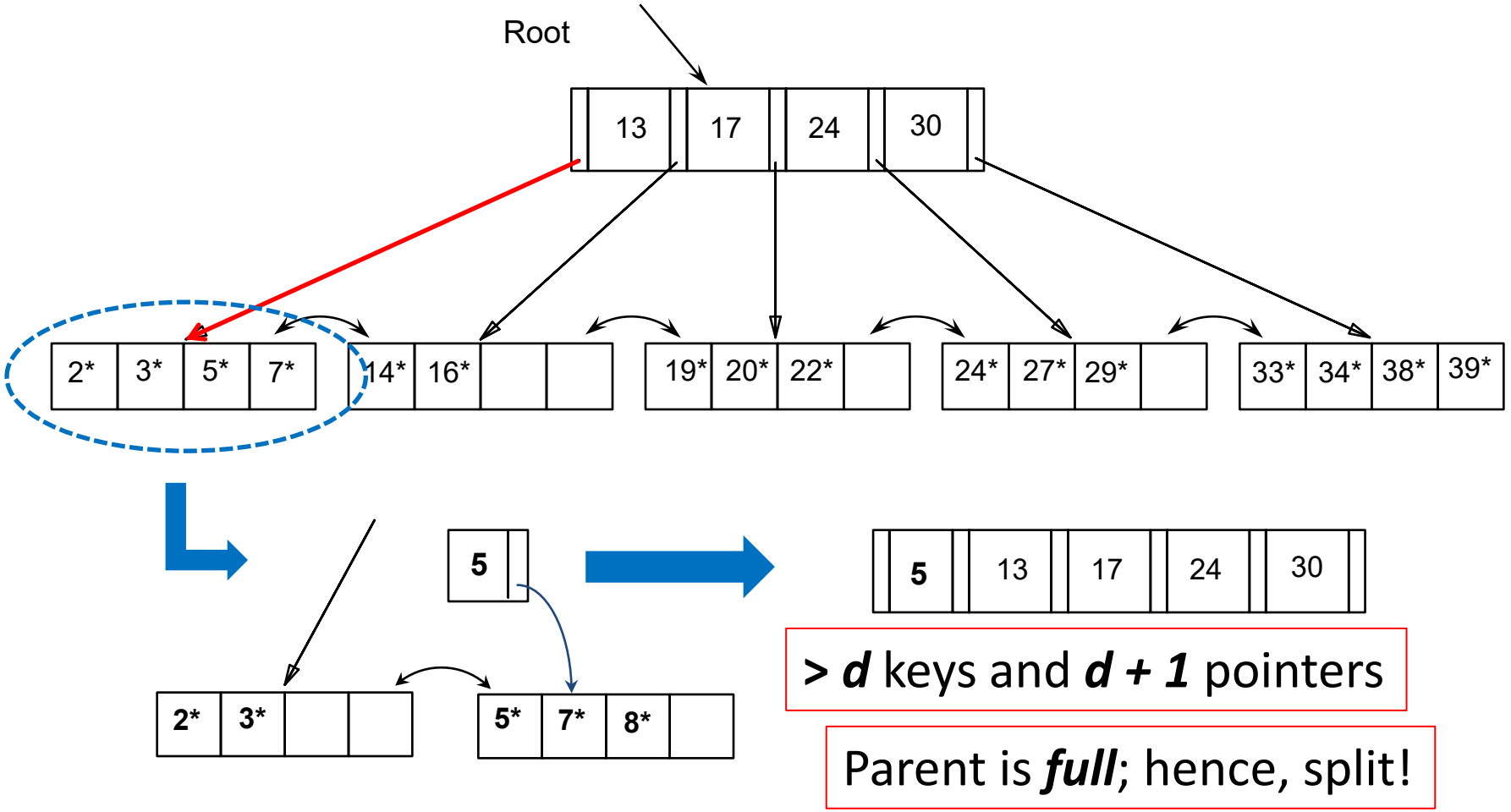


The middle key (i.e., **5**) is "copied up" and continues to appear in the leaf

B+ Tree: Examples of Insertions

- Insert entry 8*

Set the order ***d=4;*** *Max has 4 keys.*

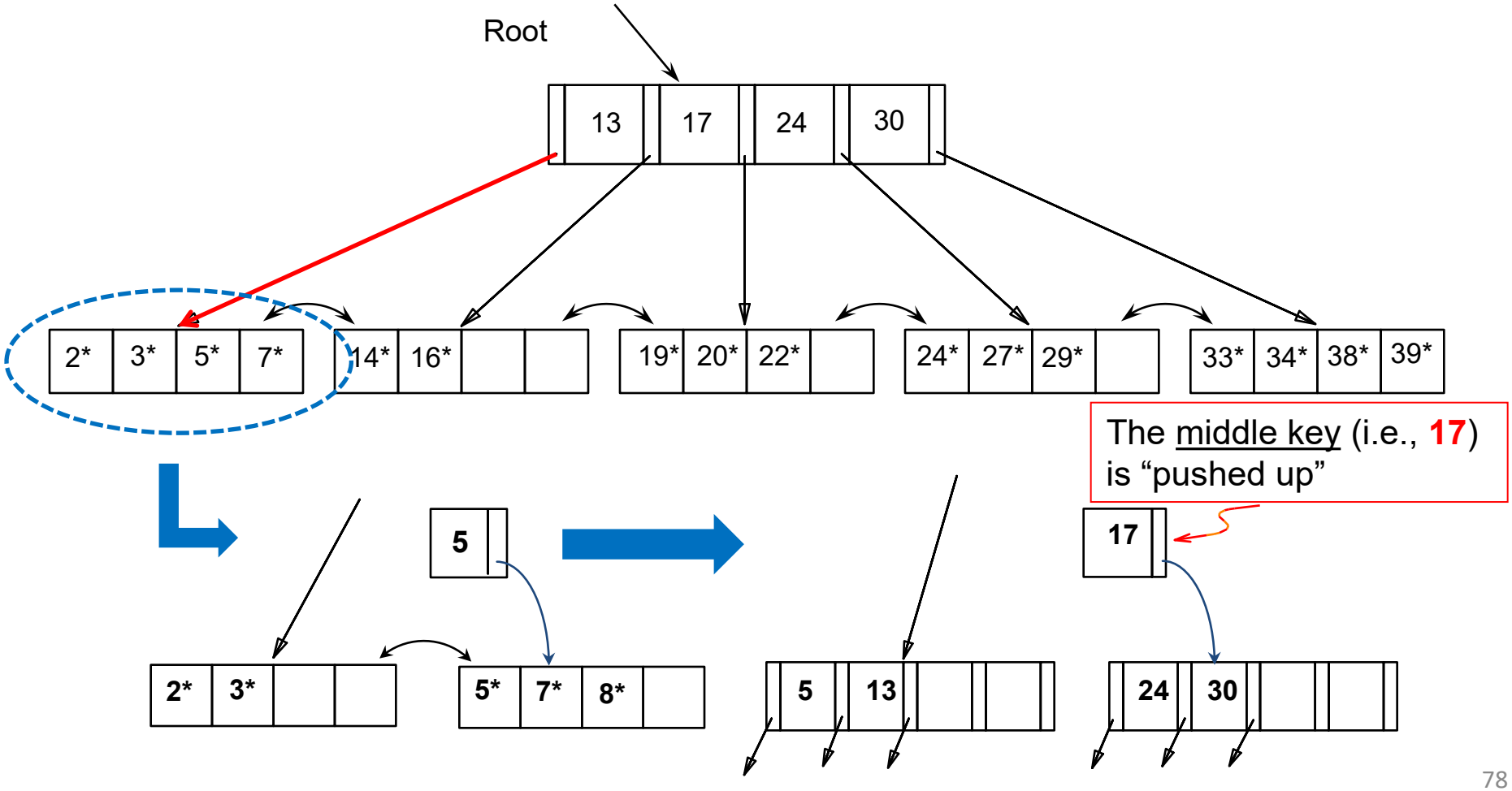


> ***d*** keys and ***d + 1*** pointers

Parent is ***full***; hence, split!

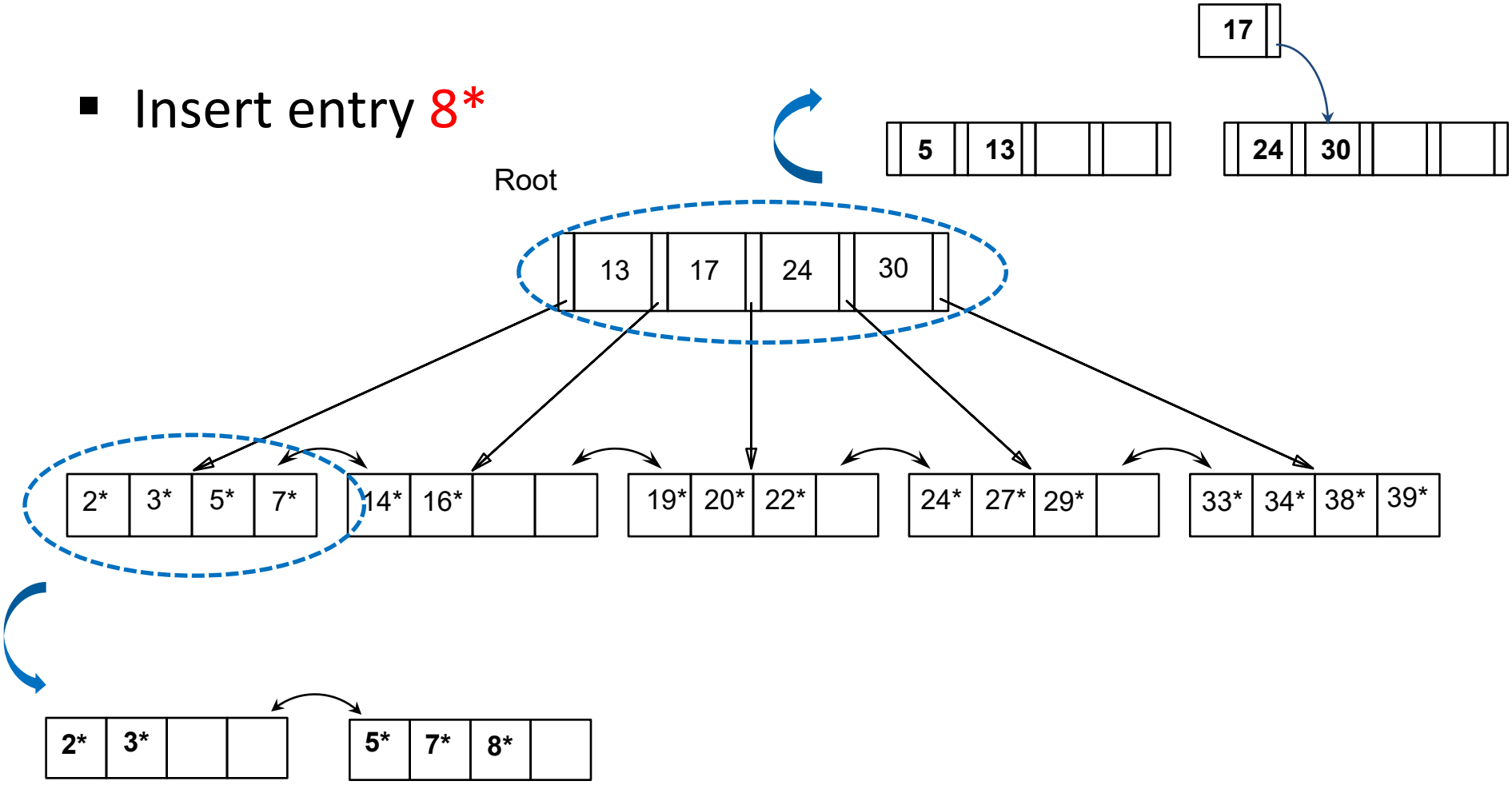
B+ Tree: Examples of Insertions

- Insert entry 8*



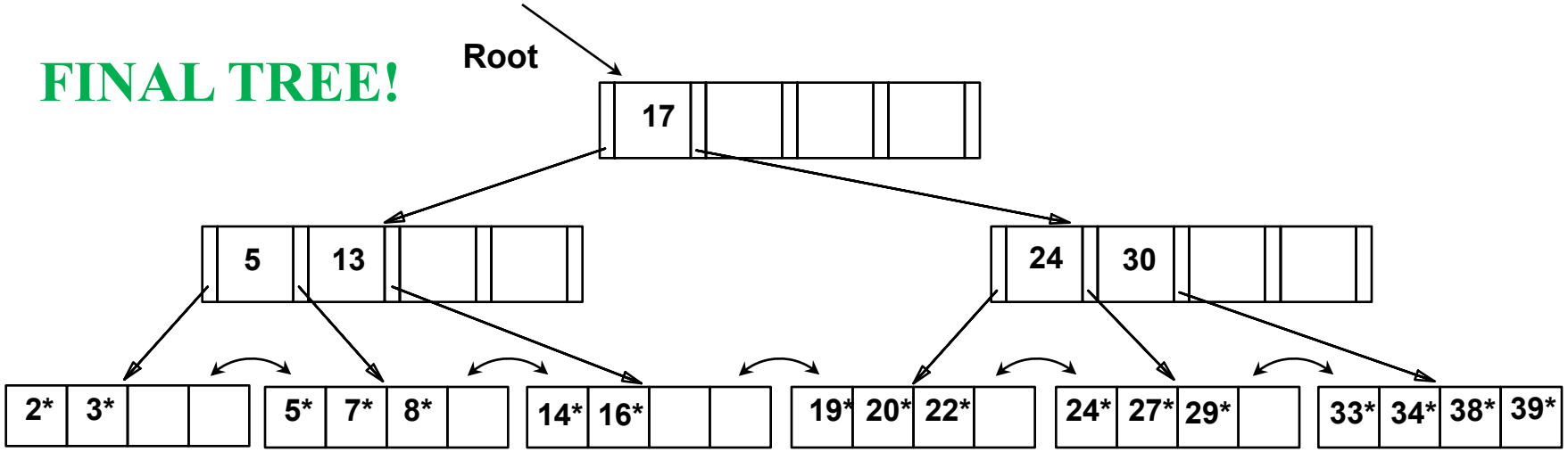
B+ Tree: Examples of Insertions

- Insert entry 8*



B+ Tree: Examples of Insertions

- Insert entry 8*

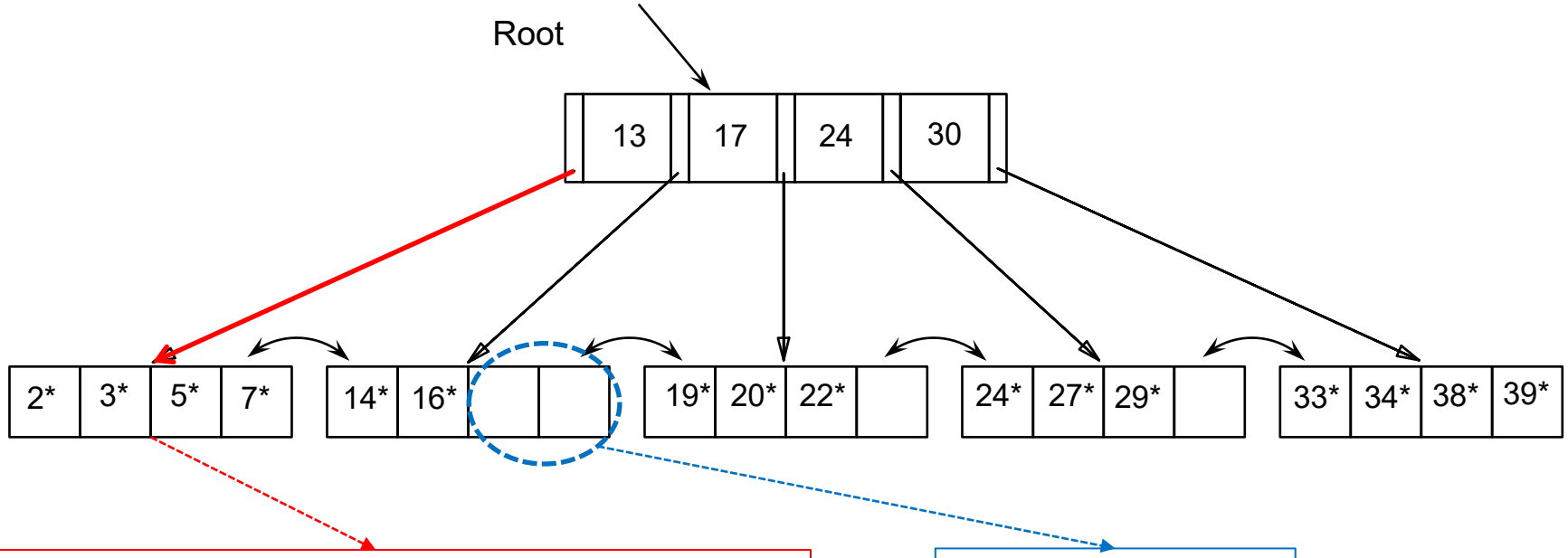


Splitting the root leads to an increase of height by 1!

What about *re-distributing* entries instead of *splitting* nodes?

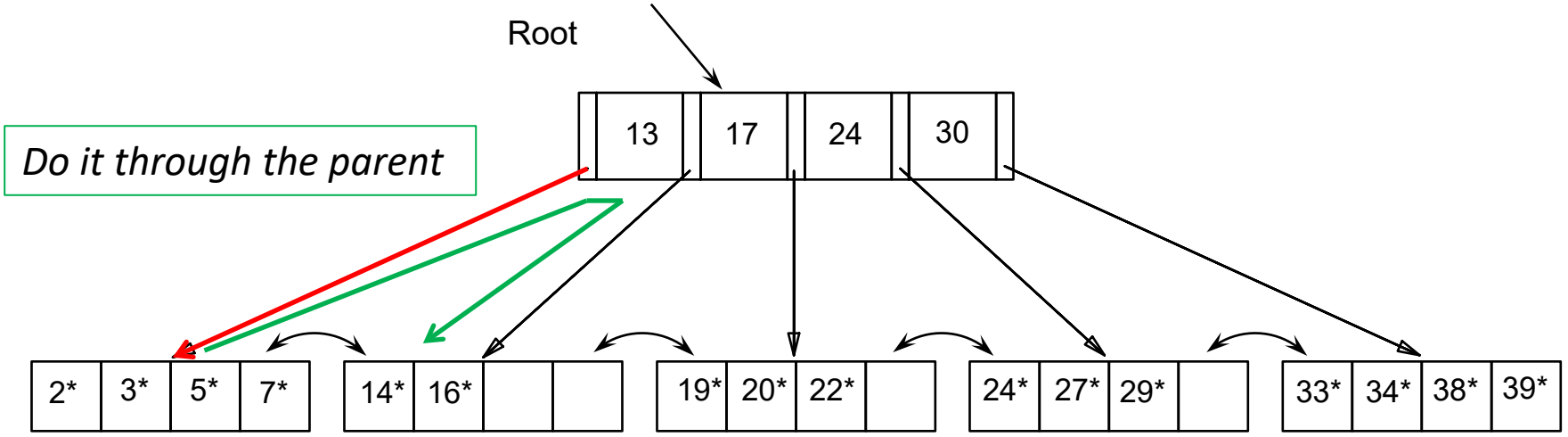
B+ Tree: Examples of Insertions

- Insert entry 8*



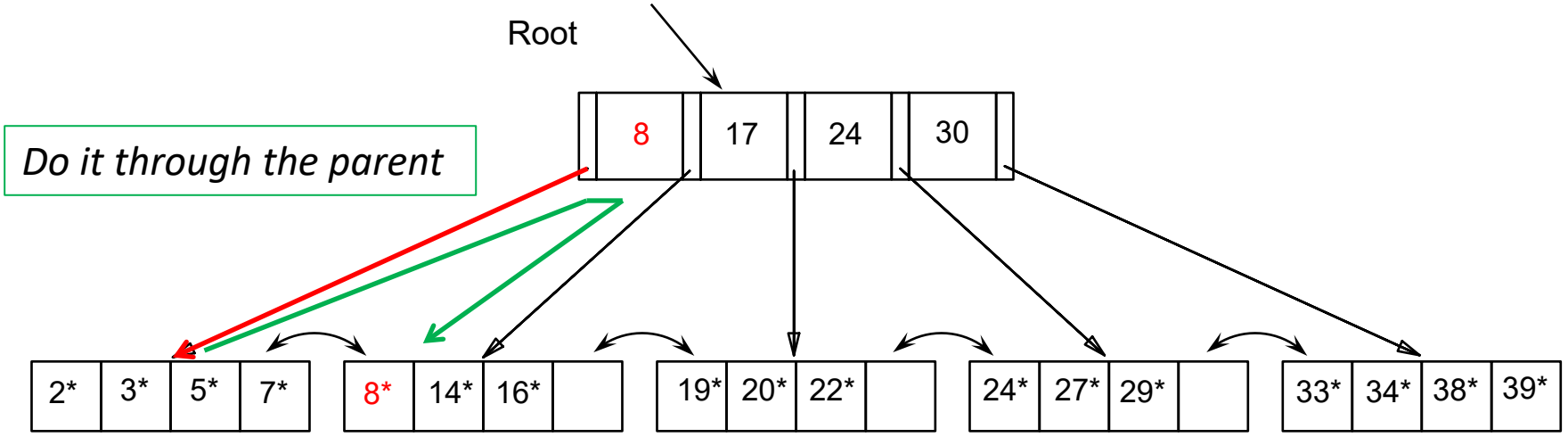
B+ Tree: Examples of Insertions

- Insert entry 8*



B+ Tree: Examples of Insertions

- Insert entry 8*



"Copy up" the new low key value!

But, when to *redistribute* and when to *split*?

Splitting vs. Redistributing

- Leaf Nodes
  - Previous and next-neighbor pointers must be updated upon insertions (*if splitting is to be pursued*)
  - Hence, checking whether redistribution is possible does not increase I/O
  - Therefore, if a sibling can spare an entry, re-distribute
- Non-Leaf Nodes
  - Checking whether redistribution is possible *usually* increases I/O
  - Splitting non-leaf nodes typically pays off!

B+ Insertions: Keep in Mind

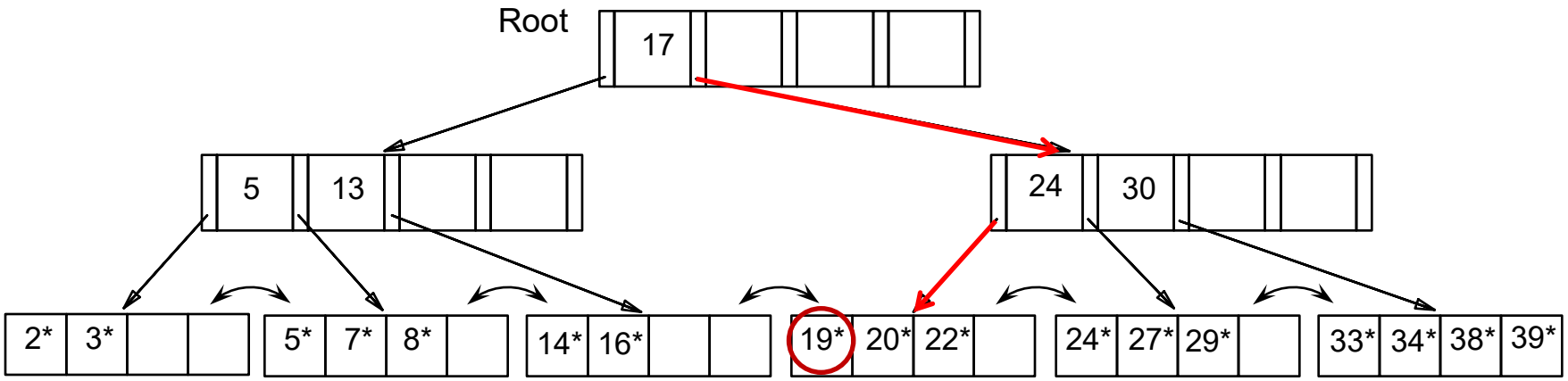
- Every data entry must appear in a leaf node; hence, “copy up” the middle key upon splitting
- When splitting index entries, simply “push up” the middle key
- Apply splitting and/or redistribution on leaf nodes
- Apply only splitting on non-leaf nodes

B+ Trees: Deleting Entries

- Start at root, find leaf ***L*** where entry belongs
- Remove the entry
  - If ***L*** is at least half-full, *done!*
  - If ***L*** *underflows*
    - Try to re-distribute (i.e., borrow from a "rich sibling" and "copy up" its *lowest key*)
    - If re-distribution fails, *merge* ***L*** and a "poor sibling"
      - Update parent
      - And possibly merge, recursively

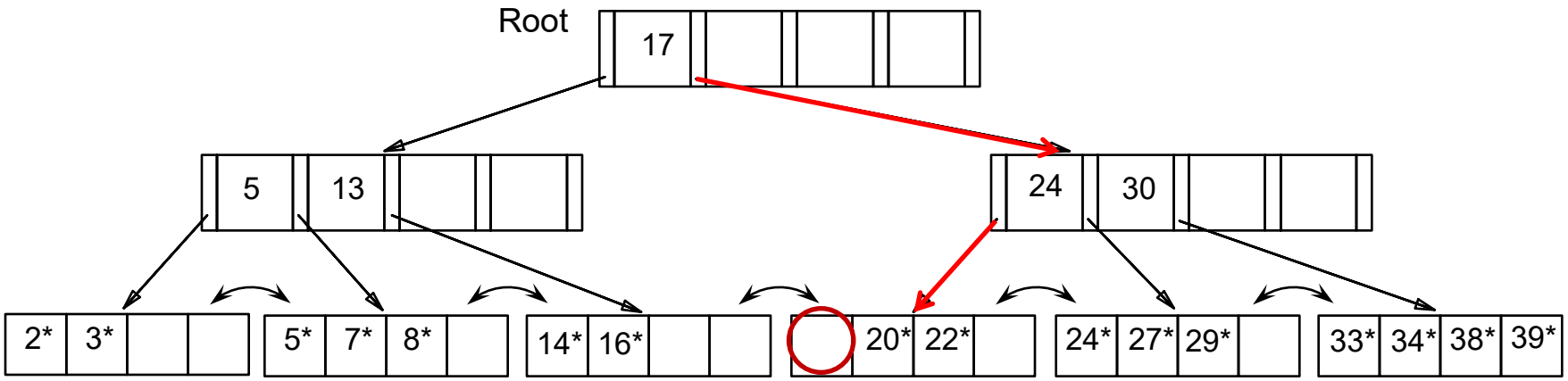
B+ Tree: Examples of Deletions

- Delete 19*



B+ Tree: Examples of Deletions

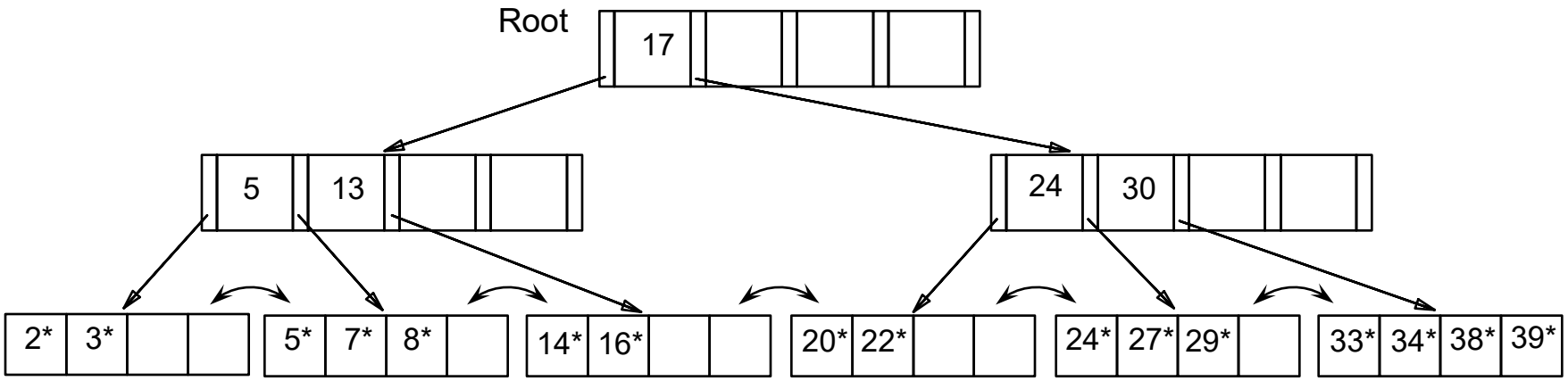
- Delete 19*



Removing 19* does not cause an underflow 下溢

B+ Tree: Examples of Deletions

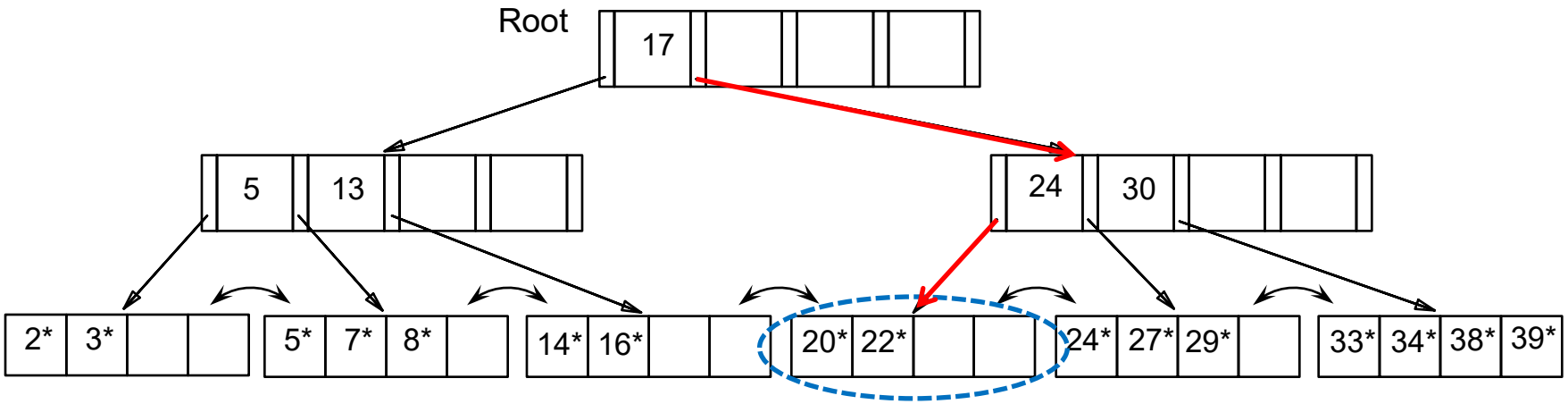
- Delete 19*



FINAL TREE!

B+ Tree: Examples of Deletions

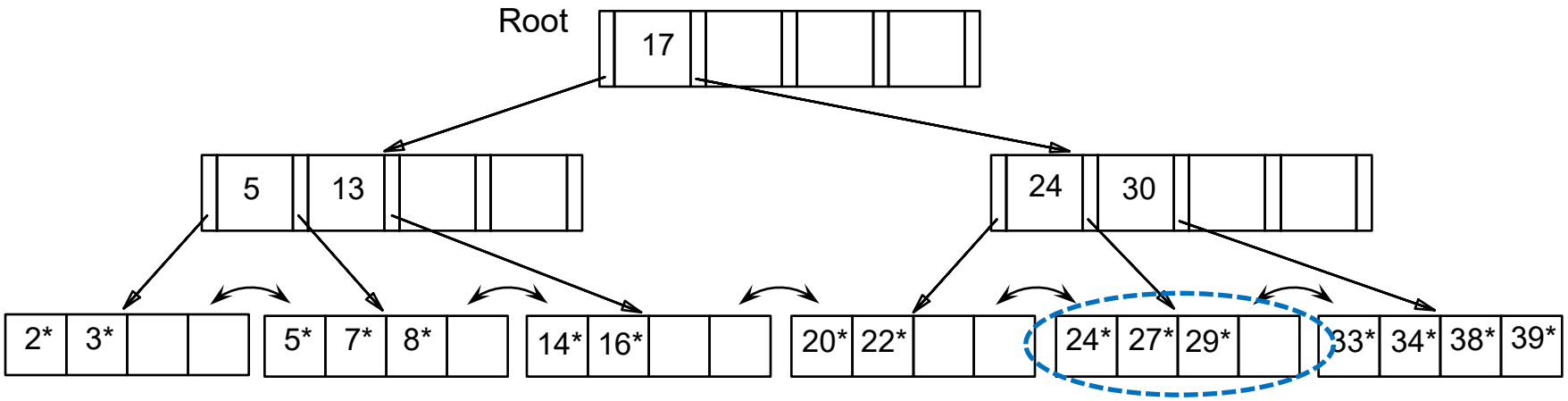
- Delete 20*



Deleting 20* causes an underflow; hence, check a sibling for redistribution

B+ Tree: Examples of Deletions

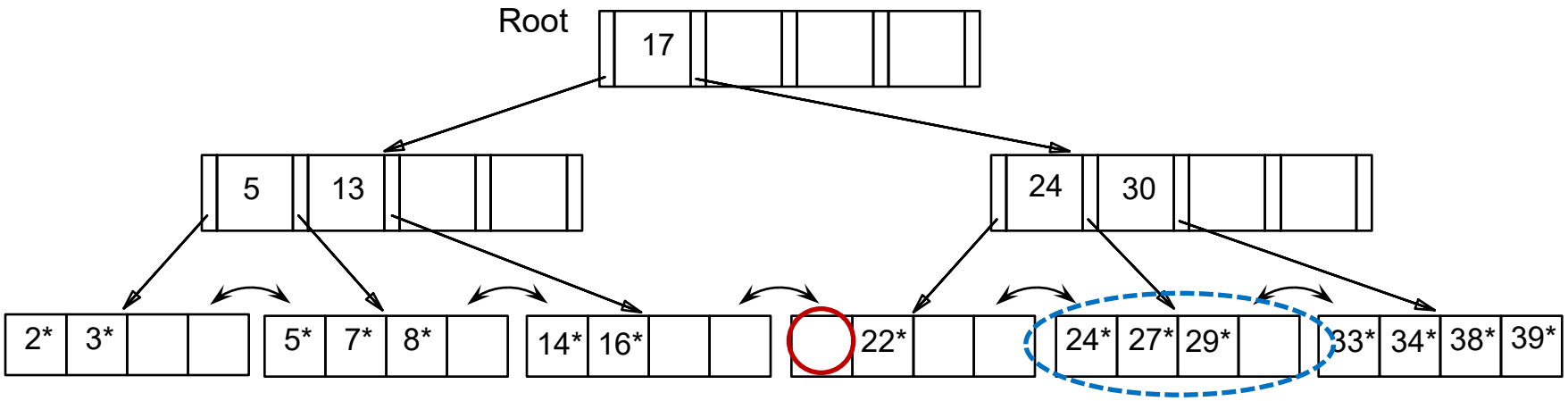
- Delete 20*



The sibling is 'rich' (i.e., can lend an entry); hence, remove 20* and redistribute!

B+ Tree: Examples of Deletions

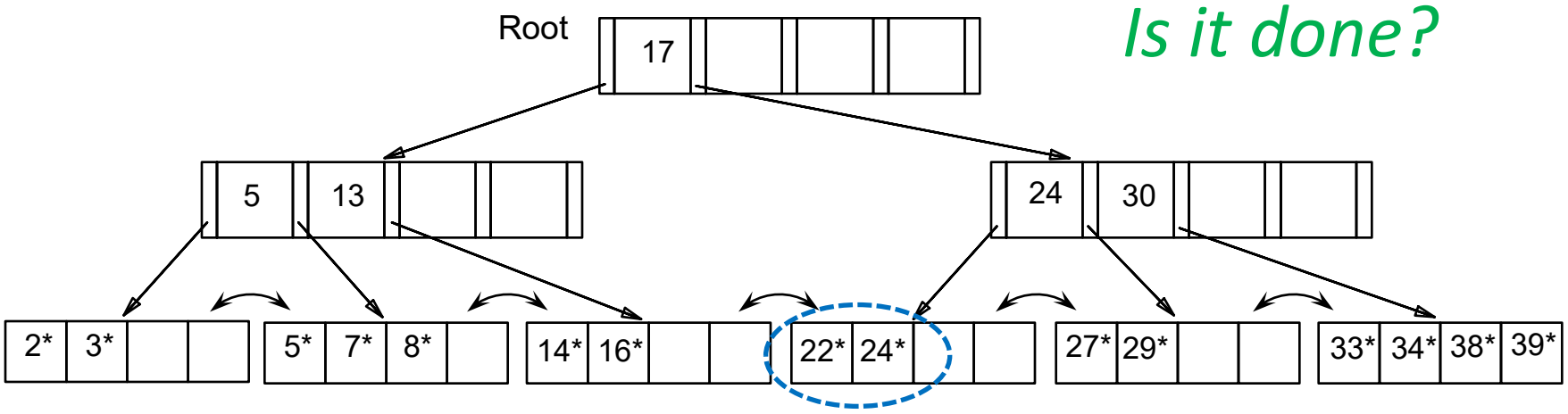
- Delete 20*



The sibling is 'rich' (i.e., can lend an entry); hence, remove 20* and redistribute!

B+ Tree: Examples of Deletions

- Delete 20*

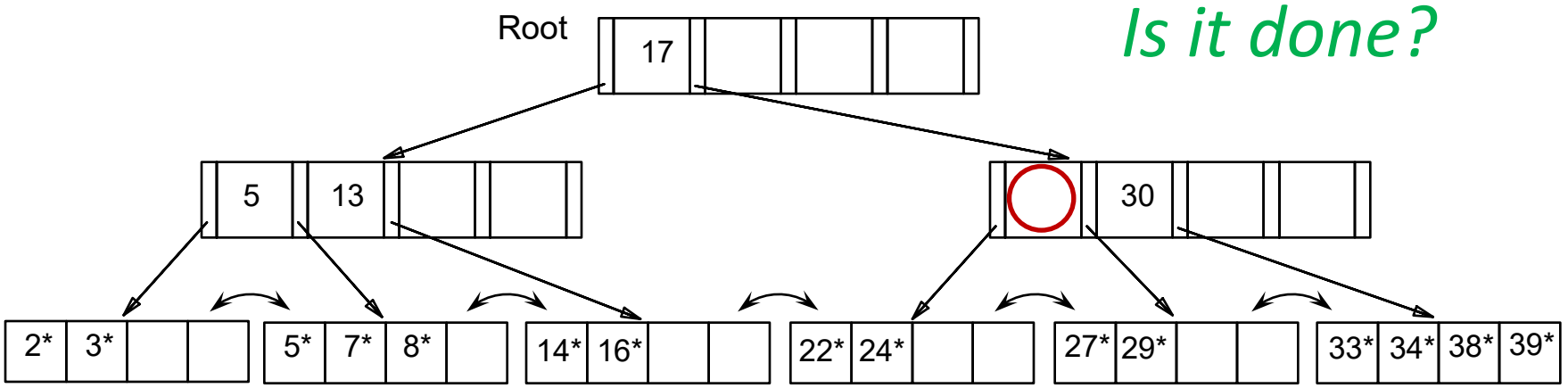


Is it done?

"Copy up" 27*, the lowest value in the leaf from which we borrowed 24*

B+ Tree: Examples of Deletions

- Delete 20*

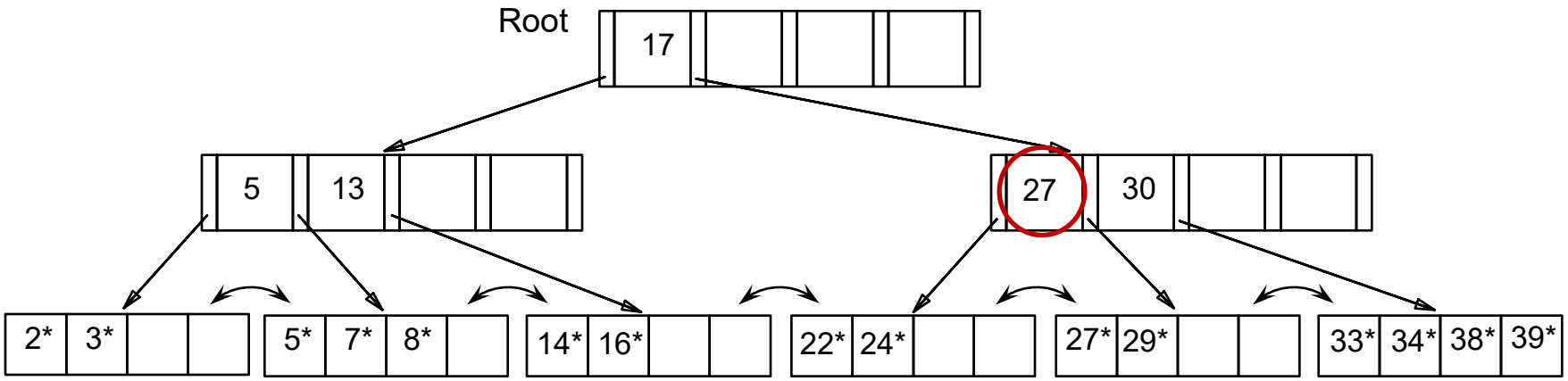


Is it done?

"Copy up" 27*, the lowest value in the leaf from which we borrowed 24*

B+ Tree: Examples of Deletions

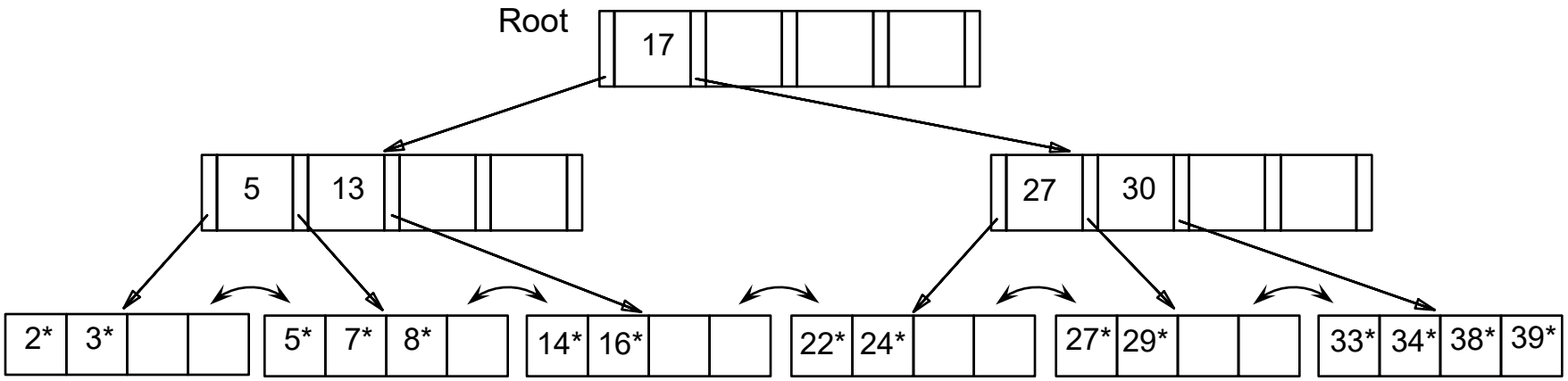
- Delete 20*



"Copy up" 27*, the lowest value in the leaf from which we borrowed 24*

B+ Tree: Examples of Deletions

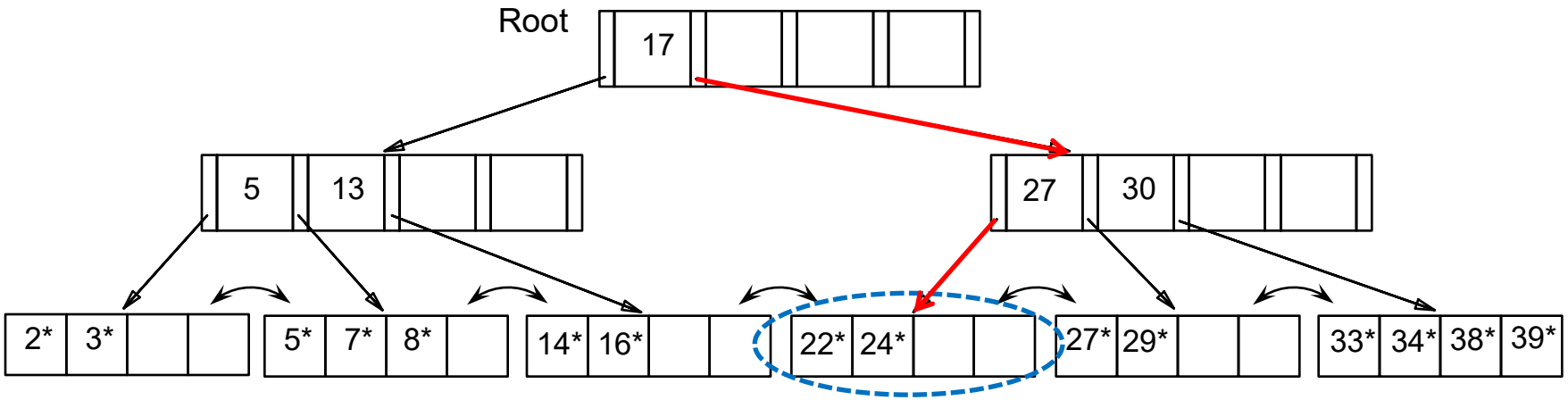
- Delete 20*



FINAL TREE!

B+ Tree: Examples of Deletions

- Delete 24*

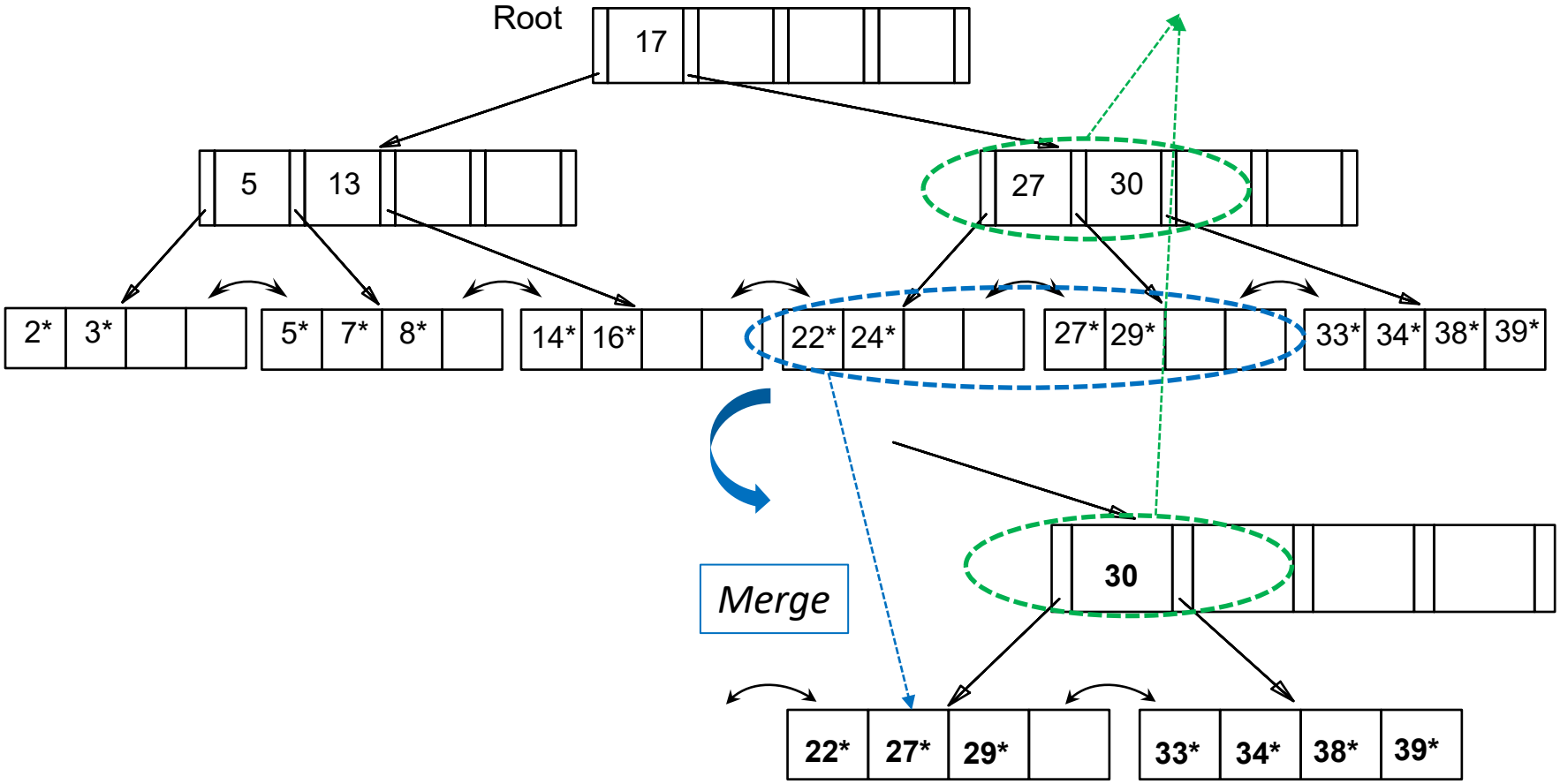


The affected leaf will contain only 1 entry and the sibling cannot lend any entry (i.e., redistribution is not applicable); hence, *merge*!

B+ Tree: Examples of Deletions

- Delete 24*

"Toss" 27 because the page that it was pointing to does not exist anymore!

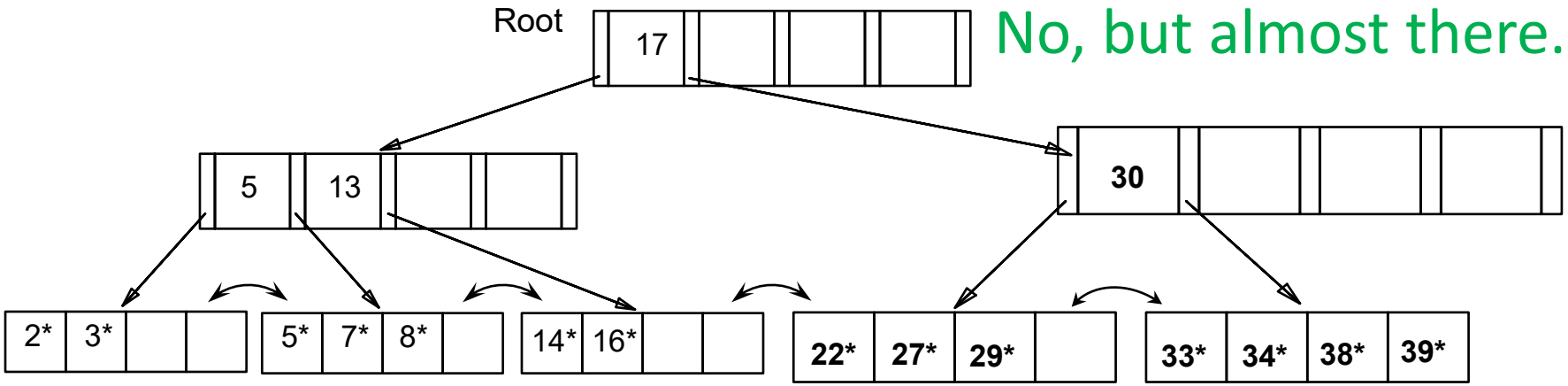


B+ Tree: Examples of Deletions

- Delete 24*

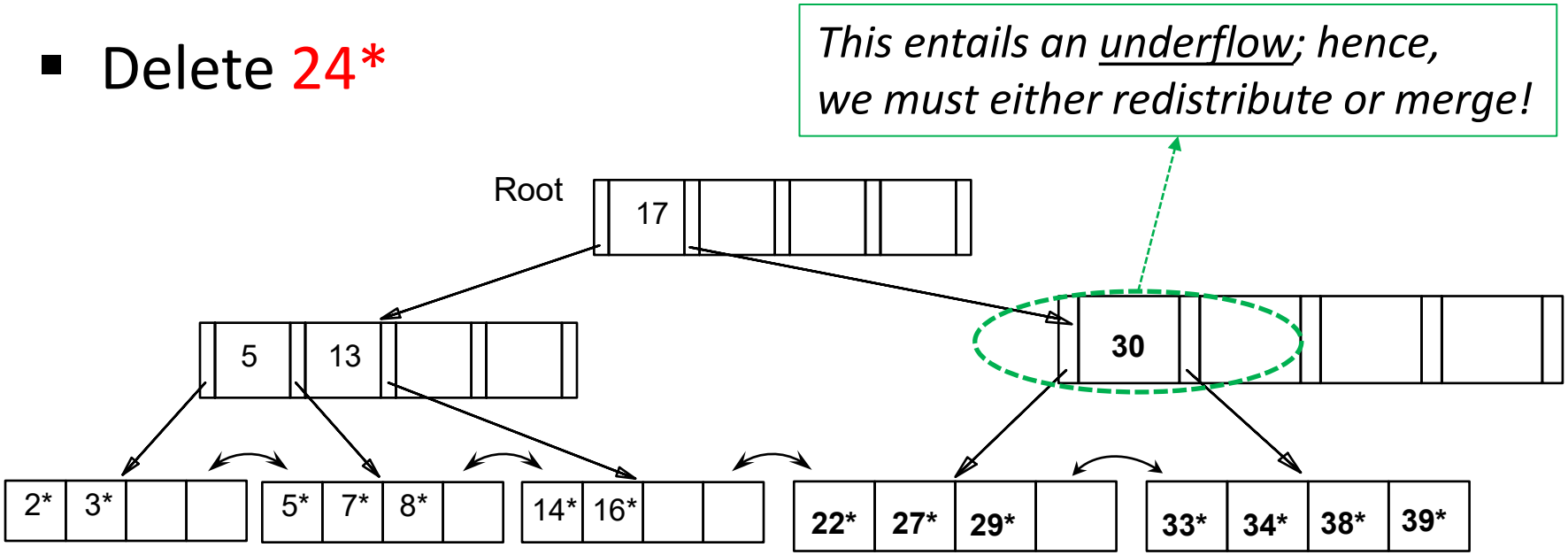
Is it done?

No, but almost there…



B+ Tree: Examples of Deletions

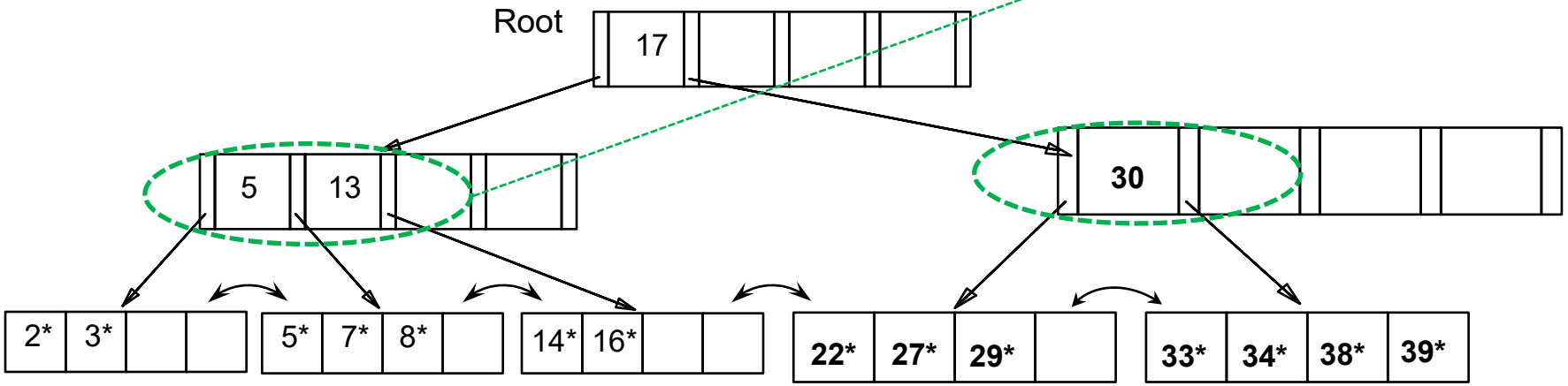
- Delete 24*



B+ Tree: Examples of Deletions

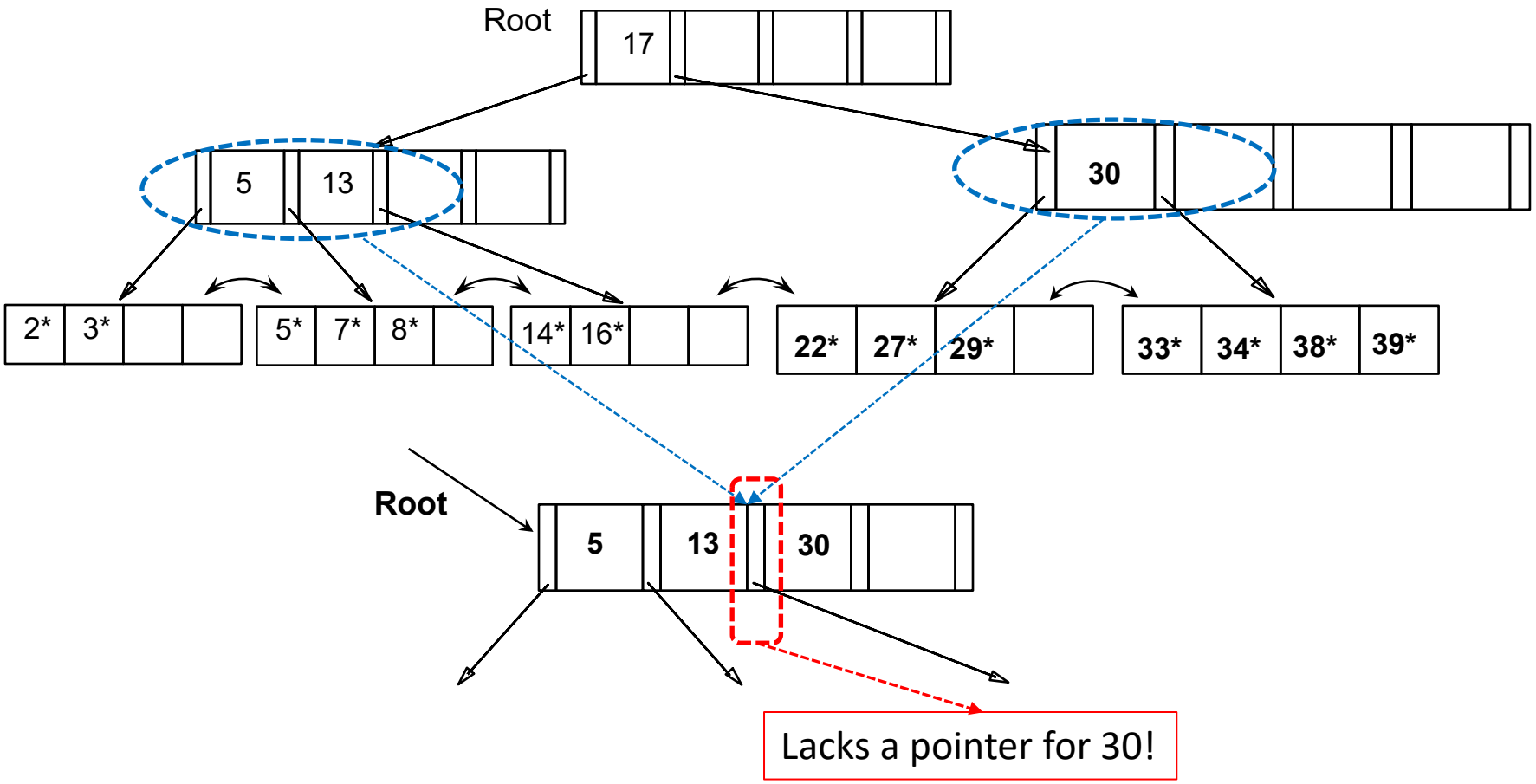
- Delete 24*

The sibling is "poor" (i.e., redistribution is not applicable)*; hence, merge!*



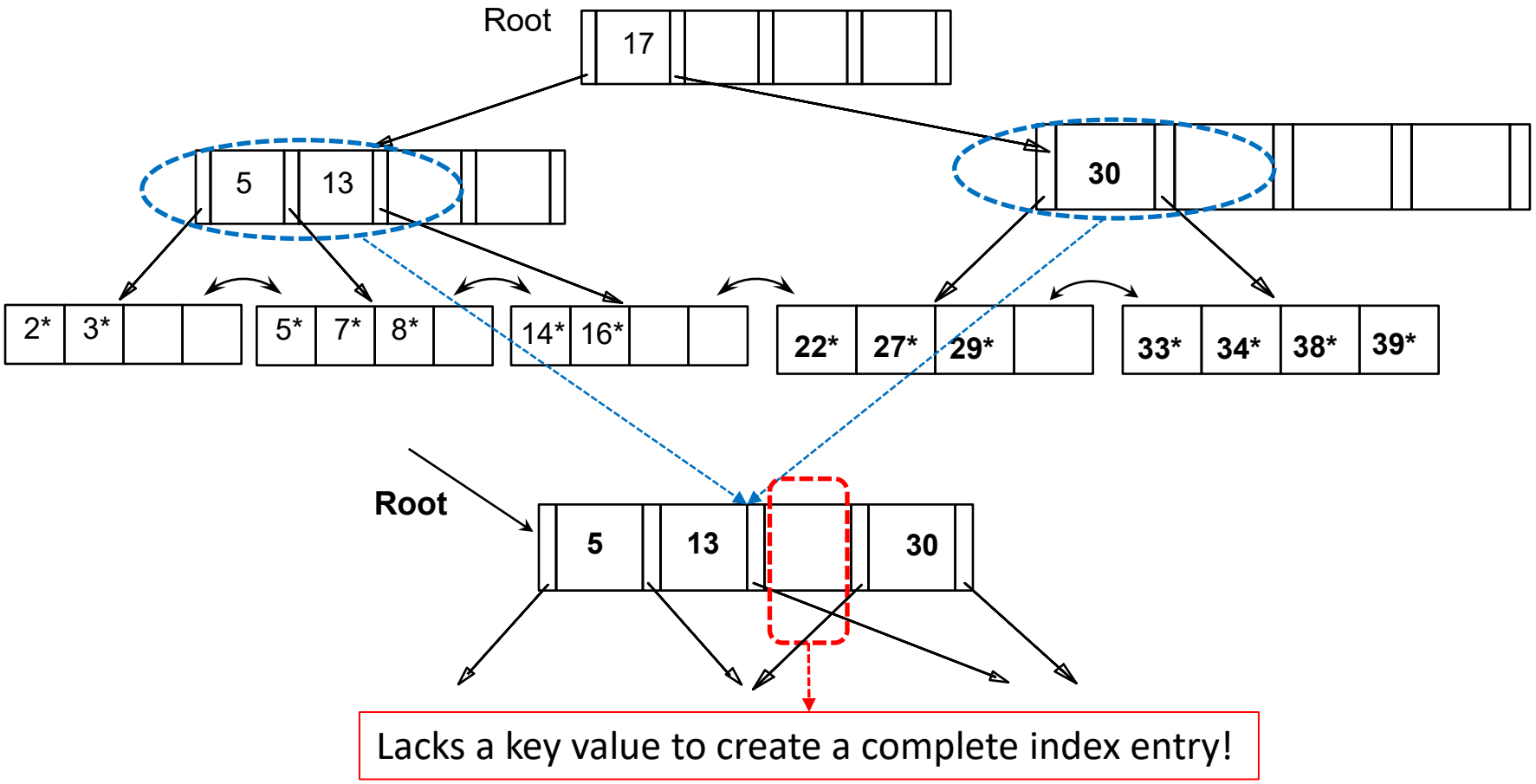
B+ Tree: Examples of Deletions

- Delete 24*



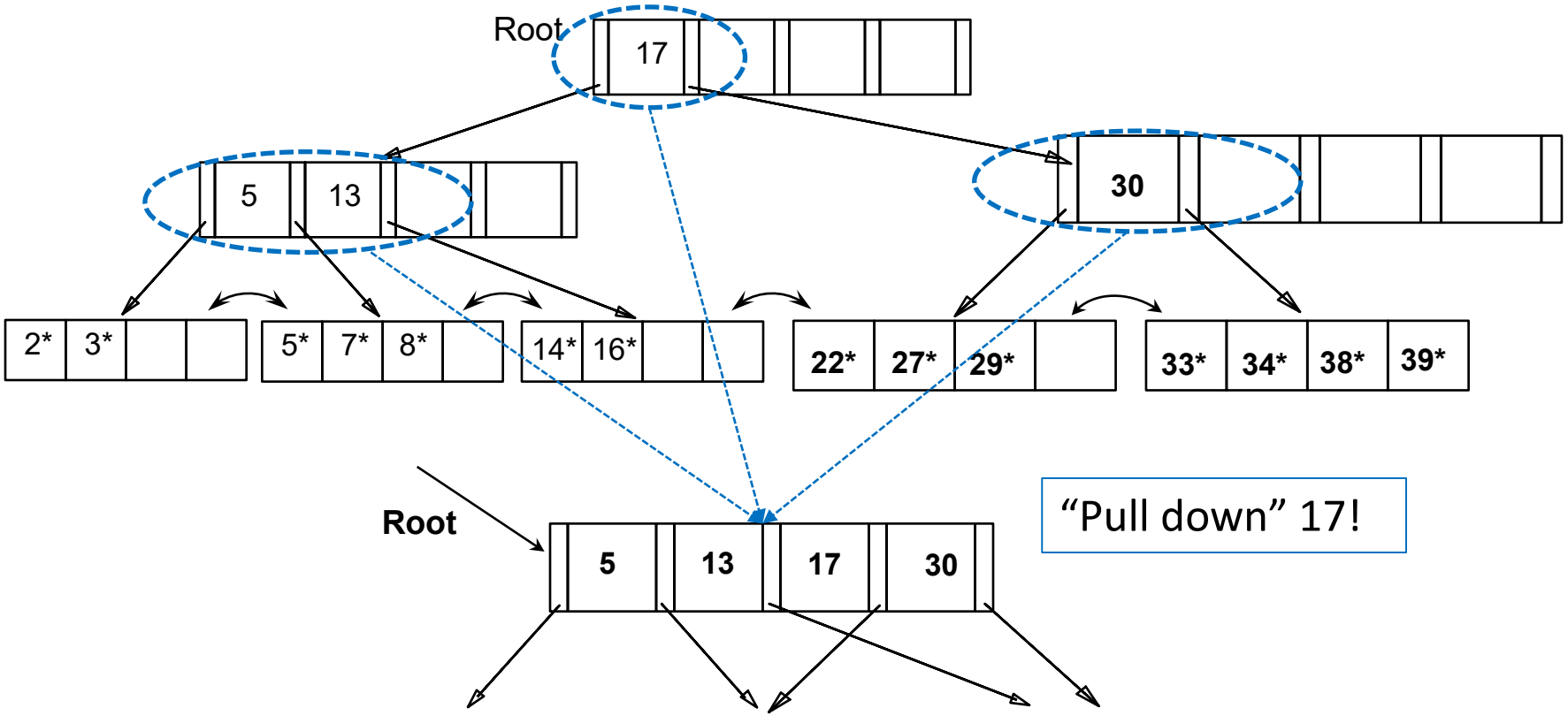
B+ Tree: Examples of Deletions

- Delete 24*



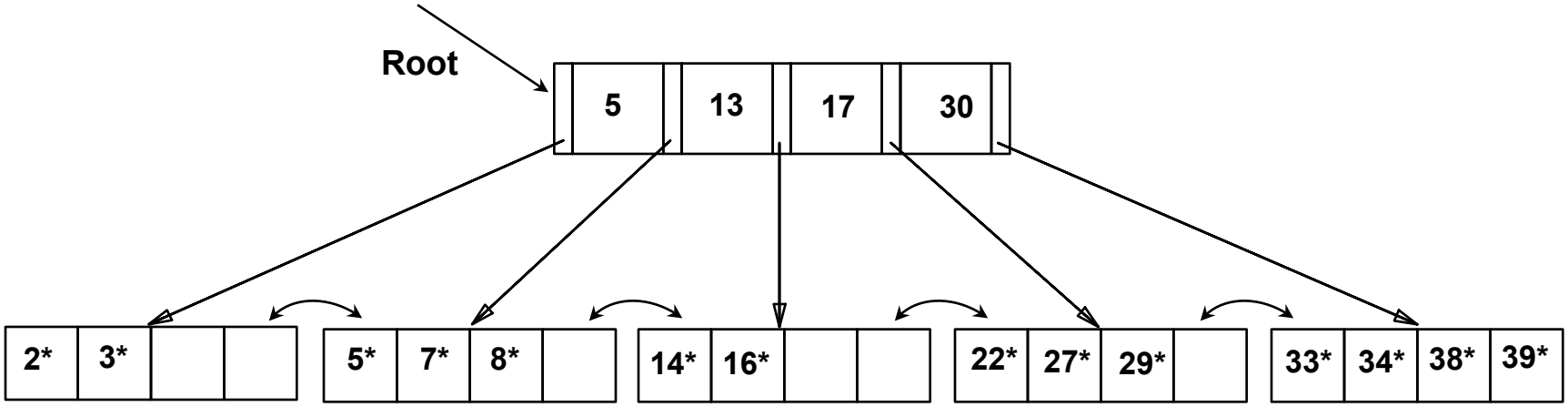
B+ Tree: Examples of Deletions

- Delete 24*



B+ Tree: Examples of Deletions

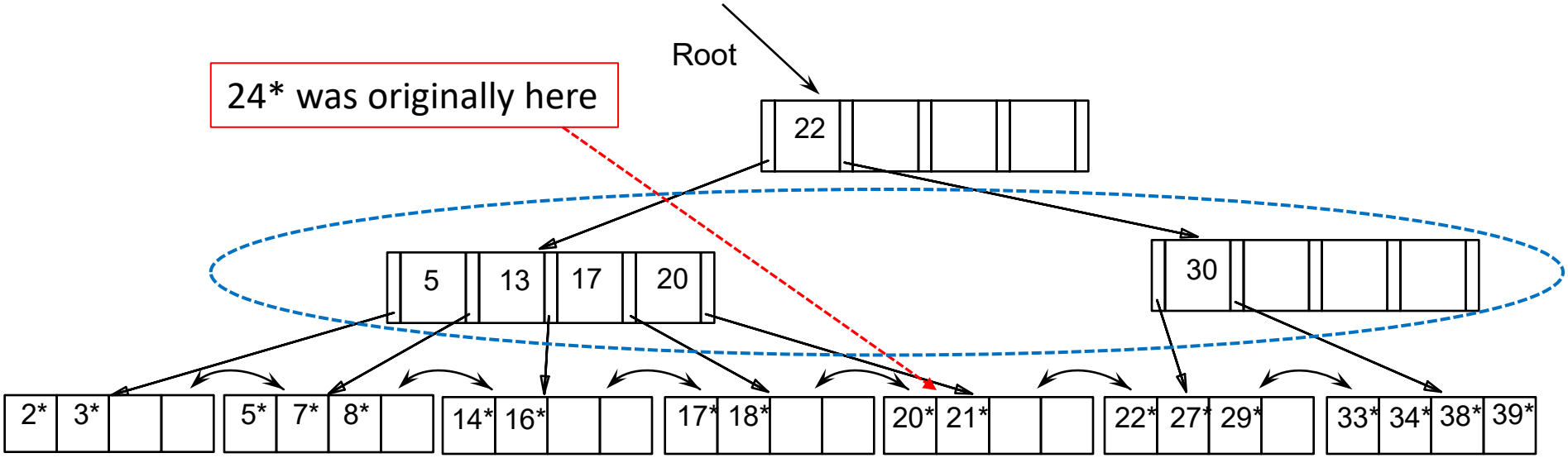
- Delete 24*



FINAL TREE!

B+ Tree: Examples of Deletions

- Delete 24*

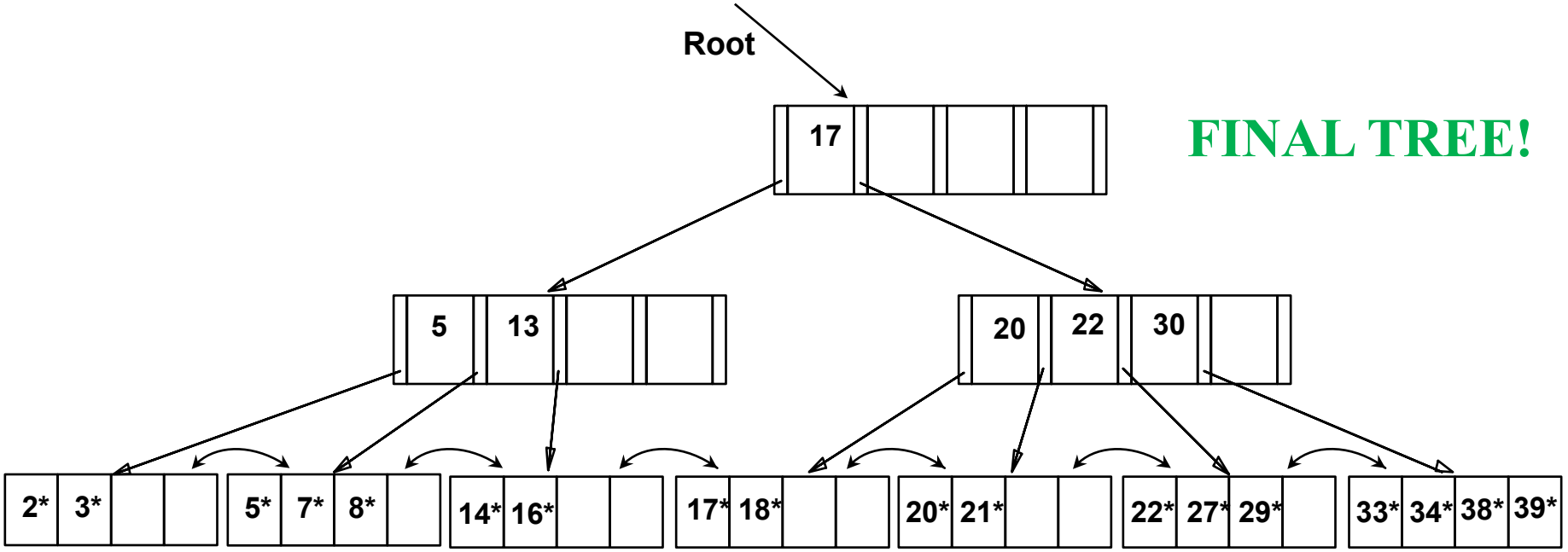


Assume (instead) the above tree *during* deleting 24*

Now we can re-distribute (*instead of merging*) keys!

B+ Tree: Examples of Deletions

- Delete 24*



FINAL TREE!

DONE! It suffices to re-distribute only 20; 17 was redistributed for illustration.

Thank you!

Thank you!